UNIVERSITAS GUNADARMA

FAKULTAS ILMU KOMPUTER DAN TEKNOLOGI INFORMASI

KLASIFIKASI TOPIK KONTEN *PODCAST* BERBAHASA INDONESIA MENGGUNAKAN *SPEECH-TO-TEXT* DAN *NATURAL LANGUAGE PROCESSING* BERBASIS WEB

Dsusun Oleh:

Nama : Arbet Thetus Sinaga
NPM : 10122204
Program Studi : Sistem Informasi
Pembimbing : Dr. Sulistyo Puspitodjati, SSi., MSc

Diajukan Guna Melengkapi Sebagian Syarat
Dalam Mencapai Gelar Sarjana Strata Satu (S1)

JAKARTA
2026

PERNYATAAN ORISINALITAS DAN PUBLIKASI

Saya yang bertanda tangan dibawah ini,

Nama	:	Arbet Thetus Sinaga
NPM	:	10122204
Judul Skripsi	:	KLASIFIKASI TOPIK KONTEN *PODCAST* BERBAHASA INDONESIA MENGGUNAKAN *SPEECH-TO-TEXT* DAN *NATURAL LANGUAGE PROCESSING* BERBASIS WEB
Tanggal Sidang	:	31 – Agustus – 2026
Tanggal Lulus	:	31 – Agustus – 2026

Menyatakan bahwa tulisan ini adalah merupakan hasil karya saya sendiri dan dapat dipublikasikan sepenuhnya oleh Universitas Gunadarma. Segala kutipan dalam bentuk apa pun telah mengikuti kaidah, etika yang berlaku. Mengenai isi dan tulisan adalah merupakan tanggung jawab Penulis, bukan Universitas Gunadarma.
Demikian pernyataan ini dibuat dengan sebenarnya dan dengan penuh kesadaran.

Jakarta, 2026



Arbet Thetus Sinaga

LEMBARAN PENGESAHAN

Judul Skripsi	:	KLASIFIKASI TOPIK KONTEN *PODCAST* BERBAHASA INDONESIA MENGGUNAKAN *SPEECH-TO-TEXT* DAN *NATURAL LANGUAGE PROCESSING* BERBASIS WEB
Nama Mahasiswa	:	Arbet Thetus Sinaga
NPM	:	10122204
Program Studi	:	Sistem Informasi
Tanggal Lulus	:	31 – Agustus – 2026

PANITIA UJIAN

NO	NAMA	KEDUDUKAN
1.		Ketua
2.		Sekretaris
3.		Anggota
4.		Anggota
5.		Anggota

Menyetujui

Pembimbing **Bagian Sidang Ujian**

(Sulistyo Puspitodjati, Dr., SSi., MSc) (Dr. Edi Sukirman, S.Si., MM., M.I.KOM)

ABSTRAK

Arbet Thetus Sinaga, 10122204

KLASIFIKASI TOPIK KONTEN *PODCAST* BERBAHASA INDONESIA MENGGUNAKAN *SPEECH-TO-TEXT* DAN *NATURAL LANGUAGE PROCESSING* BERBASIS WEB

Skripsi, Jurusan Sistem Informasi, Fakultas Ilmu Komputer dan Teknologi Informasi, Universitas Gunadarma, 2026

Kata Kunci: *podcast, Speech-to-Text, whisper, Natural Language Processing, TF-IDF, Support Vector Machine, naive bayes,* klasifikasi topik*, flask*

(xvii + 88 + Lampiran*)*

Podcast telah menjadi salah satu media digital yang paling diminati masyarakat Indonesia, namun identifikasi topik *podcast* yang hanya mengandalkan metadata seperti judul dan deskripsi belum mampu merepresentasikan keseluruhan isi pembahasan secara akurat. Penelitian ini bertujuan membangun sistem klasifikasi topik konten *podcast* berbahasa Indonesia secara otomatis berbasis web dengan memanfaatkan teknologi *Speech-to-Text* dan *Natural Language Processing*. Metode penelitian mengacu pada kerangka kerja CRISP-DM, dimulai dari transkripsi audio menggunakan model Whisper Medium, dilanjutkan dengan tahap *Preprocessing* teks (*case folding*, *text cleaning*, *tokenizing*, *stopword removal*, dan *Stemming* menggunakan PySastrawi), ekstraksi fitur menggunakan *Term Frequency-Inverse Document Frequency* (TF-IDF), serta klasifikasi menggunakan algoritma *Naive Bayes* dan *Support Vector Machine* (SVM) *Linear*. *Dataset* penelitian terdiri atas 1.120 berkas audio *podcast* dari tujuh kategori topik (Hiburan, Olahraga, Pendidikan, Teknologi, Bisnis, Kesehatan, dan Berita), yang setelah tahap *Preprocessing* menghasilkan 1.115 sampel dan dibagi menjadi 892 data latih serta 223 data uji. Hasil evaluasi menunjukkan bahwa model SVM Linear memberikan kinerja lebih baik dibandingkan Naive Bayes, dengan nilai *Accuracy* sebesar 0,7713, *Precision* sebesar 0,7733, Recall sebesar 0,7713, dan *F1-Score* sebesar 0,7714. Model terbaik tersebut selanjutnya diintegrasikan ke dalam aplikasi berbasis web menggunakan Flask, yang memungkinkan pengguna mengunggah berkas audio *podcast* dan memperoleh hasil klasifikasi topik secara otomatis. Hasil pengujian *Black Box Testing* menunjukkan bahwa seluruh fungsi utama aplikasi berjalan sesuai dengan rancangan.
Daftar Pustaka (2020-2025)

ABSTRACT

Arbet Thetus Sinaga, 10122204

WEB-BASED CLASSIFICATION OF INDONESIAN-LANGUAGE PODCAST CONTENT TOPICS USING SPEECH-TO-TEXT AND NATURAL LANGUAGE PROCESSING

Undergraduate Thesis, Department of Information Systems, Faculty of Computer Science and Information Technology, Gunadarma University, 2026

Keywords: podcast, Speech-to-Text, whisper, *Natural Language Processing*, TF-IDF, *Support Vector Machine*, naive bayes, topic classification, flask

(xvii + 88 + Appendix)

Podcasts have become one of the most popular digital media among Indonesian audiences, yet topic identification that relies solely on metadata such as titles and descriptions often fails to accurately represent the full content of an episode. This study aims to develop an automatic, web-based topic classification system for Indonesian-language podcasts by leveraging Speech-to-Text and Natural Language Processing technologies. The research follows the CRISP-DM mobile
, beginning with audio transcription using the Whisper Medium model, followed by text Preprocessing (case folding, text cleaning, tokenizing, stopword removal, and Stemming using PySastrawi), feature extraction using Term Frequency-Inverse Document Frequency (TF-IDF), and classification using the Naive Bayes and Support Vector Machine (SVM) Linear algorithms. The dataset consisted of 1,120 podcast audio files across seven topic categories (Entertainment, Sports, Education, Technology, Business, Health, and News), which, after Preprocessing, resulted in 1,115 samples split into 892 training samples and 223 testing samples. The evaluation results show that the SVM Linear model outperformed Naive Bayes, achieving an Accuracy of 0.7713, Precision of 0.7733, Recall of 0.7713, and F1-Score of 0.7714. The best-performing model was then integrated into a Flask-based web application, allowing users to upload podcast audio files and automatically obtain topic classification results. Black Box Testing results confirmed that all core application functions operated as designed.
References (2020–2025)

KATA PENGANTAR

Segala puji dan syukur penulis panjatkan ke Tuhan Yang Maha Kuasa yang telah memberikan berkah, anugerah, dan karunia yang melimpah, sehingga penulis dapat menyelesaikan tugas akhir ini.

Tugas akhir ini disusun guna melengkapi sebagian syarat dalam mencapai gelar Sarjana Satu pada jurusan Sistem Informasi, Fakultas Ilmu Komputer dan Teknologi Informasi, Universitas Gunadarma. Adapun judul skripsi ini adalah "KLASIFIKASI TOPIK KONTEN *PODCAST* BERBAHASA INDONESIA MENGGUNAKAN *SPEECH-TO-TEXT* DAN *NATURAL LANGUAGE PROCESSING* BERBASIS WEB".

Walaupun banyak kesulitan yang penulis harus hadapi ketika menyusun tugas akhir ini, namun berkat bantuan dan dorongan dari berbagai pihak akhirnya tugas ini dapat diselesaikan dengan baik. Untuk itu penulis mengucapkan banyak terima kasih kepada:

1. Prof. Dr. Hj. E. S. Margianti, SE, MM., selaku Rektor Universitas Gunadarma.
2. Prof. Dr. Tubagus Kusuma, SKom., MEngsc., selaku Dekan Fakultas Ilmu Komputer dan Teknologi Informasi Universitas Gunadarma.
3. Dr. Naeli Umniati, ST., MMSI., selaku Ketua Program Studi Sistem Informasi Fakultas Ilmu Komputer dan Teknologi Informasi Universitas Gunadarma.
4. Dr. Edi Sukirman, S.Si., MM., M.I.Kom., selaku Kepala Bagian Sidang Ujian Universitas Gunadarma.
5. Dr. Sulistyo Puspitodjati, SSi., MSc selaku Dosen Pembimbing yang telah banyak membantu, memberikan bimbingan serta masukan dan mendampingi selama penyusunan tugas akhir ini dengan sabar sampai selesai.

6. Bapak dan Ibu Dosen Universitas Gunadarma yang telah memberikan banyak ilmu pengetahuan kepada penulis.
7. Kedua orang tua penulis, yaitu Bapak Hetdi dan Ibu Donna yang telah menjadi sumber kekuatan terbesar dalam setiap langkah kehidupan penulis. Terima kasih atas segala doa yang tidak pernah putus, kasih sayang yang tulus, pengorbanan yang tanpa pamrih, serta dukungan yang selalu diberikan dalam setiap proses yang penulis jalani. Di balik setiap pencapaian yang diraih, terdapat perjuangan, kerja keras, dan ketulusan Ayah dan Ibu yang tidak pernah berhenti memperjuangkan masa depan penulis. Terima kasih karena selalu memberikan semangat ketika penulis merasa lelah, menguatkan ketika menghadapi kesulitan, serta selalu percaya bahwa penulis mampu menyelesaikan setiap tantangan. Segala bentuk pengorbanan yang telah diberikan tidak akan pernah mampu terbalaskan dengan apa pun. Semoga skripsi ini menjadi salah satu wujud bakti, rasa hormat, dan ungkapan terima kasih penulis kepada Ayah dan Ibu atas segala cinta yang telah diberikan.
8. Teman-teman PERHATIKAN JEMBER yang telah menjadi bagian dari perjalanan penulis selama masa perkuliahan. Terima kasih atas kebersamaan, dukungan, tawa, serta semangat yang selalu diberikan sehingga menjadi penyemangat dan mood booster bagi penulis dalam menyelesaikan skripsi ini. Secara khusus, penulis mengucapkan terima kasih kepada Tika, Agus, Windu, Ghiaz, Mehdi, Nafis, Naufal, Ivan, Farhan, dan Rio yang selalu hadir, membantu, dan memberikan dukungan di setiap proses yang penulis lalui.
9. Seluruh teman-teman 4KA17 angkatan 2022 dari awal semester sampai akhir semester ini selalu menjadi teman yang berbuat baik dan suportif.
10. Serta semua pihak yang tidak mungkin di sebutkan satu persatu namanya yang telah membantu memberikan dukungan dan doa-nya kepada penulis.

Penulis menyadari dalam penulisan tugas akhir ini banyak kekurangannya. Walaupun penulisan ini jauh dari sempurna, peneliti berharap agar penulisan ini

bermanfaat dan menambah wawasan bagi pihak-pihak yang membutuhkan. Akhir kata penulis mengharapkan kritik dan saran yang bersifat membangun untuk menjadi perbaikan di masa yang akan mendatang.

Jakarta,



(Arbet Thetus Sinaga)

DAFTAR ISI

PERNYATAAN ORISINALITAS DAN PUBLIKASI ii

LEMBARAN PENGESAHAN iii

ABSTRAK iv

ABSTRACT v

KATA PENGANTAR vi

DAFTAR ISI ix

DAFTAR TABEL xiii

DAFTAR GAMBAR xiv

DAFTAR LAMPIRAN xvii

1. PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Ruang Lingkup 3

1.3 Tujuan Penelitian 4

1.4 Sistematika Penulisan 4

2. TINJAUAN PUSTAKA 6

2.1 *Podcast* Berbahasa Indonesia 6

2.1.1 Definisi *Podcast* 6

2.1.2 Perkembangan *Podcast* di Indonesia 6

2.2 *Speech-to-Text* (STT) 7

2.2.1 Pengertian *Speech-to-Text* 7

2.2.2 Model Whisper 8

2.3 *Natural Language Processing* 8

2.4 *Text Preprocessing* 9

2.4.1 *Case folding* 9
2.4.2 *Text Cleaning* 9
2.4.3 Tokenisasi 10
2.4.4 *Stopword removal* 10
2.4.5 *Stemming* 11
2.5 N-Gram 11
2.6 Ekstraksi Fitur Menggunakan TF-IDF 12
2.7 Cosine Similarity 12
2.8 Klasifikasi Teks Multikelas 13
2.9 Algoritma *Machine Learning* 14
2.9.1 *Naive Bayes* 14
2.9.2 *Support Vector Machine* 15
2.10 Evaluasi Model 15
2.10.1 Pembagian Data Latih dan Data Uji 15
2.10.2 *Confusion Matrix* 16
2.10.3 *Accuracy* 17
2.10.4 *Precision* 17
2.10.5 *Recall* 18
2.10.6 *F1-Score* 19
2.10.7 Standar Deviasi *F1-Score* (Std F1) 19
2.10.8 *Stratified K-Fold Cross Validation* 20
2.10.9 *Confidience Score* 20
2.11 CRISP-DM 21
2.12 UML 21
2.12.1 *Activity Diagram* 22

2.12.2 *Use Case Diagram* 22
2.13 Flask *Framework* 23
2.14 *Black Box Testing* 23
2.15 Ngrok 24
2.16 Penelitian Terdahulu 24
3. METODOLOGI PENELITIAN 27
3.1 Jenis Penelitian 27
3.2 Tahapan Penelitian 27
3.2.1 *Business Understanding* 29
3.2.2 *Data Understanding* 29
3.2.3 *Data Preparation* 36
3.2.4 *Modeling* 41
3.2.5 *Evaluation* 44
3.2.6 *Deployment* 47
4. HASIL DAN PEMBAHASAN 57
4.1 *Business Understanding* 57
4.2 *Data Understanding* 57
4.2.1 *Dataset Podcast* 58
4.2.2 Hasil *Speech-to-Text* 59
4.3 *Data Preparation* 61
4.3.1 Hasil *Preprocessing* 61
4.3.2 Hasil Ekstraksi Fitur TF-IDF 66
4.4 *Modeling* 68
4.5 *Evaluation* 69
4.5.1 Perbandingan Kinerja Model 69

4.5.2 *Confusion Matrix* 70
4.5.3 *Classification Report* 72
4.5.4 Analisis Hasil Model 75
4.6 *Deployment* 76
4.6.1 Hasil Ekspor Model 76
4.6.2 Hasil Rancangan dan Implementasi Model pada Aplikasi Web 76
4.6.2 *Deployment* Menggunakan Ngrok 78
4.6.3 Hasil Pengujian Black Box 78
5. KESIMPULAN DAN SARAN 83
5.1 Kesimpulan 83
5.2 Saran 83
Daftar Pustaka 84
LAMPIRAN TAMPILAN APLIKASI 1
LISTING PROGRAM 2

DAFTAR TABEL

Tabel 2.1 Ringkasan Penelitian Terdahulu 24
Tabel 4.1 Inventarisasi dan Distribusi *Dataset Podcast* 58
Tabel 4.2 Sampel Hasil Transkripsi Audio *Podcast* 60
Tabel 4.3 Contoh Hasil Transformasi Pra-pemrosesan Teks Per Tahap 62
Tabel 4.4 Distribusi *Dataset Podcast* Setelah *Preprocessing*..... 63
Tabel 4.5 Perbandingan Metrik Performa Model Klasifikasi 70
Tabel 4.6 Classifiation Report Per Kategori Topik..... 72
Tabel 4.7 Hasil Pengujian Black Box Aplikasi Web Klasifikasi Topik *Podcast* . 79

DAFTAR GAMBAR

Gambar 3.1 Flowchart Tahapan Penelitian 28
Gambar 3.2 *Search Query* 30
Gambar 3.3 Pencarian Video 30
Gambar 3.4 Pemeriksaan Metadata Hasil *Scraping* 31
Gambar 3.5 *Duplicate Removal* 31
Gambar 3.6 Penentuan Parameter Seleksi *Podcast* 32
Gambar 3.7 Proses Seleksi Berdasarkan Durasi 33
Gambar 3.8 Memuat Model Whisper 33
Gambar 3.9 Penentuan Lokasi Penyimpanan *Dataset* 34
Gambar 3.10 Pemeriksaan Progress Transkripsi 34
Gambar 3.11 Proses Transkripsi Menggunakan Whisper 35
Gambar 3.12 Penyimpanan Hasil Transkripsi 35
Gambar 3.13 Penyimpanan *Dataset* Transkripsi 36
Gambar 3.14 Proses *Case folding* 37
Gambar 3.15 Proses *Text Cleaning* 37
Gambar 3.16 Proses *Tokenizing* 37
Gambar 3.17 Proses *Stopword removal* 38
Gambar 3.18 Proses *Stemming* 38
Gambar 3.19 Inisialisasi *TF-IDF Vectorizer* 39
Gambar 3.20 Transformasi Fitur TF-IDF 39
Gambar 3.21 Penyimpanan *TF-IDF Vectorizer* 39
Gambar 3.22 Persiapan *Dataset* 40
Gambar 3.23 Pembagian *Dataset* 40
Gambar 3.24 Menampilkan Jumlah Data Latih dan Data Uji 40
Gambar 3.25 Konfigurasi Model *Machine Learning* 41
Gambar 3.26 Verifikasi Konfigurasi Model 41
Gambar 3.27 Inisialisasi *Stratified 5-Fold Cross Validation* 42
Gambar 3.28 Konfigurasi Metrik Evaluasi 42
Gambar 3.29 Proses Pelatihan Model 43

Gambar 3.30 Rekapitulasi Hasil *Cross Validation* 43
Gambar 3.31 Pelatihan Model Akhir 43
Gambar 3.32 Penyimpanan Model 44
Gambar 3.33 Verifikasi Penyimpanan Model 44
Gambar 3.34 Pengujian Model pada Data Uji 44
Gambar 3.35 *Classification Report* 45
Gambar 3.36 Rekapitulasi Hasil Evaluasi 45
Gambar 3.37 Pembentukan *Confusion Matrix* 45
Gambar 3.38 Pembentukan *Confusion Matrix* 46
Gambar 3.39 Perbandingan Performa Model 46
Gambar 3.40 Pengurutan Performa Model 46
Gambar 3.41 Penentuan Model Terbaik 47
Gambar 3.42 Visualisasi Perbandingan Model. 47
Gambar 3.43 Menentukan Model Terbaik 48
Gambar 3.44 Ekspor Model dan *TF-IDF Vectorizer* 48
Gambar 3.45 Penyimpanan Konfigurasi Label 48
Gambar 3.46 *Use Case Diagram* Sistem Klasifikasi Topik *Podcast* 49
Gambar 3.47 *Activity Diagram* Alur Penggunaan Aplikasi 51
Gambar 3.48 Memuat Model 53
Gambar 3.49 Routing Aplikasi 54
Gambar 3.50 Penerimaan Masukan Audio 54
Gambar 3.51 Transkripsi Audio 55
Gambar 3.52 *Preprocessing* dan Klasifikasi 55
Gambar 4.1 Hasil Inventarisasi *Dataset Podcast* 58
Gambar 4.2 Hasil Proses *Speech-to-Text* Menggunakan *Whisper* 59
Gambar 4.3 Hasil Pembentukan Matriks *TF-IDF* 67
Gambar 4.4 Hasil Pelatihan Model *Naive Bayes* dan SVM Linear 69
Gambar 4.5 *Confusion Matrix* Model pada Data Uji 71
Gambar 4.6 Hasil *Classification Report* dan Ringkasan Peforma Model 74
Gambar 4.7 Perbandingan Kinerja Model Klasifikasi Topik *Podcast* 75
Gambar 4.8 Halaman *Upload* Audio 76

Gambar 4.9 Proses Analisis Audio 77
Gambar 4.10 Hasil Klasifikasi dan Transkripsi 77
Gambar 4.11 Hasil Uji Unggah Berkas Audio..... 80
Gambar 4.12 Hasil Uji Validasi Format 80
Gambar 4.13 Hasil Uji Transkripsi dan *Preprocessing* 81
Gambar 4.14 Hasil Uji Klasifikasi SVM 81

DAFTAR LAMPIRAN

LAMPIRAN TAMPILAN APLIKASI.....L-1
LISTING PROGRAM L-2

1. PENDAHULUAN

1.1 Latar Belakang

Podcast telah berkembang menjadi salah satu media digital yang paling diminati masyarakat Indonesia dalam beberapa tahun terakhir. Berdasarkan data yang dipublikasikan oleh GoodStats (2025), sebanyak 42,6% pengguna internet Indonesia berusia di atas 16 tahun tercatat rutin mendengarkan *podcast* setiap minggunya pada tahun 2024, jauh melampaui rata-rata global yang hanya sebesar 22,1%. Masyarakat Indonesia bahkan menghabiskan waktu rata-rata 1 jam 4 menit per hari untuk mendengarkan *podcast*, menjadikannya durasi mendengarkan terlama kesembilan di dunia (GoodStats, 2025). Data ini menegaskan bahwa *Podcast* bukan lagi sekadar tren sesaat, melainkan telah menjadi bagian dari kebiasaan konsumsi media digital masyarakat Indonesia sehari-hari.

Pertumbuhan popularitas *podcast* juga diikuti dengan meningkatnya jumlah kreator dan variasi konten yang dipublikasikan pada berbagai platform digital, termasuk *YouTube*. Beragam topik seperti teknologi, bisnis, kesehatan, dan pendidikan tersedia dalam jumlah yang terus bertambah sehingga volume konten *Podcast* juga semakin meningkat. Namun, peningkatan volume *podcast* tersebut juga menimbulkan tantangan dalam proses pengelompokan dan pencarian konten berdasarkan topik. Identifikasi topik *podcast* pada platform seperti *YouTube* umumnya masih bergantung pada metadata yang diberikan oleh kreator, seperti judul, deskripsi, dan *tag*. Metadata tersebut digunakan untuk memberikan informasi mengenai isi *Podcast*, namun tidak selalu mampu merepresentasikan keseluruhan pembahasan yang terdapat dalam suatu episode. Sharpe (2020) menyatakan bahwa metadata pada banyak *Podcast* digunakan secara tidak konsisten sehingga informasi yang disajikan tidak selalu mencerminkan isi konten secara akurat. Kondisi tersebut menyebabkan proses pengelompokan konten berdasarkan topik menjadi kurang optimal apabila hanya mengandalkan metadata. Sistem klasifikasi topik otomatis yang menganalisis isi audio *Podcast* melalui hasil transkripsi diperlukan agar topik dapat diidentifikasi berdasarkan pembahasan yang

sebenarnya, sehingga menghasilkan klasifikasi yang lebih akurat dan konsisten.

Permasalahan ini dapat didekati melalui kombinasi teknologi *Automatic Speech Recognition* (ASR) dan *Natural Language Processing* (NLP). Kemunculan model *Whisper* yang dikembangkan oleh OpenAI menghadirkan solusi transkripsi ucapan-ke-teks yang robust, karena model ini dilatih pada lebih dari 680.000 jam data audio multibahasa secara *weakly supervised* sehingga mampu melakukan generalisasi dengan baik terhadap berbagai bahasa, aksen, dan kondisi akustik, termasuk Bahasa Indonesia (Radford et al., 2022). Dengan memanfaatkan *Whisper*, konten audio *Podcast* dapat diubah menjadi data teks yang selanjutnya dapat diolah lebih lanjut menggunakan pendekatan NLP untuk keperluan klasifikasi topik.

Setelah audio berhasil ditranskripsi menjadi teks, tahap *Natural Language Processing* (NLP) dilakukan untuk membersihkan dan menyiapkan data teks sebelum memasuki proses klasifikasi. Salah satu tahapan *Preprocessing* yang penting adalah *Stemming*, yaitu mengubah kata yang memiliki imbuhan menjadi bentuk dasar sehingga kata-kata dengan makna yang sama dapat diperlakukan sebagai satu representasi. Dalam penelitian ini digunakan pustaka *PySastrawi* karena dirancang khusus untuk Bahasa Indonesia dan terbukti mampu meningkatkan kualitas *Preprocessing* sehingga menghasilkan representasi teks yang lebih baik untuk proses klasifikasi (Rosid et al., 2020; Jabbar et al., 2023).

Teks yang telah melalui tahap *Preprocessing* kemudian direpresentasikan ke dalam bentuk numerik menggunakan metode *Term Frequency–Inverse Document Frequency* (TF-IDF) agar dapat diproses oleh algoritma *machine learning* (Das et al., 2023). Pada penelitian ini, proses klasifikasi dilakukan menggunakan dua algoritma, yaitu *Naive Bayes* dan *Support Vector Machine* (SVM). Kedua algoritma tersebut dipilih karena telah digunakan dalam penelitian klasifikasi teks dan dapat diterapkan pada data berbahasa Indonesia (Iskandar & Nataliani, 2021; Maulana et al., 2024). Oleh karena itu, penelitian ini membandingkan kedua algoritma tersebut untuk menentukan model yang memberikan performa terbaik dalam mengklasifikasikan topik *Podcast* berbahasa Indonesia.

Penelitian ini mengembangkan sistem klasifikasi topik *Podcast* berbahasa

Indonesia yang diawali dengan proses transkripsi audio menggunakan model *Whisper*. Teks hasil transkripsi kemudian diproses melalui tahapan *Preprocessing Natural Language Processing* (NLP), direpresentasikan menggunakan metode TF-IDF, dan diklasifikasikan menggunakan algoritma *Naive Bayes* dan *Support Vector Machine* (SVM). Model terbaik selanjutnya diintegrasikan ke dalam aplikasi berbasis web sehingga proses klasifikasi topik *Podcast* dapat dilakukan secara otomatis.

Berdasarkan uraian tersebut, penelitian ini mengembangkan sebuah sistem klasifikasi topik *Podcast* berbahasa Indonesia yang mengintegrasikan transkripsi audio otomatis berbasis *Whisper*, *Preprocessing* NLP dengan *Sastrawi*, ekstraksi fitur TF-IDF, serta perbandingan kinerja model *Naive Bayes* dan SVM. Model dengan performa terbaik selanjutnya diimplementasikan ke dalam sebuah aplikasi web berbasis *Flask*, sehingga sistem klasifikasi yang dihasilkan tidak hanya bersifat eksperimental, tetapi juga dapat digunakan secara praktis oleh pengguna akhir untuk mengklasifikasikan topik *Podcast* berbahasa Indonesia secara otomatis. Penelitian ini diharapkan dapat memberikan kontribusi dalam pengembangan penelitian NLP berbahasa Indonesia, khususnya pada bidang klasifikasi topik berbasis transkripsi audio, serta membantu platform penyedia *Podcast*, kreator konten, maupun pengguna dalam melakukan pengelompokan topik *Podcast* secara lebih cepat, konsisten, dan efisien tanpa memerlukan proses pelabelan manual.

1.2 Ruang Lingkup

Ruang lingkup dalam penelitian ini ditetapkan untuk membatasi pembahasan agar lebih terarah dan fokus pada permasalahan utama. Adapun batasan-batasan penelitian ini adalah sebagai berikut:

1) Penelitian ini berfokus pada klasifikasi topik konten *Podcast* berbahasa Indonesia menggunakan data audio *Podcast* yang dikonversi menjadi teks melalui model *Speech-to-Text OpenAI Whisper*
2) Kategori topik yang diklasifikasikan dibatasi pada tujuh kelas, yaitu Teknologi, Bisnis, Kesehatan, Berita, Olahraga, Hiburan, dan Pendidikan
3) *Dataset* penelitian diperoleh dari hasil transkripsi audio *Podcast* berbahasa

Indonesia yang berasal dari tujuh kategori topik yang telah ditentukan sebelumnya.

4) Sistem yang dikembangkan diimplementasikan dalam bentuk aplikasi berbasis web menggunakan *framework Flask* sehingga pengguna dapat mengunggah berkas audio *Podcast* dan memperoleh hasil klasifikasi topik secara otomatis.

1.3 Tujuan Penelitian

Berdasarkan latar belakang dan ruang lingkup yang telah ditetapkan, tujuan penelitian ini adalah menghasilkan aplikasi berbasis web yang mampu menerima *input* berupa file audio *podcast* berbahasa Indonesia, melakukan transkripsi audio menggunakan Whisper Medium, melakukan *preprocessing* teks dan ekstraksi fitur menggunakan TF-IDF, serta menghasilkan prediksi kategori topik secara otomatis. Penelitian ini juga bertujuan membandingkan performa algoritma *Multinomial Naive Bayes* dan *Support Vector Machine* berdasarkan nilai *Accuracy*, *Precision*, *Recall*, dan *F1-Score*, sehingga diperoleh algoritma dengan performa terbaik untuk digunakan dalam proses klasifikasi topik *podcast* pada aplikasi berbasis web.

1.4 Sistematika Penulisan

Sistematika penulisan dalam penelitian ini disusun secara terstruktur untuk memberikan gambaran mengenai keterkaitan antar bab dalam laporan penelitian secara kronologis. Bab 1 Pendahuluan menguraikan latar belakang penelitian, rumusan masalah, ruang lingkup penelitian, tujuan penelitian, serta sistematika penulisan sebagai landasan dalam pelaksanaan penelitian. Bab 2 Tinjauan Pustaka menyajikan landasan teori dan penelitian terdahulu yang relevan, meliputi konsep *Podcast*, teknologi *Speech-to-Text* (STT) menggunakan model *Whisper*, *Natural Language Processing* (NLP), metode pembobotan TF-IDF, algoritma klasifikasi *Machine learning* yang terdiri atas *Naive Bayes* dan *Support Vector Machine* (SVM), serta teori mengenai pengembangan aplikasi berbasis web menggunakan *Flask*. Bab 3 Metodologi Penelitian menjelaskan metode penelitian yang digunakan berdasarkan pendekatan CRISP-DM (*Cross Industry Standard Process for Data Mining*), mulai dari tahapan *Business Understanding*, *Data Understanding*, *Data*

Preparation, *Modeling*, *Evaluation*, hingga *Deployment*. Pada bab ini juga dijelaskan proses pengumpulan *dataset Podcast*, konversi audio menjadi teks menggunakan *Whisper*, tahapan *Preprocessing* NLP, ekstraksi fitur menggunakan TF-IDF, proses pelatihan dan pengujian model klasifikasi, evaluasi kinerja model, serta implementasi model terbaik ke dalam aplikasi berbasis web. Bab 4 Hasil dan Pembahasan menyajikan hasil implementasi penelitian, meliputi analisis *dataset*, hasil transkripsi audio, hasil *Preprocessing*, perbandingan performa algoritma klasifikasi berdasarkan metrik evaluasi, analisis *Confusion Matrix*, pemilihan model terbaik, serta pengujian aplikasi berbasis web yang telah dikembangkan. Bab 5 Penutup berisi kesimpulan yang diperoleh dari hasil penelitian serta saran yang dapat digunakan sebagai bahan pengembangan penelitian maupun sistem pada masa mendatang.

2. TINJAUAN PUSTAKA

2.1 *Podcast* Berbahasa Indonesia

2.1.1 Definisi *Podcast*

Podcast merupakan konten audio digital yang disajikan dalam bentuk episode dan dapat diakses melalui internet secara fleksibel, sehingga setiap episode dapat didengarkan secara terpisah sesuai kebutuhan pendengar (Rime et al., 2022). Penyajian secara episodik menjadi salah satu karakteristik *podcast* yang membedakannya dari siaran radio konvensional yang terikat pada jadwal penyiaran tertentu. Setiap episode dapat berdiri sebagai satu kesatuan konten sekaligus menjadi bagian dari rangkaian pembahasan yang lebih luas dalam suatu program *podcast* (Rime et al., 2022).

Podcast memiliki beragam bentuk penyajian, seperti wawancara, diskusi, percakapan, monolog, dan narasi tematik, sehingga karakteristik isi dan gaya penyampaiannya dapat berbeda antarepisode maupun antarprogram (Rime et al., 2022). Sifat *podcast* sebagai dokumen lisan juga menghasilkan variasi bahasa yang berbeda dari dokumen tertulis formal, sehingga *podcast* dapat dipandang sebagai kumpulan data bahasa lisan dalam jumlah besar yang dapat digunakan untuk kajian berbasis korpus (Clifton et al., 2020). Keberagaman isi dan karakteristik bahasa lisan tersebut menjadi bagian penting dalam memahami *podcast* sebagai objek pengolahan informasi berbasis teks (Jones et al., 2021).

2.1.2 Perkembangan *Podcast* di Indonesia

Perkembangan *podcast* di Indonesia berlangsung dalam konteks pertumbuhan ekosistem digital yang ditandai oleh peningkatan akses internet, penggunaan perangkat seluler, dan pemanfaatan berbagai layanan digital oleh masyarakat (DataReportal, 2025). Kondisi tersebut menyediakan lingkungan digital yang mendukung distribusi dan konsumsi berbagai jenis konten berbasis internet, termasuk konten audio seperti *podcast* (DataReportal, 2025). Data mengenai penggunaan internet dan layanan digital memberikan gambaran mengenai lingkungan tempat konsumsi media berbasis internet berkembang di

Indonesia.

Podcast juga menunjukkan tingkat konsumsi yang cukup luas sebagai bagian dari perkembangan media audio digital (Edison Research, 2024). Perilaku pendengar *podcast* dapat dilihat melalui berbagai indikator, seperti frekuensi mendengarkan, durasi konsumsi, perangkat yang digunakan, serta platform yang dimanfaatkan untuk mengakses *podcast* (Edison Research, 2024). Karakteristik konsumsi tersebut menunjukkan bahwa *podcast* telah berkembang sebagai salah satu bentuk media digital yang memiliki pola penggunaan tersendiri di antara konsumen media audio.

2.2 *Speech-to-Text* (STT)

2.2.1 Pengertian *Speech-to-Text*

Speech-to-Text (STT) merupakan teknologi yang digunakan untuk mengubah ucapan manusia menjadi representasi dalam bentuk teks tertulis. STT berkaitan erat dengan *Automatic Speech Recognition* (ASR), yaitu bidang yang berfokus pada pengenalan bahasa lisan oleh sistem komputer dan menghasilkan keluaran berupa teks (Malik et al., 2021). Teknologi tersebut memungkinkan informasi yang disampaikan melalui suara direpresentasikan dalam bentuk teks sehingga dapat diproses dan dianalisis menggunakan pendekatan pengolahan bahasa.

Sistem ASR telah dikembangkan menggunakan berbagai pendekatan, mulai dari metode berbasis statistik hingga pembelajaran mendalam yang memanfaatkan data audio dan transkripsi dalam jumlah besar (Malik et al., 2021). Perkembangan pendekatan tersebut meningkatkan kemampuan sistem dalam menangani variasi ucapan, termasuk perbedaan aksen, kecepatan bicara, dan kondisi perekaman (Malik et al., 2021). Perkembangan tersebut menjadi dasar bagi penggunaan model ASR modern yang memiliki kemampuan pengenalan ucapan dalam berbagai kondisi dan bahasa.

2.2.2 Model Whisper

Whisper merupakan model *Automatic Speech Recognition* yang dikembangkan menggunakan arsitektur Transformer dan dilatih melalui pendekatan *weak supervision* pada data audio berskala besar (Radford et al., 2022). Model tersebut dilatih menggunakan sekitar 680.000 jam data audio yang berasal dari berbagai sumber di internet beserta transkripsi yang tersedia, sehingga memperoleh representasi terhadap beragam bahasa, aksen, dan kondisi audio (Radford et al., 2022). Whisper juga dirancang sebagai model multibahasa yang memiliki kemampuan melakukan transkripsi dalam berbagai bahasa melalui pendekatan *zero-shot*, sehingga model dapat digunakan pada bahasa yang beragam tanpa memerlukan pelatihan ulang khusus untuk setiap bahasa dalam konfigurasi yang dilaporkan oleh pengembangnya (Radford et al., 2022).

Penerapan Whisper pada Bahasa Indonesia telah diteliti pada beberapa konteks, termasuk transkripsi tuturan formal pada debat calon wakil presiden Republik Indonesia (Khoiroh et al., 2024). Pengembangan lebih lanjut untuk pemrosesan audio berbahasa Indonesia juga dilakukan melalui pendekatan *hybrid cloud* dan *multi-engine* untuk mendukung kinerja transkripsi Whisper pada kondisi penggunaan yang beragam (Fathony et al., 2025). Hasil penelitian tersebut menunjukkan bahwa Whisper telah digunakan pada berbagai konteks audio berbahasa Indonesia dengan karakteristik data yang berbeda.

2.3 *Natural Language Processing*

Natural Language Processing (NLP) merupakan bidang yang mempelajari metode untuk memungkinkan komputer mengolah dan memahami bahasa alami yang digunakan manusia. Ruang lingkup NLP mencakup berbagai bentuk pengolahan bahasa, mulai dari analisis pada tingkat kata dan kalimat hingga pengolahan makna dan informasi yang terdapat dalam suatu dokumen (Jurafsky & Martin, 2025). Berbagai tugas dalam NLP meliputi pengolahan teks, analisis struktur bahasa, dan klasifikasi dokumen berdasarkan karakteristik yang terdapat di dalamnya (Jurafsky & Martin, 2025).

Pengolahan teks berbahasa Indonesia memiliki karakteristik tersendiri

karena penggunaan imbuhan dapat menghasilkan berbagai variasi bentuk kata dari satu kata dasar (Albab et al., 2023). Karakteristik tersebut perlu diperhatikan dalam tahapan pengolahan teks, khususnya pada proses normalisasi dan *Stemming* agar variasi bentuk kata dapat direpresentasikan secara lebih konsisten (Albab et al., 2023). Tahapan *Preprocessing* berperan dalam menyiapkan teks sebelum digunakan pada proses analisis dan klasifikasi lebih lanjut, sehingga kualitas hasil pengolahan awal dapat memengaruhi representasi teks yang digunakan pada tahap berikutnya.

2.4 *Text Preprocessing*

2.4.1 *Case folding*

Case folding merupakan tahapan *text Preprocessing* yang bertujuan menyeragamkan penggunaan huruf pada dokumen teks, umumnya dengan mengubah huruf kapital menjadi huruf kecil. Penyeragaman tersebut membantu mengurangi perbedaan representasi yang hanya disebabkan oleh variasi penggunaan huruf kapital dan huruf kecil pada kata yang sama.

Penerapan *case folding* mendukung konsistensi representasi teks pada tahapan pengolahan berikutnya, terutama ketika teks akan diubah menjadi bentuk fitur untuk proses analisis. Tahapan normalisasi teks merupakan bagian dari proses *Preprocessing* yang digunakan untuk mengurangi variasi bentuk teks sebelum data dianalisis lebih lanjut (Suhaeni et al., 2025).

2.4.2 *Text Cleaning*

Text cleaning merupakan tahapan *Preprocessing* yang bertujuan mengurangi atau menghilangkan unsur yang tidak diperlukan dalam dokumen teks sehingga data menjadi lebih sesuai untuk proses pengolahan selanjutnya (Rosid et al., 2020). Unsur yang termasuk *noise* dapat berupa karakter, simbol, atau informasi tertentu yang dianggap tidak relevan berdasarkan kebutuhan analisis teks. Proses ini membantu menyeragamkan isi dokumen sebelum teks digunakan pada tahapan pengolahan berikutnya.

Pembersihan teks berperan dalam menyiapkan data sebelum dilakukan

tahapan *Preprocessing* lanjutan dan analisis teks (Suhaeni et al., 2025). Hasil *text cleaning* dapat memengaruhi bentuk data yang diteruskan ke proses seperti tokenisasi dan *stopword removal*, sehingga proses pembersihan perlu disesuaikan dengan karakteristik data dan kebutuhan analisis. Penyesuaian tersebut penting agar informasi yang relevan tetap dipertahankan selama proses pengolahan teks.

2.4.3 Tokenisasi

Tokenisasi merupakan proses memecah dokumen teks menjadi satuan-satuan yang disebut *token*. *Token* dapat berupa kata, subkata, atau satuan teks lain sesuai dengan metode tokenisasi yang digunakan dalam proses pengolahan bahasa (Jurafsky & Martin, 2025). Proses tersebut menghasilkan unit dasar yang digunakan untuk merepresentasikan dan mengolah teks pada tahapan NLP berikutnya.

Hasil tokenisasi menjadi dasar bagi berbagai tahapan pengolahan teks karena proses seperti *stopword removal*, *Stemming*, dan ekstraksi fitur dapat dilakukan berdasarkan *token* yang telah terbentuk. Konsistensi *token* yang dihasilkan membantu menjaga keseragaman representasi teks sebelum data digunakan pada tahap analisis atau klasifikasi lebih lanjut (Jurafsky & Martin, 2025).

2.4.4 *Stopword removal*

Stopword merupakan kata yang sering muncul dalam teks dan pada konteks analisis tertentu dapat dianggap memiliki kontribusi informasi yang relatif rendah, sehingga kata tersebut dapat dihilangkan melalui proses *stopword removal* (Suhaeni et al., 2025). Penghapusan *stopword* merupakan salah satu tahapan *text Preprocessing* yang bertujuan mengurangi kata atau unsur teks yang dianggap kurang relevan terhadap kebutuhan analisis.

Penerapan *stopword removal* perlu mempertimbangkan karakteristik dan tujuan pengolahan teks karena kata yang dianggap tidak penting pada suatu analisis belum tentu tidak memiliki makna pada analisis lainnya. Pemilihan daftar *stopword* dengan mempertimbangkan karakteristik data membantu menjaga agar kata yang masih memiliki nilai informasi tidak dihilangkan secara berlebihan (Suhaeni et al.,

2025).

2.4.5 *Stemming*

Stemming merupakan proses dalam *text Preprocessing* yang bertujuan menemukan bentuk dasar dari suatu kata (Albab et al., 2023). Proses ini digunakan untuk mengurangi perbedaan bentuk kata yang memiliki kata dasar yang sama, terutama pada kata yang memiliki imbuhan. Pada teks berbahasa Indonesia, keberadaan berbagai bentuk imbuhan dapat menghasilkan banyak bentuk kata dari satu kata dasar sehingga diperlukan proses untuk menyederhanakan bentuk tersebut (Albab et al., 2023).

Penerapan *Stemming* pada teks berbahasa Indonesia dapat menggunakan pustaka Sastrawi sebagai salah satu alat untuk memperoleh bentuk dasar kata dalam proses *Preprocessing* (Rosid et al., 2020). Penggunaan *Stemming* membantu menyamakan berbagai bentuk kata yang berasal dari kata dasar yang sama sehingga teks dapat diproses dalam bentuk yang lebih seragam sebelum digunakan pada tahap pengolahan berikutnya (Albab et al., 2023; Rosid et al., 2020).

2.5 N-Gram

N-Gram merupakan representasi teks yang dibentuk dari sejumlah *token* yang muncul secara berurutan dalam suatu dokumen. Nilai *N* menunjukkan jumlah *token* yang digunakan dalam satu rangkaian, sehingga *unigram* terdiri atas satu *token*, *bigram* terdiri atas dua *token* yang berurutan, dan *trigram* terdiri atas tiga token yang berurutan (Jurafsky & Martin, 2025).

Representasi *N-Gram* memungkinkan informasi mengenai urutan *token* yang berdekatan tetap dipertahankan dalam bentuk data teks (Jurafsky & Martin, 2025). Penggunaan *N-Gram* dapat membantu menggambarkan pola kemunculan satu atau beberapa kata secara berurutan, sehingga hubungan antarkata yang berdekatan tetap dapat dipertimbangkan dalam pengolahan teks (Jurafsky & Martin, 2025).

2.6 Ekstraksi Fitur Menggunakan TF-IDF

Term Frequency-Inverse Document Frequency (TF-IDF) merupakan metode pembobotan fitur yang digunakan untuk menunjukkan tingkat kepentingan suatu kata dalam sebuah dokumen dibandingkan dengan kumpulan dokumen secara keseluruhan (Das et al., 2023). *Term Frequency* menunjukkan jumlah kemunculan suatu kata dalam sebuah dokumen, sedangkan *Inverse Document Frequency* menunjukkan tingkat keberadaan atau kelangkaan kata tersebut pada kumpulan dokumen. Kedua komponen tersebut digunakan secara bersama-sama untuk menghasilkan bobot yang menunjukkan tingkat kepentingan suatu kata dalam dokumen.

Kata yang sering muncul dalam suatu dokumen tetapi relatif jarang ditemukan pada dokumen lain akan memperoleh bobot yang lebih tinggi karena kata tersebut lebih mampu membedakan isi suatu dokumen dari dokumen lainnya (Das et al., 2023). Hasil pembobotan TF-IDF dapat digunakan sebagai bentuk numerik dari teks yang selanjutnya menjadi fitur bagi algoritma klasifikasi (Das et al., 2023). Penerapan TF-IDF pada klasifikasi teks telah digunakan bersama berbagai algoritma, seperti *Naive Bayes* dan *Support Vector Machine*, pada penelitian dengan karakteristik data yang berbeda (Ardi & Kurniawan, 2024; Pranata et al., 2024).

TF-IDF memiliki keterbatasan karena pembobotannya terutama didasarkan pada pola kemunculan kata dalam dokumen dan kumpulan dokumen, sehingga informasi mengenai hubungan antarkata tidak menjadi bagian utama dari proses pembobotan (Das et al., 2023). Informasi mengenai urutan kata dapat dipertahankan melalui penggunaan representasi tambahan seperti *N-Gram*, yang memungkinkan beberapa *token* yang berurutan dipertimbangkan sebagai satu rangkaian fitur (Gerliandeva et al., 2024).

2.7 Cosine Similarity

Cosine similarity merupakan metode yang digunakan untuk mengukur tingkat kemiripan antara dua objek yang direpresentasikan dalam bentuk vektor (Januzaj & Luma, 2022). Pengukuran dilakukan berdasarkan sudut antara kedua

vektor tersebut, sehingga tingkat kemiripan ditentukan berdasarkan kesamaan arah kedua vektor. Nilai kemiripan yang lebih tinggi menunjukkan bahwa kedua vektor memiliki arah yang semakin serupa.

Pada pengolahan dokumen teks, *cosine similarity* dapat digunakan setelah setiap dokumen direpresentasikan dalam bentuk vektor, misalnya menggunakan pembobotan TF-IDF (Januzaj & Luma, 2022; Gahman & Elangovan, 2023). Perbandingan sudut antarvektor kemudian digunakan untuk menentukan tingkat kemiripan antara dokumen yang dibandingkan. Pendekatan tersebut dapat digunakan dalam pengukuran kemiripan dokumen pada berbagai tugas pengolahan teks (Gahman & Elangovan, 2023).

2.8 Klasifikasi Teks Multikelas

Klasifikasi teks multikelas merupakan tugas pembelajaran mesin yang digunakan untuk mengelompokkan dokumen ke dalam salah satu dari lebih dari dua kelas berdasarkan fitur yang terdapat pada data (Grandini et al., 2020). Klasifikasi multikelas berbeda dari klasifikasi biner karena klasifikasi biner hanya melibatkan dua kelas, sedangkan klasifikasi multikelas mengharuskan model membedakan data ke dalam beberapa kategori secara bersamaan (Grandini et al., 2020). Setiap dokumen pada klasifikasi multikelas akan ditempatkan pada salah satu kelas yang telah ditentukan berdasarkan karakteristik fitur yang digunakan dalam proses klasifikasi.

Klasifikasi multikelas dapat diterapkan pada berbagai jenis data teks yang memiliki lebih dari dua kategori. Pembagian data ke dalam beberapa kelas membuat model perlu membedakan karakteristik setiap kelas secara lebih menyeluruh dibandingkan klasifikasi biner. Evaluasi pada klasifikasi multikelas juga perlu memperhatikan kinerja model pada masing-masing kelas agar hasil evaluasi dapat menggambarkan kemampuan model secara lebih lengkap (Grandini et al., 2020).

2.9 Algoritma *Machine Learning*

2.9.1 *Naive Bayes*

Naive Bayes merupakan algoritma klasifikasi yang menggunakan konsep probabilitas untuk menentukan kelas suatu data berdasarkan fitur yang dimilikinya. Dalam klasifikasi teks, probabilitas tersebut digunakan untuk menentukan kemungkinan suatu dokumen termasuk ke dalam kelas tertentu berdasarkan pola fitur yang terdapat pada dokumen tersebut (Iskandar & Nataliani, 2021). Konsep *Naive Bayes* didasarkan pada Teorema Bayes yang menghubungkan probabilitas suatu kelas dengan probabilitas fitur yang terdapat pada data.

Salah satu ciri utama *Naive Bayes* adalah adanya asumsi bahwa setiap fitur dianggap saling bebas dalam proses perhitungan probabilitas (Gerliandeva et al., 2024). Asumsi tersebut merupakan penyederhanaan yang digunakan agar perhitungan dapat dilakukan secara lebih sederhana, meskipun fitur-fitur dalam data teks pada kenyataannya dapat memiliki keterkaitan satu sama lain. Kondisi tersebut menjadi salah satu keterbatasan *Naive Bayes* ketika digunakan pada data teks yang memiliki hubungan kuat antarfitur.

Multinomial Naive Bayes merupakan salah satu varian *Naive Bayes* yang banyak digunakan pada klasifikasi teks. Varian ini mempertimbangkan jumlah atau bobot kemunculan fitur kata dalam proses penentuan kelas dan dapat digunakan pada data teks yang direpresentasikan dalam bentuk fitur numerik (Gerliandeva et al., 2024). Penggunaan *Multinomial Naive Bayes* pada klasifikasi teks juga diterapkan pada berbagai penelitian dengan data dan tujuan klasifikasi yang berbeda.

Naive Bayes memiliki proses perhitungan yang relatif sederhana dan dapat digunakan pada data dengan jumlah fitur yang besar. Karakteristik tersebut membuat algoritma ini sering digunakan sebagai salah satu pendekatan dalam klasifikasi teks, termasuk ketika dokumen telah diubah menjadi fitur numerik menggunakan pembobotan seperti TF-IDF (Ardi & Kurniawan, 2024). Asumsi independensi fitur tetap menjadi keterbatasan utama karena hubungan antarfitur dalam teks tidak selalu dapat dianggap bebas satu sama lain (Gerliandeva et al., 2024).

2.9.2 *Support Vector Machine*

Support Vector Machine (SVM) merupakan algoritma *supervised learning* yang digunakan untuk melakukan klasifikasi dengan menentukan batas pemisah antara beberapa kelas pada ruang fitur. Batas pemisah tersebut disebut *hyperplane* dan digunakan untuk membedakan data dari kelas yang berbeda (Iskandar & Nataliani, 2021). Data yang paling dekat dengan *hyperplane* disebut *support vector* dan memiliki peran penting dalam menentukan posisi batas pemisah.

SVM berusaha menentukan *hyperplane* dengan mempertahankan jarak yang optimal antara batas pemisah dan data terdekat dari masing-masing kelas. Jarak tersebut disebut *margin* dan menjadi salah satu konsep utama dalam SVM karena berkaitan dengan kemampuan model dalam bekerja pada data baru (Hidayat & Wibowo, 2024). SVM juga menyediakan penggunaan *kernel* untuk menangani data yang tidak dapat dipisahkan secara langsung menggunakan batas linear pada ruang fitur tertentu (Maulana et al., 2024).

SVM memiliki kelebihan berupa kemampuan untuk digunakan pada data dengan jumlah fitur yang besar, termasuk data teks yang memiliki banyak fitur (Iskandar & Nataliani, 2021). Penggunaan SVM pada klasifikasi teks juga telah diterapkan pada berbagai jenis data dengan karakteristik yang berbeda, termasuk data berbahasa Indonesia (Pranata et al., 2024).

Kemampuan SVM dalam membentuk batas pemisah yang jelas menjadi salah satu karakteristik yang mendukung penggunaannya dalam klasifikasi. Kinerja SVM tetap dipengaruhi oleh karakteristik data, representasi fitur, dan pengaturan model yang digunakan, sehingga hasil yang diperoleh dapat berbeda pada setiap permasalahan klasifikasi (Maulana et al., 2024; Rakajati & Hidayat, 2024).

2.10 Evaluasi Model

2.10.1 Pembagian Data Latih dan Data Uji

Pembagian data latih (*training set*) dan data uji (*test set*) merupakan proses pemisahan data berdasarkan fungsi penggunaannya dalam pembentukan dan penilaian model. Data latih digunakan untuk membentuk model berdasarkan pola yang terdapat pada data, sedangkan data uji digunakan untuk menilai kinerja model

menggunakan data yang tidak digunakan pada proses pembentukan model (Azmi et al., 2023). Pemisahan tersebut memungkinkan kinerja model dinilai menggunakan data yang berbeda dari data yang digunakan untuk membentuk model.

Proporsi pembagian data latih dan data uji merupakan salah satu hal yang perlu diperhatikan dalam evaluasi model karena perubahan proporsi dapat memengaruhi hasil klasifikasi yang diperoleh (Vrigazova, 2021). Penelitian mengenai berbagai proporsi pembagian data menunjukkan bahwa rasio *training* dan *testing* dapat memberikan hasil kinerja yang berbeda pada metode klasifikasi yang digunakan (Azmi et al., 2023; Vrigazova, 2021). Pemilihan proporsi pembagian perlu disesuaikan dengan jumlah data dan kebutuhan evaluasi agar data yang tersedia dapat dimanfaatkan secara memadai untuk pembentukan dan penilaian model.

2.10.2 *Confusion Matrix*

Confusion matrix merupakan tabel yang digunakan untuk membandingkan label sebenarnya dengan hasil prediksi model pada suatu tugas klasifikasi (Opitz, 2024). Tabel tersebut menunjukkan jumlah prediksi yang benar dan salah pada setiap kelas sehingga dapat digunakan untuk melihat pola kesalahan yang dihasilkan oleh model. Pada matriks tersebut, prediksi yang benar berada pada bagian yang sesuai antara kelas prediksi dan kelas sebenarnya, sedangkan bagian lainnya menunjukkan kesalahan klasifikasi (Opitz, 2024).

Pada klasifikasi biner, *confusion matrix* umumnya dijelaskan menggunakan empat komponen, yaitu *true positive*, *true negative*, *false positive*, dan *false negative*. Pada klasifikasi multikelas, *confusion matrix* memuat kombinasi antara setiap kelas hasil prediksi dan setiap kelas sebenarnya, sehingga kinerja model dapat dilihat pada masing-masing kelas (Opitz, 2024). Informasi dalam *confusion matrix* kemudian dapat digunakan sebagai dasar untuk menghitung berbagai metrik evaluasi, seperti *accuracy*, *precision*, *recall*, dan *F1-score* (Grandini et al., 2020; Opitz, 2024).

2.10.3 *Accuracy*

Accuracy merupakan metrik evaluasi yang menunjukkan bagian dari keseluruhan data yang berhasil diklasifikasikan dengan benar oleh suatu model (Grandini et al., 2020; Opitz, 2024). Nilai *accuracy* diperoleh dengan membandingkan jumlah prediksi yang benar dengan jumlah keseluruhan data yang diuji, sehingga nilai yang lebih tinggi menunjukkan proporsi prediksi benar yang lebih besar.

$$Accuracy = \frac{(TP + TN)}{TP + TN + FP + FN} \quad (2.1)$$

Keterangan:

- *TP (True Positive)* : jumlah data yang diprediksi benar sebagai kelas positif.
- *TN (True Negative)* : jumlah data yang diprediksi benar sebagai kelas negatif.
- *FP (False Positive)* : jumlah data yang diprediksi sebagai kelas positif tetapi sebenarnya termasuk kelas negatif.
- *FN (False Negative)* : jumlah data yang diprediksi sebagai kelas negatif tetapi sebenarnya termasuk kelas positif.

Nilai *accuracy* perlu memperhatikan distribusi kelas pada data yang digunakan. Pada data dengan distribusi kelas yang tidak seimbang, nilai *accuracy* dapat memberikan gambaran yang kurang lengkap mengenai kinerja model pada masing-masing kelas sehingga perlu dipertimbangkan bersama metrik evaluasi lainnya (Opitz, 2024). Metrik seperti *precision*, *recall*, dan *F1-score* dapat digunakan bersama *accuracy* untuk memberikan gambaran kinerja model yang lebih menyeluruh (Grandini et al., 2020; Opitz, 2024).

2.10.4 *Precision*

Precision merupakan metrik yang menunjukkan bagian dari data yang diprediksi sebagai suatu kelas dan benar-benar termasuk ke dalam kelas tersebut (Grandini et al., 2020). Metrik ini menunjukkan ketepatan prediksi suatu kelas dengan melihat perbandingan antara prediksi yang benar dan seluruh data yang diprediksi sebagai kelas tersebut (Opitz, 2024).

$$Precision = \frac{TP}{TP + FP} \quad (2.2)$$

Keterangan:

- *TP (True Positive)* : jumlah data yang diprediksi benar sebagai kelas tersebut.
- *FP (False Positive)* : jumlah data yang diprediksi sebagai kelas tersebut tetapi sebenarnya termasuk kelas lain.

Nilai *precision* yang tinggi menunjukkan bahwa kesalahan berupa *false positive* pada kelas yang dinilai relatif rendah (Opitz, 2024). Pada klasifikasi multikelas, *precision* dapat dihitung untuk setiap kelas, kemudian nilai dari masing-masing kelas dapat diringkas menggunakan metode perataan tertentu untuk memperoleh nilai keseluruhan (Grandini et al., 2020).

2.10.5 *Recall*

Recall merupakan metrik yang menunjukkan bagian dari seluruh data yang sebenarnya termasuk dalam suatu kelas dan berhasil diprediksi dengan benar oleh model (Grandini et al., 2020). Metrik ini menggambarkan kemampuan model dalam menemukan data yang benar-benar termasuk ke dalam suatu kelas (Opitz, 2024).

$$Recall = \frac{TP}{TP + FN} \quad (2.3)$$

Keterangan:

- *TP (True Positive)* : jumlah data yang diprediksi benar sebagai kelas tersebut.
- *FN (False Negative)* : jumlah data yang sebenarnya termasuk kelas tersebut tetapi diprediksi sebagai kelas lain.

Nilai *recall* yang tinggi menunjukkan bahwa kesalahan berupa *false negative* pada kelas yang dinilai relatif rendah (Opitz, 2024). *Precision* dan *recall* dapat menunjukkan hasil yang berbeda karena peningkatan ketepatan prediksi suatu kelas tidak selalu diikuti oleh peningkatan kemampuan model dalam menemukan seluruh data pada kelas tersebut (Grandini et al., 2020). Kedua metrik tersebut dapat digunakan secara bersama-sama untuk memberikan gambaran yang lebih lengkap mengenai kinerja model.

2.10.6 *F1-Score*

F1-score merupakan metrik evaluasi yang dihitung sebagai rata-rata harmonik antara *precision* dan *recall* (Grandini et al., 2020). Metrik ini menggabungkan kedua ukuran tersebut sehingga dapat memberikan gambaran mengenai ketepatan prediksi dan kemampuan model dalam menemukan data yang termasuk ke dalam suatu kelas (Opitz, 2024). Nilai *F1-score* akan dipengaruhi oleh nilai *precision* dan *recall*, sehingga nilai yang rendah pada salah satu metrik tersebut dapat menyebabkan nilai *F1-score* ikut menurun.

$$F1 - Score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{2.4}$$

Keterangan:

- *Precision* : nilai *precision* yang telah dihitung sebelumnya pada kelas yang sama.
- *Recall* : nilai *recall* yang telah dihitung sebelumnya pada kelas yang sama.

F1-score dapat digunakan sebagai ukuran tambahan ketika *accuracy* belum memberikan gambaran yang cukup mengenai kinerja model pada setiap kelas (Opitz, 2024). Pada klasifikasi multikelas, *F1-score* dapat dihitung untuk masing-masing kelas dan kemudian diringkas menggunakan metode perataan seperti *macro average* atau *weighted average* untuk memperoleh nilai keseluruhan (Grandini et al., 2020).

2.10.7 Standar Deviasi *F1-Score* (Std F1)

Standar deviasi merupakan ukuran yang digunakan untuk menunjukkan tingkat penyebaran nilai terhadap nilai rata-ratanya. Nilai standar deviasi yang kecil menunjukkan bahwa nilai-nilai dalam suatu kumpulan data cenderung berada dekat dengan rata-rata, sedangkan nilai yang lebih besar menunjukkan penyebaran nilai yang lebih tinggi (Illowsky & Dean, 2023). Ukuran ini dapat digunakan untuk melihat seberapa besar perbedaan nilai yang diperoleh dari beberapa pengamatan terhadap nilai rata-ratanya.

Dalam evaluasi model, nilai rata-rata dan standar deviasi *F1-score* dapat digunakan untuk menggambarkan tingkat kestabilan hasil kinerja model pada

beberapa proses validasi atau pengujian yang dilakukan secara berulang (Opoku Asare et al., 2021). Nilai rata-rata menunjukkan tingkat kinerja model secara umum, sedangkan standar deviasi memberikan informasi mengenai variasi nilai *F1-score* yang diperoleh selama proses evaluasi. Standar deviasi yang lebih kecil menunjukkan bahwa hasil evaluasi memiliki variasi yang lebih rendah sehingga kinerja model cenderung lebih konsisten pada pengujian yang dilakukan berulang (Opoku Asare et al., 2021).

2.10.8 *Stratified K-Fold Cross Validation*

Cross validation merupakan teknik evaluasi model yang membagi data ke dalam beberapa bagian atau *fold*, kemudian menggunakan bagian tersebut secara bergantian untuk proses pelatihan dan validasi (Widodo et al., 2022). Pada *K-Fold Cross Validation*, data dibagi menjadi sejumlah *fold* dan setiap *fold* digunakan secara bergantian sebagai data validasi, sedangkan bagian lainnya digunakan untuk pelatihan. Proses tersebut dilakukan sampai seluruh *fold* memperoleh giliran sebagai data validasi (Widodo et al., 2022).

Stratified K-Fold Cross Validation merupakan pengembangan dari *K-Fold Cross Validation* yang mempertahankan proporsi kelas pada setiap *fold* yang terbentuk (Widodo et al., 2022). Pendekatan ini sesuai digunakan pada data klasifikasi yang memiliki distribusi kelas yang tidak seimbang karena pembagian data tetap mempertahankan perbandingan kelas pada setiap *fold*. Dengan demikian, setiap bagian data yang digunakan dalam proses validasi memiliki distribusi kelas yang lebih mendekati distribusi data secara keseluruhan (Widodo et al., 2022).

2.10.9 *Confidence Score*

Confidence score merupakan nilai yang menunjukkan tingkat keyakinan model *machine learning* terhadap kelas yang diprediksinya, dinyatakan dalam rentang 0 sampai 1 atau 0% sampai 100% (Ultralytics, 2026). Nilai ini diperoleh dari probabilitas tertinggi di antara seluruh kelas yang mungkin diprediksi oleh model. Metrik ini bersifat spesifik terhadap satu prediksi tunggal, berbeda dengan *accuracy* yang mengukur performa model secara keseluruhan terhadap

suatu *dataset* (Ultralytics, 2026).

Algoritma seperti *Support Vector Machine* pada dasarnya hanya menghasilkan skor jarak terhadap *hyperplane* tanpa makna probabilistik langsung. Proses kalibrasi probabilitas (*probability calibration*) diperlukan untuk mengubah skor tersebut menjadi nilai probabilitas yang dapat merepresentasikan tingkat keyakinan model secara lebih akurat (Silva Filho et al., 2023). Model yang terkalibrasi dengan baik menghasilkan prediksi dengan *confidence score* tertentu yang sesuai dengan proporsi kebenaran prediksi yang sebenarnya, sehingga nilai tersebut dapat dipercaya sebagai representasi peluang kebenaran suatu prediksi (Silva Filho et al., 2023).

2.11 CRISP-DM

CRISP-DM atau *Cross-Industry Standard Process for Data Mining* merupakan model proses yang digunakan sebagai kerangka kerja untuk melaksanakan proyek *data mining* secara sistematis (Schröer et al., 2021). Model ini bersifat iteratif sehingga tahapan di dalamnya dapat dilakukan secara berulang sesuai dengan kebutuhan proyek dan hasil yang diperoleh pada setiap tahap (Schröer et al., 2021).

CRISP-DM terdiri atas enam tahap, yaitu *Business Understanding*, *Data Understanding*, *Data Preparation*, *Modeling*, *Evaluation*, dan *Deployment* (Schröer et al., 2021). *Business Understanding* berkaitan dengan pemahaman terhadap tujuan proyek, *Data Understanding* berkaitan dengan pemahaman terhadap data yang digunakan, dan *Data Preparation* berkaitan dengan persiapan data sebelum pemodelan. *Modeling* berkaitan dengan penerapan metode pemodelan, *Evaluation* berkaitan dengan penilaian hasil model terhadap tujuan yang telah ditetapkan, sedangkan *Deployment* berkaitan dengan penerapan hasil proyek pada lingkungan yang sesuai (Schröer et al., 2021).

2.12 UML

Unified Modeling Language (UML) merupakan bahasa visual yang digunakan untuk mendefinisikan, menggambarkan, dan mendokumentasikan suatu

sistem dalam pengembangan perangkat lunak (Koç et al., 2021). UML menyediakan berbagai jenis diagram yang dapat digunakan untuk menggambarkan bagian tertentu dari sistem sesuai dengan kebutuhan pemodelan, seperti struktur, aktivitas, dan interaksi dalam sistem (Koç et al., 2021).

Penggunaan UML membantu menggambarkan sistem dalam bentuk visual sehingga kebutuhan dan ruang lingkup sistem dapat dipahami dengan lebih jelas (Koç et al., 2021). Diagram UML juga banyak digunakan untuk keperluan perancangan dan pemodelan sistem serta dapat menjadi bagian dari dokumentasi dalam pengembangan perangkat lunak (Koç et al., 2021).

2.12.1 *Activity Diagram*

Activity diagram merupakan salah satu jenis diagram UML yang digunakan untuk menggambarkan alur kerja suatu proses atau sistem (Aliman, 2021). Diagram ini menunjukkan rangkaian aktivitas yang dilakukan dalam suatu proses secara terstruktur, mulai dari titik awal hingga titik akhir proses. Setiap aktivitas direpresentasikan menggunakan notasi tertentu sesuai dengan fungsi yang dimilikinya (Aliman, 2021).

Activity diagram juga dapat digunakan untuk menggambarkan percabangan proses melalui elemen *decision* serta menunjukkan hubungan antaraktivitas melalui *control flow* (Maatuk & Abdelnabi, 2021). Penggunaan elemen tersebut membantu menggambarkan perubahan alur berdasarkan kondisi tertentu dan hubungan antaraktivitas dalam suatu proses secara lebih jelas (Maatuk & Abdelnabi, 2021).

2.12.2 *Use Case Diagram*

Use case diagram merupakan salah satu jenis diagram UML yang digunakan untuk menggambarkan fungsi yang disediakan oleh suatu sistem dan hubungan antara sistem dengan pihak yang berinteraksi dengannya (Aliman, 2021; Aquino et al., 2020). Pihak yang berinteraksi dengan sistem disebut *actor*, sedangkan fungsi yang ditawarkan oleh sistem direpresentasikan sebagai *use case* (Aquino et al., 2020). *Use case diagram* juga memiliki *system boundary* yang digunakan untuk menunjukkan batas antara fungsi yang berada di dalam sistem dan

lingkungan di luar sistem (Aquino et al., 2020).

Hubungan antara *actor* dan *use case* dapat digambarkan melalui hubungan yang menunjukkan interaksi keduanya. Diagram ini juga dapat memuat relasi *include* dan *extend* antar*use case*. Relasi *include* menunjukkan bahwa suatu *use case* membutuhkan fungsi dari *use case* lainnya, sedangkan *extend* menunjukkan adanya fungsi tambahan yang dapat digunakan pada kondisi tertentu (Aquino et al., 2020). Penggunaan elemen tersebut membantu menggambarkan fungsi dan hubungan dalam suatu sistem secara terstruktur.

2.13 Flask *Framework*

Flask merupakan *micro web framework* berbasis Python yang digunakan untuk membangun aplikasi berbasis web (Walingkas & Saian, 2023). Sebagai *micro framework*, Flask menyediakan komponen dasar untuk mendukung pengembangan aplikasi web dan memberikan keleluasaan dalam menentukan struktur aplikasi sesuai kebutuhan (Walingkas & Saian, 2023).

Flask memiliki karakteristik yang ringan dan fleksibel sehingga dapat digunakan dalam pengembangan aplikasi web dengan kebutuhan yang berbeda (Ciksadan et al., 2024). Penerapan Flask juga dapat digunakan pada aplikasi web yang terintegrasi dengan fungsi pengolahan data atau model *machine learning* sesuai kebutuhan sistem (Ciksadan et al., 2024).

2.14 *Black Box Testing*

Black box testing merupakan metode pengujian perangkat lunak yang digunakan untuk memeriksa fungsi sistem berdasarkan masukan (*input*) dan keluaran (*output*) tanpa memerlukan pengetahuan mengenai kode program atau struktur internal sistem (Huda et al., 2022). Pengujian ini dilakukan untuk mengetahui apakah fungsi yang tersedia pada perangkat lunak telah berjalan sesuai dengan kebutuhan dan spesifikasi yang ditetapkan (Zidan et al., 2022).

Equivalence partitioning merupakan salah satu teknik dalam *black box testing* yang membagi data masukan ke dalam beberapa kelompok berdasarkan karakteristik atau fungsi yang sama, sehingga setiap kelompok dapat digunakan

sebagai perwakilan kondisi tertentu dalam pengujian (Huda et al., 2022). Teknik ini digunakan untuk menguji berbagai kondisi masukan pada suatu fungsi dan mengelompokkannya berdasarkan hasil atau karakteristik pengujiannya (Huda et al., 2022).

2.15 Ngrok

Ngrok merupakan layanan *tunneling* yang dapat digunakan untuk menghubungkan layanan yang berjalan pada *server* lokal dengan jaringan eksternal sehingga layanan tersebut dapat diakses dari luar jaringan lokal (Koentjoro & Budiardjo, 2025). Penggunaan Ngrok berkaitan dengan kebutuhan untuk mengakses *server* lokal yang berada di balik *firewall* atau *Network Address Translation* (NAT), sehingga layanan lokal dapat diakses melalui jaringan eksternal (Koentjoro & Budiardjo, 2025).

Dalam model *client-server*, Ngrok dapat digunakan sebagai penghubung antara klien dan layanan yang berjalan pada *server* lokal melalui jaringan eksternal (Koentjoro & Budiardjo, 2025). Konsep *tunneling* tersebut memungkinkan layanan yang berjalan pada lingkungan lokal digunakan dari jaringan luar tanpa menjadikan pembahasan terfokus pada konfigurasi jaringan atau implementasi *server* secara teknis.

2.16 Penelitian Terdahulu

Sejumlah penelitian terdahulu yang relevan dengan topik *Speech-to-Text*, *Natural Language Processing*, ekstraksi fitur TF-IDF, serta algoritma *Naive Bayes* dan *Support Vector Machine* dirangkum pada Tabel 2.3 berikut, beserta persamaan dan perbedaannya dengan kajian teori yang telah diuraikan pada subbab sebelumnya.

Tabel 2.1 Ringkasan Penelitian Terdahulu

Peneliti (Tahun)	**Judul**	**Metode**	**Hasil**	**Persamaan**	**Perbedaan**
Radford et al. (2022)	Robust Speech Recognitio	Whisper (encoder-decoder	Whisper menghasilkan transkripsi	Menggunakan model Whisper sebagai teknik	Berfokus pada pengembangan model ASR,

Peneliti (Tahun)	Judul	Metode	Hasil	Persamaan	Perbedaan
	n via Large-Scale Weak Supervision	Transformer), pelatihan weakly supervised 680.000 jam audio	multibahasa dengan tingkat kesalahan rendah tanpa fine-tuning	*Speech-to-Text*	bukan pada klasifikasi topik teks hasil transkripsi
Khoiroh et al. (2024)	Implementasi Speech Recognition Whisper Pada Debat Calon Wakil Presiden Republik Indonesia	Whisper untuk transkripsi otomatis audio debat formal	Whisper menghasilkan transkripsi dengan Word Error Rate yang kompetitif	Menerapkan Whisper pada domain audio berbahasa Indonesia	Berfokus pada notulensi debat, tidak membahas klasifikasi topik multikelas
Iskandar & Nataliani (2021)	Perbandingan Naïve Bayes, SVM, dan k-NN untuk Analisis Sentimen Gadget Berbasis Aspek	*Naive Bayes*, SVM, dan k-NN dengan fitur TF-IDF	*Naive Bayes* dan SVM menunjukkan performa yang baik dibandingkan k-NN dalam klasifikasi teks yang diteliti.	Sama-sama menggunakan TF-IDF serta membandingkan *Naive Bayes* dan SVM untuk klasifikasi teks.	Penelitian tersebut melakukan analisis sentimen pada ulasan gadget, sedangkan penelitian ini melakukan klasifikasi topik *podcast* berbahasa Indonesia.
Rosid et al. (2020)	Improving *Text Preprocessing* for Student Complaint Document Classification Using Sastrawi	*Stemming* Sastrawi dikombinasikan dengan *Support Vector Machine*	*Preprocessing* berbasis Sastrawi meningkatkan akurasi klasifikasi dokumen berbahasa Indonesia	Menggunakan Sastrawi pada tahap *Stemming* teks Bahasa Indonesia	Diterapkan pada dokumen keluhan mahasiswa, bukan pada transkripsi audio *podcast*
Rakajati & Hidayat (2024)	Perbandingan Metode *Naive*	*Naive Bayes* dan SVM untuk klasifikasi	SVM memperoleh akurasi lebih tinggi	Sama-sama menggunakan *Naive Bayes* dan SVM pada	Kelas yang diklasifikasikan berupa bahasa

Peneliti (Tahun)	Judul	Metode	Hasil	Persamaan	Perbedaan
	Bayes dan *Support Vector Machine* Pada Klasifikasi 22 Bahasa Daerah	multikelas	dibandingkan *Naive Bayes* pada klasifikasi bahasa daerah yang diteliti.	permasalahan klasifikasi multikelas.	daerah, sedangkan penelitian ini menggunakan kategori topik *podcast* berbahasa Indonesia.

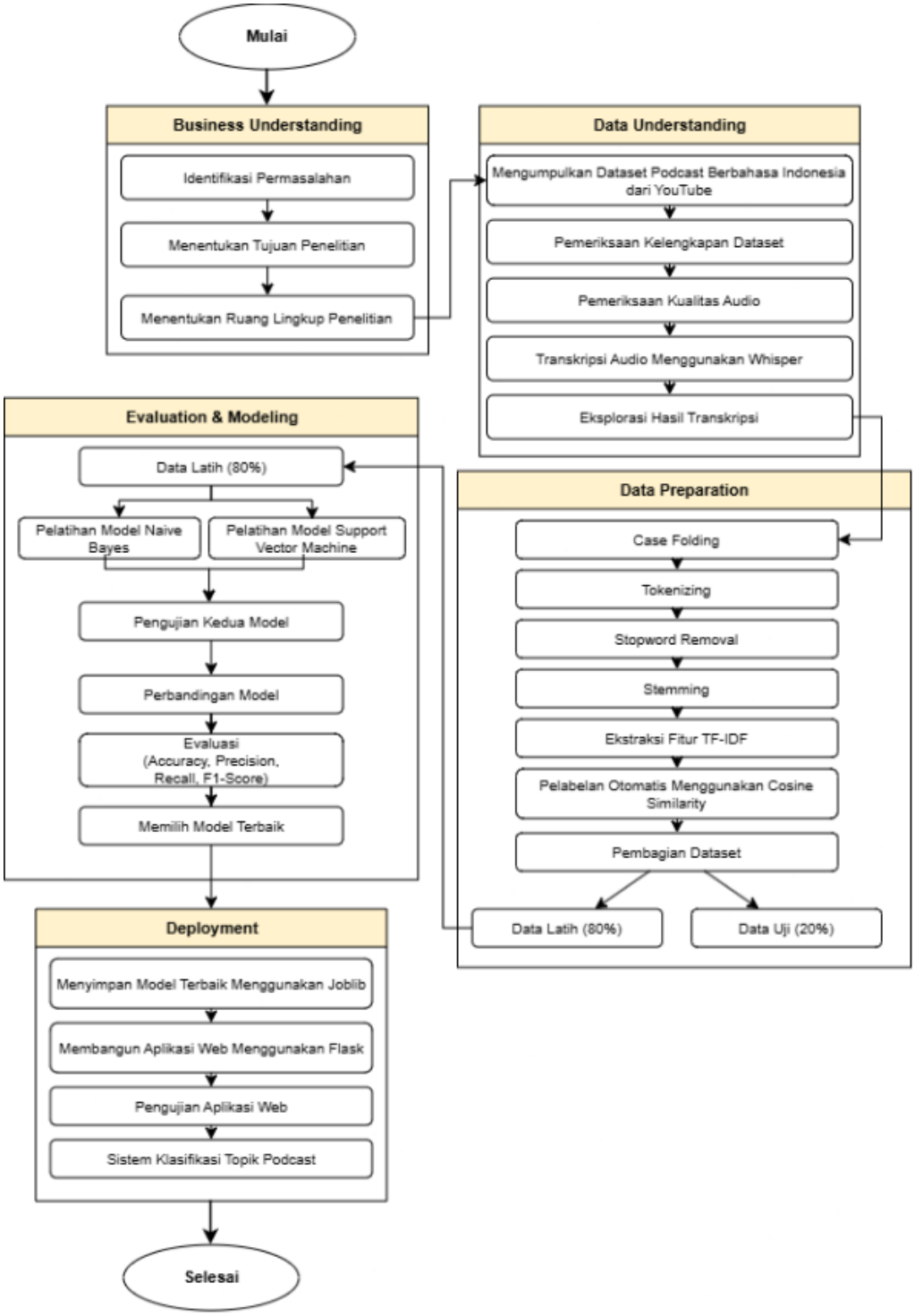
3. METODOLOGI PENELITIAN

3.1 Jenis Penelitian

Penelitian ini merupakan penelitian terapan (*applied research*) yang berfokus pada pengembangan sistem klasifikasi topik konten *podcast* berbahasa Indonesia menggunakan teknologi *Speech-to-Text* (STT), *Natural Language Processing* (NLP), dan algoritma *Machine Learning*. Penelitian ini bersifat eksperimental, yaitu dengan melakukan serangkaian proses pengumpulan data *podcast* dari platform YouTube, transkripsi audio menjadi teks menggunakan model Whisper, pengolahan teks melalui tahapan *Natural Language Processing*, pelatihan dan pengujian model klasifikasi menggunakan algoritma *Multinomial Naive Bayes* dan *Support Vector Machine*, serta mengevaluasi performa model berdasarkan metrik klasifikasi berupa *accuracy*, *precision*, *recall*, dan F1-*scor*e.

3.2 Tahapan Penelitian

Penelitian ini dilaksanakan berdasarkan tahapan dalam metodologi *Cross-Industry Standard Process for Data Mining* (CRISP-DM) yang terdiri atas enam tahap utama, yaitu *Business Understanding*, *Data Understanding*, *Data Preparation*, *Modeling*, *Evaluation*, dan *Deployment*. Setiap tahapan dilakukan secara sistematis agar proses pembangunan model klasifikasi topik *podcast* berlangsung terstruktur, mulai dari identifikasi permasalahan hingga implementasi model ke dalam aplikasi berbasis web. Alur tahapan penelitian dapat dilihat pada Gambar 3.1.



Gambar 3.1 Flowchart Tahapan Penelitian

Gambar 3.1 menunjukkan tahapan penelitian berdasarkan metodologi CRISP-DM, yang dimulai dari *Business Understanding* untuk menentukan permasalahan, tujuan, dan ruang lingkup penelitian. Tahap *Data Understanding*

dilakukan untuk mengumpulkan, memeriksa, dan memahami *dataset podcast* serta melakukan transkripsi audio menggunakan Whisper. Selanjutnya, *Data Preparation* mencakup *Preprocessing*, ekstraksi fitur TF-IDF, pelabelan data, dan pembagian *dataset* menjadi data latih dan data uji. Tahap *Modeling* dan *Evaluation* digunakan untuk membangun serta membandingkan model *Naive Bayes* dan SVM Linear berdasarkan metrik evaluasi. Model terbaik kemudian digunakan pada tahap *Deployment* dengan mengintegrasikannya ke dalam aplikasi web berbasis Flask untuk melakukan klasifikasi topik *podcast*.

3.2.1 *Business Understanding*

Tahap *Business Understanding* bertujuan untuk mengidentifikasi permasalahan, menetapkan tujuan, dan menentukan ruang lingkup penelitian. Permasalahan yang diangkat adalah belum optimalnya proses klasifikasi topik *podcast* berbahasa Indonesia karena masih bergantung pada judul dan deskripsi video yang belum tentu merepresentasikan isi pembahasan. Penelitian ini membangun sistem klasifikasi topik *podcast* dengan memanfaatkan teknologi *Speech-to-Text* menggunakan Whisper, teknik *Natural Language Processing* (NLP), serta algoritma *Multinomial Naive Bayes* dan *Support Vector Machine* (SVM). Ruang lingkup penelitian dibatasi pada *podcast* berbahasa Indonesia yang bersumber dari platform YouTube dengan tujuh kategori topik, yaitu teknologi, bisnis, pendidikan, kesehatan, olahraga, berita, dan hiburan. Model klasifikasi dengan performa terbaik diimplementasikan ke dalam aplikasi berbasis web menggunakan Flask.

3.2.2 *Data Understanding*

Pengumpulan data dilakukan menggunakan bahasa pemrograman Python dengan memanfaatkan pustaka yt-dlp. Pustaka tersebut digunakan untuk melakukan pencarian video pada YouTube berdasarkan *search query* yang telah ditentukan.

3.2.2.1 Pengumpulan Data

Pengumpulan data dilakukan pada tanggal 26 juli 2026 menggunakan script Python dengan memanfaatkan pustaka yt-dlp. Script melakukan pencarian *podcast* pada YouTube menggunakan tujuh *search query* sesuai kategori penelitian.

```
SEARCH_QUERIES = [
    "podcast teknologi indonesia",
    "bincang olahraga indonesia",
    "podcast hiburan indonesia",
    "podcast pendidikan indonesia",
    "podcast bisnis indonesia",
    "bincang kesehatan indonesia",
    "podcast berita indonesia",
]
```

Gambar 3.2 *Search Query*

Setiap *search query* menghasilkan maksimal 250 kandidat video menggunakan parameter ytsearch250.

```
res = ydl.extract_info(
    f"ytsearch250:{query}",
    download=False
)
```

Gambar 3.3 Pencarian Video

Metadata yang diambil dari setiap video meliputi:

- Video ID
- Judul video
- Deskripsi

Video yang diperoleh belum langsung dimasukkan ke dalam *dataset*. Script melakukan proses pelabelan otomatis menggunakan TF-IDF dan Cosine Similarity sebelum audio diunduh.

Pemeriksaan dilakukan terhadap metadata hasil *scraping* untuk memastikan setiap *podcast* memiliki informasi yang diperlukan sebelum proses pengunduhan audio.

Script memeriksa keberadaan:

- Video ID

- Judul video
- Deskripsi video

```python
vid_id = e.get("id")
title = (e.get("title") or "").strip()
description = (e.get("description") or "").strip()

if not vid_id or not title or not description:
    continue
```

Gambar 3.4 Pemeriksaan Metadata Hasil *Scraping*

Script mengambil metadata setiap video yang terdiri atas Video ID**,** judul video**,** dan deskripsi video. Pemeriksaan dilakukan menggunakan kondisi **if not** untuk memastikan ketiga informasi tersebut tersedia. Video yang tidak memiliki salah satu metadata tersebut tidak diproses lebih lanjut sehingga hanya video dengan metadata lengkap yang digunakan pada penelitian. Script juga menghapus data duplikat menggunakan Video ID dan judul video.

```
if not vid_id or vid_id in processed_ids:
    continue

if title.lower() in processed_titles:
    continue
```

Gambar 3.5 *Duplicate Removal*

Script melakukan pemeriksaan data duplikat menggunakan Video ID sebagai identitas unik setiap video YouTube. Pemeriksaan tambahan dilakukan terhadap judul video untuk menghindari konten yang sama tersimpan lebih dari satu kali. Video yang telah diproses sebelumnya akan dilewati sehingga setiap *podcast* hanya muncul satu kali pada *dataset*.

Seleksi audio *podcast* dilakukan untuk menentukan *podcast* yang akan digunakan pada proses transkripsi. Seleksi dilakukan berdasarkan nilai *Cosine Similarity*, durasi video, serta jumlah data yang ditetapkan pada setiap kategori. Video yang tidak memenuhi salah satu kriteria tersebut tidak diproses lebih lanjut sehingga hanya *podcast* yang sesuai dengan kebutuhan penelitian yang diunduh.

Script menetapkan nilai *Similarity Threshold* sebesar 0**,**05 sebagai batas minimum kemiripan antara metadata video dengan *seed keyword* pada setiap kategori. Video hanya dipilih apabila memiliki nilai Cosine Similarity lebih besar atau sama dengan 0**,**05.

```
TARGET_PER_TOPIK = 160
SIMILARITY_THRESHOLD = 0.05

DURASI_MIN = 300
DURASI_MAX = 1800
```

Gambar 3.6 Penentuan Parameter Seleksi *Podcast*

Nilai 0,05 dipilih karena data masukan yang digunakan pada proses pelabelan berupa judul video yang umumnya terdiri atas 5–15 kata. Representasi judul menggunakan TF-IDF menghasilkan vektor dengan jumlah fitur yang relatif sedikit sehingga nilai Cosine Similarity cenderung lebih rendah dibandingkan dokumen yang memiliki teks panjang. Nilai batas minimum kemiripan, diambil 0,05 dengan harapan kata dalam judul beriirisan dengan minimal 1 kata dalam keyword.

Script juga menetapkan target sebanyak 160 *podcast* pada setiap kategori. Pembatasan tersebut dilakukan agar jumlah data pada setiap kategori tetap seimbang (*balanced dataset*) sehingga seluruh kategori memiliki jumlah sampel yang sama sebelum dilakukan pembagian menjadi data latih dan data uji.

Selain menggunakan nilai *similarity*, script juga melakukan penyaringan berdasarkan durasi video. *Podcast* yang digunakan memiliki durasi minimum 300 detik (5 menit) dan durasi maksimum 1800 detik (30 menit). *Podcast* dengan durasi di bawah lima menit tidak dipilih karena umumnya memiliki isi pembahasan yang terlalu singkat, sedangkan *podcast* dengan durasi lebih dari tiga puluh menit tidak dipilih untuk menjaga efisiensi proses pengunduhan, transkripsi, dan pelatihan model.

Penyaringan durasi diterapkan secara langsung pada proses pengunduhan menggunakan parameter match_filter pada pustaka yt-dlp.

```
dl_opts = {
    'format': 'bestaudio/best',
    'outtmpl': os.path.join(output_dir, f'{label}_{vid_id}.%(ext)s'),
    'postprocessors': [{
        'key': 'FFmpegExtractAudio',
        'preferredcodec': 'mp3',
        'preferredquality': '128',
    }],
    'match_filter': yt_dlp.utils.match_filter_func(
        f'duration >= {DURASI_MIN} & duration <= {DURASI_MAX}'
    ),
    'quiet': True,
    'no_warnings': True,
}
```

Gambar 3.7 Proses Seleksi Berdasarkan Durasi

Podcast yang memenuhi seluruh kriteria seleksi kemudian diunduh menggunakan yt-dlp dan dikonversi ke dalam format MP3 menggunakan FFmpeg dengan kualitas 128 kbps. File audio hasil unduhan disimpan ke dalam folder sesuai kategori sehingga siap digunakan pada proses transkripsi menggunakan model Whisper.

3.2.2.4 Transkripsi Audio Menggunakan Whisper

Transkripsi dilakukan untuk mengubah data audio *podcast* menjadi data teks menggunakan model Whisper. Seluruh file audio hasil seleksi pada tahap sebelumnya diproses secara otomatis sehingga menghasilkan transkripsi yang akan digunakan sebagai *dataset* pada tahap *Data Preparation*. Model Whisper yang digunakan pada penelitian ini ditentukan melalui variabel WHISPER_MODEL_SIZE, kemudian dimuat ke dalam memori sebelum proses transkripsi dimulai. Pemilihan model tersebut mempertimbangkan keseimbangan antara kecepatan proses transkripsi dan kualitas hasil transkripsi sehingga sesuai digunakan untuk mentranskripsikan *podcast* berbahasa Indonesia..

```
whisper_model = whisper.load_model(WHISPER_MODEL_SIZE)
```

Gambar 3.8 Memuat Model Whisper

Lokasi penyimpanan hasil transkripsi ditentukan dalam bentuk berkas CSV dan JSON sehingga hasil transkripsi dapat digunakan kembali tanpa perlu

melakukan proses transkripsi ulang. Script juga membuat direktori penyimpanan secara otomatis apabila direktori tersebut belum tersedia.

```
SAVE_PATH = f'{BASE_DIR}/data/dataset/dataset_transkripsi.csv'
JSON_PATH = f'{BASE_DIR}/data/dataset/transkripsi_detail.json'

os.makedirs(os.path.dirname(SAVE_PATH), exist_ok=True)
```

Gambar 3.9 Penentuan Lokasi Penyimpanan *Dataset*

Sebelum proses transkripsi dimulai, script melakukan pemeriksaan terhadap berkas hasil transkripsi yang telah tersimpan. Apabila proses transkripsi sebelumnya belum selesai, data yang telah diproses akan dimuat kembali sehingga file audio yang sudah berhasil ditranskripsikan tidak diproses ulang.

```
if os.path.exists(SAVE_PATH):
    df_done = pd.read_csv(SAVE_PATH)

    if 'audio_file' not in df_done.columns:
        df_done['audio_file'] = ''

    done_list = df_done.to_dict('records')
else:
    done_list = []

done_files = {
    str(d.get('audio_file', '')).strip()
    for d in done_list
}
```

Gambar 3.10 Pemeriksaan Progress Transkripsi

Setiap file audio kemudian diproses menggunakan model Whisper dengan parameter yang telah ditentukan. Parameter language='id' digunakan agar proses transkripsi difokuskan pada bahasa Indonesia. Parameter task='transcribe' digunakan untuk menghasilkan teks transkripsi tanpa melakukan proses penerjemahan. Parameter no_speech_threshold = 0.6 digunakan untuk mengurangi kemungkinan model menghasilkan transkripsi pada bagian audio yang tidak mengandung percakapan, sedangkan condition_on_previous_*text* = True digunakan agar model mempertimbangkan hasil transkripsi sebelumnya sehingga menghasilkan transkripsi yang lebih konsisten.

```
hasil = whisper_model.transcribe(
    path,
    language='id',
    task='transcribe',
    verbose=False,
    no_speech_threshold=0.6,
    condition_on_previous_text=True,
)
```

Gambar 3.11 Proses Transkripsi Menggunakan Whisper

Setelah proses transkripsi selesai, script mengambil teks hasil transkripsi, menghitung jumlah kata pada setiap dokumen, serta menyimpan informasi label kategori dan nama file audio ke dalam sebuah struktur data. Nilai *n*_kata dihitung untuk mengetahui jumlah kata yang dihasilkan pada setiap dokumen hasil transkripsi dan digunakan sebagai informasi tambahan selama proses pemeriksaan *dataset*.

```
teks = hasil['text'].strip()
n_kata = len(teks.split())

record = {
    'teks': teks,
    'label': label,
    'n_kata': n_kata,
    'audio_file': audio_name,
}

hasil_semua.append(record)
```

Gambar 3.12 Penyimpanan Hasil Transkripsi

Untuk menghindari kehilangan data apabila proses transkripsi terhenti, script menyimpan hasil transkripsi secara bertahap ke dalam berkas *dataset*_transkripsi.csv setiap kali satu file audio berhasil diproses. Seluruh hasil transkripsi juga disimpan dalam format JSON sebagai data cadangan yang berisi informasi lengkap setiap dokumen.

```
df_progress = pd.DataFrame(hasil_semua)

df_progress[
    ['teks', 'label', 'audio_file']
].to_csv(
    SAVE_PATH,
    index=False,
    encoding='utf-8-sig'
)

with open(JSON_PATH, 'w', encoding='utf-8') as f:
    json.dump(
        hasil_semua,
        f,
        ensure_ascii=False,
        indent=2
    )
```

Gambar 3.13 Penyimpanan *Dataset* Transkripsi

Dataset hasil transkripsi yang tersimpan pada *dataset*_transkripsi.csv selanjutnya digunakan sebagai masukan pada tahap *Data Preparation*, yaitu proses *Preprocessing* teks dan ekstraksi fitur menggunakan TF-IDF.

3.2.3 *Data Preparation*

Tahap *Data Preparation* bertujuan untuk mempersiapkan data hasil transkripsi agar siap digunakan pada proses pembangunan model klasifikasi. Pada tahap ini dilakukan serangkaian proses *Preprocessing* terhadap data teks, ekstraksi fitur menggunakan TF-IDF, serta pembagian *dataset* menjadi data latih dan data uji. Adapun tahapan yang dilakukan adalah sebagai berikut.

3.2.3.1 *Case folding*

Tahap *case folding* dilakukan di dalam fungsi preprocess_*text*() sebagai proses awal *Preprocessing* teks. Script memeriksa terlebih dahulu apakah data yang diproses bertipe string. Apabila data bukan bertipe string, fungsi akan mengembalikan nilai kosong sehingga proses *Preprocessing* tidak menghasilkan kesalahan. Setelah itu, seluruh karakter pada hasil transkripsi diubah menjadi huruf kecil menggunakan fungsi lower(). Hasil dari proses ini kemudian digunakan pada

tahap *Text Cleaning*.

```
text = text.lower()
```

Gambar 3.14 Proses *Case folding*

3.2.3.2 *Text Cleaning*

Tahap *text cleaning* dilakukan setelah proses *case folding* di dalam fungsi preprocess_*text*(). Script membersihkan hasil transkripsi dengan menghapus URL, alamat email, angka, simbol, serta huruf tunggal menggunakan regular expression. Setelah proses tersebut selesai, script melakukan normalisasi spasi dengan menghilangkan spasi yang berlebihan sehingga teks memiliki format yang lebih seragam sebelum memasuki tahap tokenisasi.

```
text = re.sub(r'http\S+|www\S+', ' ', text)          # Hapus URL
text = re.sub(r'\S+@\S+', ' ', text)                 # Hapus email
text = re.sub(r'[^a-zA-Z\s]', ' ', text)             # Hapus angka & simbol
text = re.sub(r'\b[a-z]\b', ' ', text)               # Hapus huruf tunggal
text = re.sub(r'\s+', ' ', text).strip()             # Normalisasi spasi
```

Gambar 3.15 Proses *Text Cleaning*

3.2.3.3 *Tokenizing*

Tahap *tokenizing* dilakukan untuk memisahkan setiap kalimat hasil transkripsi menjadi kumpulan *token* berupa kata tunggal. Proses tokenisasi diimplementasikan menggunakan fungsi *word_tokenize*() terhadap teks yang telah melalui proses *text cleaning*. Hasil tokenisasi kemudian digunakan sebagai masukan pada tahap *Stopword removal*.

```
tokens = word_tokenize(text)
```

Gambar 3.16 Proses *Tokenizing*

3.2.3.4 *Stopword removal*

Tahap *stopword removal* dilakukan dengan membandingkan setiap *token* terhadap kumpulan ALL_STOPWORDS yang telah disiapkan sebelumnya. *Token* yang terdapat di dalam daftar stopword tidak disertakan pada proses berikutnya.

Script juga menghapus *token* yang memiliki panjang karakter kurang dari atau sama dengan dua karakter sehingga hanya *token* yang memenuhi kriteria yang dipertahankan.

```
tokens = [t for t in tokens if t not in ALL_STOPWORDS and len(t) > 2]
```

Gambar 3.17 Proses *Stopword removal*

3.2.3.5 *Stemming*

Tahap *Stemming* dilakukan menggunakan pustaka Sastrawi melalui objek stemmer. Script melakukan proses *Stemming* terhadap setiap *token* apabila parameter use_*Stemming* bernilai True dan masih terdapat *token* yang akan diproses. Hasil dari proses ini digunakan pada tahap berikutnya sebelum data disimpan sebagai hasil *Preprocessing*.

```
if use_stemming and tokens:
    tokens = [stemmer.stem(t) for t in tokens]
```

Gambar 3.18 Proses *Stemming*

3.2.3.6 Ekstraksi Fitur Menggunakan TF-IDF

Tahap ekstraksi fitur dilakukan setelah *dataset* dibagi menjadi data latih dan data uji. Proses ini menggunakan kelas TfidfVectorizer dari pustaka Scikit-learn untuk mengubah setiap dokumen hasil *Preprocessing* menjadi representasi numerik berbentuk matriks TF-IDF. Pada penelitian ini, *vectorizer* dikonfigurasi menggunakan unigram dan bigram (ngram_range=(1,2)), jumlah fitur maksimum sebanyak 5000 term (max_features=5000), serta hanya mempertahankan term yang muncul minimal pada dua dokumen (min_df=2). Selain itu, parameter sublinear_tf=True digunakan untuk menerapkan pembobotan frekuensi kata menggunakan skala logaritmik. Seluruh konfigurasi tersebut diterapkan pada saat inisialisasi objek TfidfVectorizer.

```
tfidf_vectorizer = TfidfVectorizer(
    ngram_range=(1, 2),
    max_features=5000,
    sublinear_tf=True,
    min_df=2,
    analyzer='word',
    strip_accents='unicode',
)
```

Gambar 3.19 Inisialisasi *TF-IDF Vectorizer*

Proses pembentukan matriks TF-IDF dilakukan menggunakan fungsi fit_transform() pada data latih dan transform() pada data uji. Proses fit() hanya dilakukan terhadap data latih untuk membentuk kosakata (vocabulary) dan menghitung bobot TF-IDF, sedangkan data uji hanya ditransformasikan menggunakan kosakata yang telah diperoleh dari data latih.

```
X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)
X_test_tfidf = tfidf_vectorizer.transform(X_test)
```

Gambar 3.20 Transformasi Fitur TF-IDF

Setelah proses pembobotan selesai, objek TfidfVectorizer disimpan menggunakan pustaka Joblib agar dapat digunakan kembali pada proses klasifikasi di aplikasi web tanpa perlu melakukan pelatihan ulang.

```
VECTORIZER_PATH = f'{BASE_DIR}/models/sklearn/tfidf_vectorizer.pkl'
joblib.dump(tfidf_vectorizer, VECTORIZER_PATH)
```

Gambar 3.21 Penyimpanan *TF-IDF Vectorizer*

3.2.3.7 Pembagian *Dataset*

Tahap pembagian *dataset* dilakukan setelah seluruh dokumen hasil transkripsi selesai melalui proses *Preprocessing*. *Dataset* bersih dibaca dari berkas *dataset*_clean.csv, kemudian data yang tidak memiliki nilai pada atribut teks_bersih maupun label_id dihapus menggunakan fungsi dropna(). Atribut teks_bersih digunakan sebagai data masukan (*feature*), sedangkan atribut label_id digunakan sebagai target kelas (label) pada proses pelatihan model.

```
df = pd.read_csv(f'{BASE_DIR}/data/dataset/dataset_clean.csv', encoding='utf-8')
df = df.dropna(subset=['teks_bersih', 'label_id'])

X = df['teks_bersih'].values
y = df['label_id'].values.astype(int)
```

Gambar 3.22 Persiapan *Dataset*

Proses pembagian *dataset* dilakukan menggunakan fungsi train_test_split() dari pustaka Scikit-learn. *Dataset* dibagi menjadi 80% data latih dan 20% data uji menggunakan parameter test_size=0.2. Parameter random_state=42 digunakan untuk menghasilkan pembagian data yang konsisten pada setiap proses pelatihan. Parameter stratify=y diterapkan untuk mempertahankan proporsi jumlah data pada setiap kategori sehingga distribusi kelas pada data latih dan data uji tetap seimbang.

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Gambar 3.23 Pembagian *Dataset*

Script menampilkan jumlah sampel pada data latih dan data uji sebagai verifikasi bahwa proses pembagian *dataset* telah berhasil dilakukan.

```
print(f"[INFO] Dataset terbagi untuk pemodelan:")
print(f"   - Sampel Data Latih (Training) : {len(X_train)}")
print(f"   - Sampel Data Uji (Testing)    : {len(X_test)}")
```

Gambar 3.24 Menampilkan Jumlah Data Latih dan Data Uji

Hasil pembagian *dataset* menghasilkan 892 dokumen sebagai data latih (80%) dan 223 dokumen sebagai data uji (20%) dari total 1.115 dokumen yang digunakan pada penelitian. Data latih dan data uji tersebut digunakan pada tahap ekstraksi fitur TF-IDF.

3.2.4 *Modeling*

Tahap *modeling* dilakukan setelah proses ekstraksi fitur TF-IDF selesai. Tahap ini meliputi konfigurasi model klasifikasi, proses pelatihan menggunakan *Stratified 5-Fold Cross Validation*, serta penyimpanan model hasil pelatihan untuk digunakan pada proses evaluasi dan implementasi aplikasi.

3.2.4.1 Konfigurasi Model *Machine Learning*

Tahap konfigurasi model dilakukan dengan mendefinisikan algoritma klasifikasi yang digunakan pada penelitian ke dalam variabel MODELS berbentuk dictionary. Penelitian ini menggunakan dua algoritma klasifikasi, yaitu *Multinomial Naive Bayes* dan *Support Vector Machine* (Linear). Model *Support Vector Machine* diimplementasikan menggunakan LinearSVC yang dibungkus dengan CalibratedClassifierCV sehingga model dapat menghasilkan nilai probabilitas pada saat proses prediksi. Parameter alpha=1.0 digunakan pada model *Multinomial Naive Bayes*, sedangkan model *Support Vector Machine* menggunakan parameter C=1.0, max_iter=10000, dan random_state=42.

```
from sklearn.calibration import CalibratedClassifierCV
from sklearn.svm import LinearSVC
from sklearn.naive_bayes import MultinomialNB

MODELS = {
    'Naive Bayes': MultinomialNB(alpha=1.0),

    'SVM (Linear)': CalibratedClassifierCV(
        LinearSVC(
            C=1.0,
            max_iter=10000,
            random_state=42
        )
    )
}
```

Gambar 3.25 Konfigurasi Model *Machine Learning*

Script menampilkan nama model yang berhasil dikonfigurasi sebagai verifikasi bahwa proses inisialisasi model telah selesai dilakukan.

```
print(f"[OK] Model ML Klasik berhasil dikonfigurasi: {list(MODELS.keys())}")
```

Gambar 3.26 Verifikasi Konfigurasi Model

3.2.4.2 Pelatihan Model Menggunakan *Stratified 5-Fold Cross Validation*

Tahap pelatihan model diawali dengan inisialisasi StratifiedKFold menggunakan 5-fold. Parameter shuffle=True digunakan untuk mengacak urutan data sebelum proses pembagian fold, sedangkan random_state=42 digunakan agar proses pelatihan menghasilkan pembagian data yang konsisten pada setiap eksekusi program.

```
skf = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)
```

Gambar 3.27 Inisialisasi *Stratified 5-Fold Cross Validation*

Metrik evaluasi yang digunakan selama proses *cross validation* terdiri atas *Accuracy*, *Precision*, *Recall*, dan F1-Score. Seluruh metrik disimpan ke dalam variabel SCORING dan digunakan pada proses evaluasi masing-masing model.

```
SCORING = [
    'accuracy',
    'precision_weighted',
    'recall_weighted',
    'f1_weighted'
]
```

Gambar 3.28 Konfigurasi Metrik Evaluasi

Pelatihan model dilakukan menggunakan perulangan for sehingga setiap model yang terdapat pada variabel MODELS diproses secara otomatis. Fungsi cross_validate() digunakan untuk melakukan pelatihan dan validasi menggunakan data latih hasil ekstraksi fitur TF-IDF dengan skema *Stratified 5-Fold Cross Validation*.

```
for model_name, model in MODELS.items():

    cv_results = cross_validate(
        model,
        X_train_tfidf,
        y_train,
        cv=skf,
        scoring=SCORING,
        return_train_score=False,
        n_jobs=-1
    )
```

Gambar 3.29 Proses Pelatihan Model

Nilai rata-rata *Accuracy*, *Precision*, *Recall*, F1-Score, simpangan baku F1-Score(Std F1), serta waktu pelatihan setiap model dihitung dan disimpan ke dalam variabel cv_results_summary sebagai hasil proses *cross validation.*

```
summary = {
    'Accuracy'  : cv_results['test_accuracy'].mean(),
    'Precision' : cv_results['test_precision_weighted'].mean(),
    'Recall'    : cv_results['test_recall_weighted'].mean(),
    'F1-Score'  : cv_results['test_f1_weighted'].mean(),
    'Std F1'    : cv_results['test_f1_weighted'].std(),
    'Waktu (s)' : round(elapsed_time, 2)
}
```

Gambar 3.30 Rekapitulasi Hasil *Cross Validation*

3.2.4.3 Pelatihan Model Akhir dan Penyimpanan Model

Kedua model dilatih menggunakan data latih (*training set)* yang telah dipersiapkan pada tahap sebelumnya. Proses pelatihan bertujuan agar setiap algoritma dapat mempelajari hubungan antara fitur TF-IDF dan label kategori. Hasil pelatihan berupa model yang mampu mengklasifikasikan topik *podcast* berdasarkan pola yang telah dipelajari.

```
model.fit(X_train_tfidf, y_train)
trained_models[model_name] = model
```

Gambar 3.31 Pelatihan Model Akhir

Objek model yang telah selesai dilatih disimpan menggunakan pustaka Joblib ke dalam direktori models/sklearn. Penyimpanan model dilakukan agar model dapat digunakan kembali pada proses implementasi aplikasi tanpa perlu melakukan pelatihan ulang.

```
safe_name = model_name.lower().replace(' ', '_').replace('(', '').replace(')', '')
model_path = f'{BASE_DIR}/models/sklearn/{safe_name}_model.pkl'

joblib.dump(model, model_path)
```

Gambar 3.32 Penyimpanan Model

Script menampilkan lokasi penyimpanan setiap model sebagai verifikasi bahwa proses ekspor model telah berhasil dilakukan.

```
print(f"[INFO] Objek model berhasil diekspor ke: {model_path}")
```

Gambar 3.33 Verifikasi Penyimpanan Model

3.2.5 *Evaluation*

Tahap evaluasi dilakukan menggunakan data uji (*hold-out test set*) yang telah dipisahkan pada proses pembagian *dataset*. Evaluasi bertujuan untuk mengukur performa masing-masing model klasifikasi berdasarkan hasil prediksi terhadap data uji serta menentukan model dengan performa terbaik.

3.2.5.1 Pengujian Model pada Data Uji

Tahap pengujian dilakukan terhadap seluruh model yang telah selesai dilatih menggunakan fungsi predict(). Hasil prediksi setiap model dibandingkan dengan label aktual pada data uji untuk menghitung nilai *Accuracy*, *Precision*, *Recall*, dan F1-Score. Nilai setiap metrik disimpan ke dalam variabel test_results sehingga dapat digunakan pada proses perbandingan performa model.

```
for model_name, model in trained_models.items():

    y_pred = model.predict(X_test_tfidf)

    test_results[model_name] = {
        'Accuracy'  : accuracy_score(y_test, y_pred),
        'Precision' : precision_score(y_test, y_pred, average='weighted'),
        'Recall'    : recall_score(y_test, y_pred, average='weighted'),
        'F1-Score'  : f1_score(y_test, y_pred, average='weighted'),
    }
```

Gambar 3.34 Pengujian Model pada Data Uji

Script menghasilkan *classification report* untuk setiap model sehingga nilai

evaluasi masing-masing kategori dapat ditampilkan secara rinci.

```
print(classification_report(
    y_test,
    y_pred,
    target_names=label_names_cap
))
```

Gambar 3.35 *Classification Report*

Hasil evaluasi kemudian direkap ke dalam DataFrame sehingga memudahkan proses perbandingan performa seluruh model. Rekapitulasi tersebut juga diekspor ke dalam format CSV sebagai dokumentasi hasil pengujian.

```
df_test_results = pd.DataFrame(test_results).T.round(4)

df_test_results.to_csv(report_csv_path)
```

Gambar 3.36 Rekapitulasi Hasil Evaluasi

3.2.5.2 Visualisasi *Confusion Matrix*

Tahap visualisasi dilakukan menggunakan *confusion matrix* untuk menampilkan hasil prediksi setiap model terhadap seluruh kategori *podcast*. Script membentuk *confusion matrix* menggunakan fungsi confusion_matrix(), kemudian menampilkan hasilnya dalam bentuk heatmap menggunakan pustaka Seaborn. Setiap *confusion matrix* ditempatkan pada subplot yang berbeda sehingga seluruh model dapat diamati dalam satu tampilan.

```
y_pred = model.predict(X_test_tfidf)

cm = confusion_matrix(
    y_test,
    y_pred
)
```

Gambar 3.37 Pembentukan *Confusion Matrix*

Visualisasi *confusion matrix* menampilkan jumlah data yang diprediksi dengan benar maupun salah pada setiap kategori. Nilai *F1-Score* masing-masing model juga ditampilkan pada judul grafik sebagai informasi tambahan selama proses evaluasi.

```
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=label_names_cap,
    yticklabels=label_names_cap,
    ax=ax
)
```

Gambar 3.38 Pembentukan *Confusion Matrix*

3.2.5.3 Perbandingan Performa Model

Tahap perbandingan dilakukan dengan menghitung kembali nilai *Accuracy*, *Precision*, *Recall*, dan *F1-Score* dari setiap model menggunakan data uji. Nilai seluruh metrik disimpan ke dalam variabel comparison_data, kemudian diubah menjadi DataFrame agar dapat dibandingkan secara langsung.

```
comparison_data[model_name] = {
    'Accuracy'  : round(accuracy_score(y_test, y_pred), 4),
    'Precision' : round(precision_score(y_test, y_pred, average='weighted'), 4),
    'Recall'    : round(recall_score(y_test, y_pred, average='weighted'), 4),
    'F1-Score'  : round(f1_score(y_test, y_pred, average='weighted'), 4),
}
```

Gambar 3.39 Perbandingan Performa Model

DataFrame hasil perbandingan diurutkan berdasarkan nilai *F1-Score* tertinggi sehingga model dengan performa terbaik dapat diidentifikasi secara otomatis.

```
df_comparison = pd.DataFrame(comparison_data).T

df_comparison = df_comparison.sort_values(
    'F1-Score',
    ascending=False
)
```

Gambar 3.40 Pengurutan Performa Model

Script menentukan model terbaik menggunakan fungsi idxmax() terhadap kolom F1-Score. Pemilihan model terbaik didasarkan pada nilai *F1-Score*, bukan

Accutacy. *Accuracy* hanya menunjukan satu nilai rata-rata secara keseluruhan sehingga dapat menutupi kekurangan model pada kategori tertentu. *F1-Score* merupakan gabungan dari *precision* dan *recall*, nilai *F1-Score* mampu menunjukan performa model secara merata di setiap kategori. Nama model dan nilai *F1-Score* tertinggi kemudian ditampilkan sebagai hasil akhir proses evaluasi.

```
best_model_name = df_comparison['F1-Score'].idxmax()

best_f1 = df_comparison.loc[
    best_model_name,
    'F1-Score'
]
```

Gambar 3.41 Penentuan Model Terbaik

Hasil perbandingan seluruh model diekspor ke dalam format CSV dan divisualisasikan dalam bentuk grafik batang sehingga perbedaan performa setiap model dapat diamati dengan lebih mudah.

```
ax.bar(
    x + offset,
    values,
    width,
    label=model_name,
    color=color,
    alpha=0.85
)
```

Gambar 3.42 Visualisasi Perbandingan Model.

3.2.6 *Deployment*

Tahap *deployment* dilakukan dengan mengintegrasikan model klasifikasi terbaik, *TF-IDF Vectorizer*, konfigurasi label, dan aplikasi berbasis Flask menjadi sebuah sistem klasifikasi topik *podcast* yang dapat diakses melalui web. Proses *deployment* meliputi ekspor model terbaik, inisialisasi direktori aplikasi, pembangunan aplikasi Flask, serta publikasi aplikasi menggunakan Ngrok.

3.2.6.1 Ekspor Model Terbaik

Tahap pertama *deployment* dilakukan dengan mengekspor model klasifikasi terbaik berdasarkan hasil evaluasi. Nama model terbaik diperoleh secara otomatis dari variabel best_model_name, kemudian objek model, *TF-IDF Vectorizer*, serta

konfigurasi label disimpan agar dapat digunakan kembali oleh aplikasi web tanpa melakukan proses pelatihan ulang.

```
BEST_MODEL_NAME = best_model_name

best_model_obj = trained_models[BEST_MODEL_NAME]
```

Gambar 3.43 Menentukan Model Terbaik

Model terbaik disimpan menggunakan pustaka Joblib ke dalam direktori models/sklearn. *TF-IDF Vectorizer* juga diekspor pada direktori yang sama sehingga proses ekstraksi fitur pada aplikasi menggunakan konfigurasi yang identik dengan proses pelatihan.

```
joblib.dump(
    best_model_obj,
    f'{BASE_DIR}/models/sklearn/best_model.pkl'
)

joblib.dump(
    tfidf_vectorizer,
    f'{BASE_DIR}/models/sklearn/tfidf_vectorizer.pkl'
)
```

Gambar 3.44 Ekspor Model dan *TF-IDF Vectorizer*

Konfigurasi label diekspor ke dalam berkas label_config.json yang berisi pasangan label2id, id2label, serta nama model terbaik. Berkas ini digunakan oleh aplikasi Flask untuk mengubah hasil prediksi numerik menjadi nama kategori *podcast*.

```
label_config = {
    'label2id': label2id,
    'id2label': id2label,
    'best_model': BEST_MODEL_NAME
}

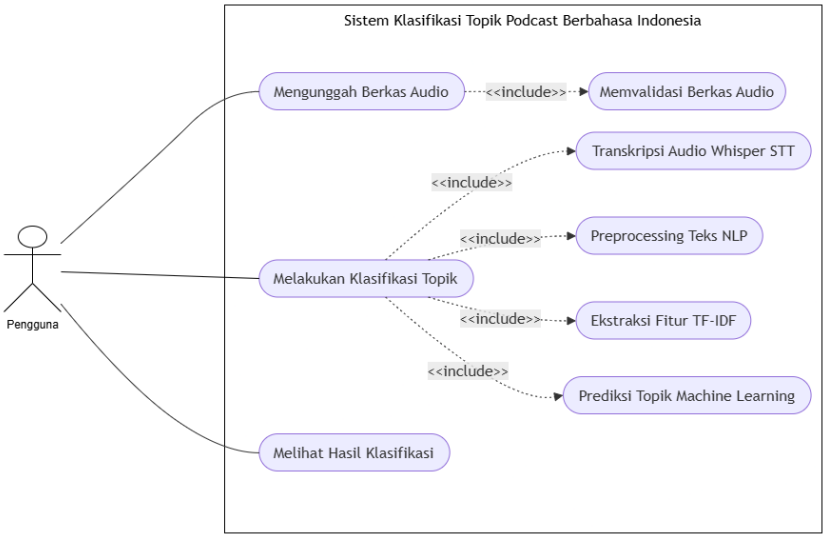
with open(config_json_path,'w',encoding='utf-8') as f_cfg:
    json.dump(
        label_config,
        f_cfg,
        indent=2,
        ensure_ascii=False
    )
```

Gambar 3.45 Penyimpanan Konfigurasi Label

Laporan klasifikasi model terbaik kemudian diekspor dalam format TXT dan CSV sebagai dokumentasi hasil evaluasi.

3.2.6.2 *Use Case Diagram*

Use case diagram disusun untuk menggambarkan kebutuhan fungsional sistem klasifikasi topik *podcast* berbahasa Indonesia dari sudut pandang pengguna sebelum dilakukan perancangan antarmuka dan implementasi aplikasi. Diagram ini menunjukkan interaksi antara pengguna dengan sistem serta fungsi-fungsi utama yang tersedia dalam aplikasi. Pada diagram tersebut terdapat satu aktor, yaitu Pengguna, yang berinteraksi dengan tiga *use case* utama, yaitu Mengunggah Berkas Audio, Melakukan Klasifikasi Topik, dan Melihat Hasil Klasifikasi.



Gambar 3.46 *Use Case Diagram* Sistem Klasifikasi Topik *Podcast*

Gambar 3.46 menunjukkan bahwa pengguna memulai proses dengan mengunggah berkas audio *podcast* ke dalam sistem. *Use case* Mengunggah Berkas Audio memiliki relasi *include* dengan Memvalidasi Berkas Audio, yang menunjukkan bahwa setiap berkas audio yang diunggah akan melalui proses validasi format dan kelengkapan sebelum diproses lebih lanjut oleh sistem.

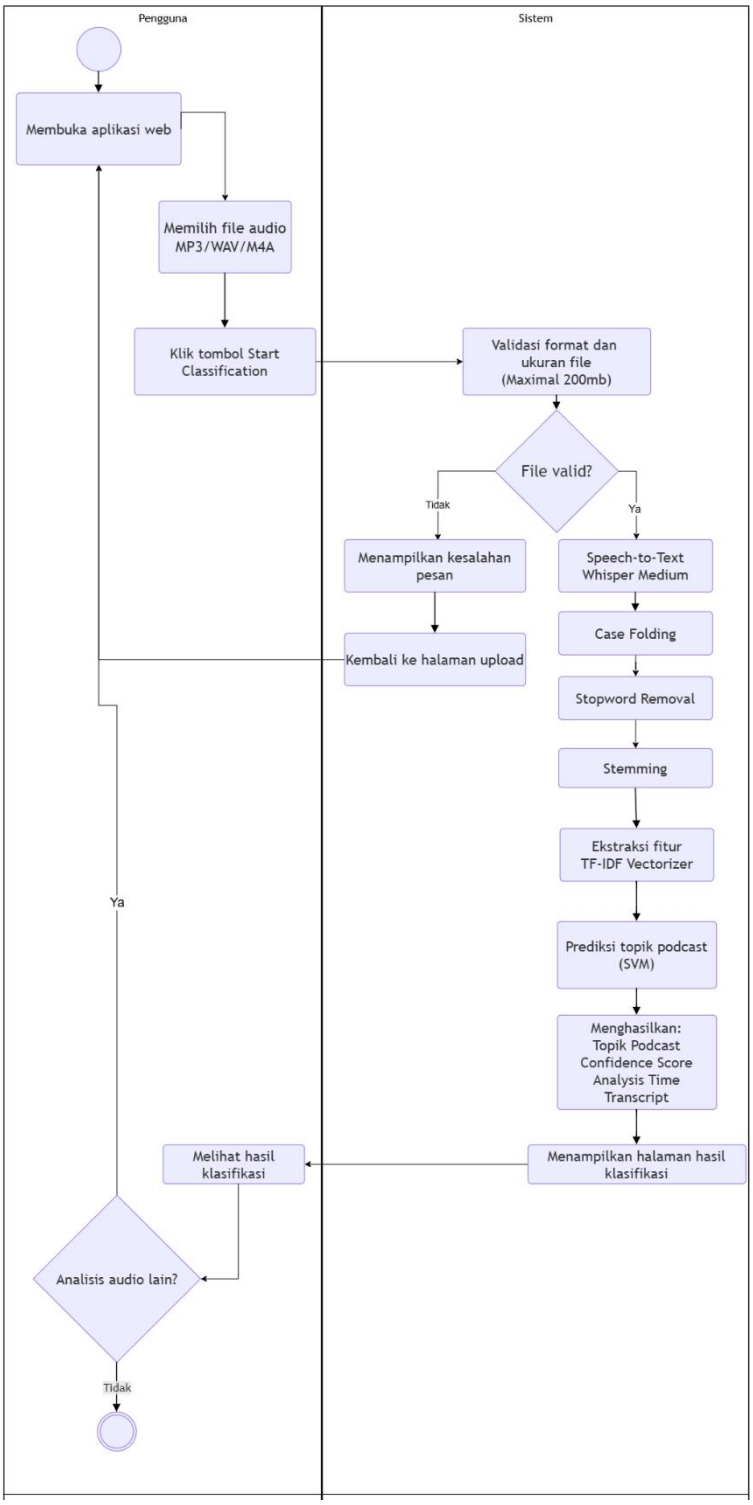
Setelah berkas dinyatakan valid, pengguna dapat menjalankan *use case* Melakukan Klasifikasi Topik. *Use case* ini memiliki beberapa relasi *include* yang menggambarkan tahapan proses yang selalu dijalankan oleh sistem secara berurutan, yaitu Transkripsi Audio Whisper STT, *Preprocessing* Teks NLP, Ekstraksi Fitur TF-IDF, dan Prediksi Topik *Machine Learning.* Proses transkripsi dilakukan menggunakan model Whisper untuk mengubah data audio menjadi teks,

kemudian teks hasil transkripsi diproses melalui tahapan *Natural Language Processing* (NLP), diekstraksi menjadi representasi numerik menggunakan metode TF-IDF, dan selanjutnya diklasifikasikan menggunakan model *Machine Learning* untuk menentukan kategori topik *podcast*.

Setelah seluruh proses klasifikasi selesai dilakukan, pengguna dapat mengakses *use case* Melihat Hasil Klasifikasi untuk memperoleh informasi hasil prediksi topik *podcast*. Halaman hasil menampilkan kategori topik yang diprediksi berdasarkan isi audio *podcast* yang telah diproses oleh sistem. Dengan adanya relasi *include* pada setiap tahapan proses klasifikasi, diagram ini menunjukkan bahwa seluruh proses mulai dari transkripsi hingga prediksi merupakan bagian yang tidak dapat dipisahkan dalam pelaksanaan fungsi klasifikasi topik *podcast*.

3.2.6.3 Rancangan Aplikasi

Pada tahap ini dilakukan perancangan aplikasi web yang digunakan sebagai media implementasi model klasifikasi topik *podcast*. Rancangan aplikasi menggambarkan alur interaksi antara pengguna dengan sistem mulai dari proses unggah berkas audio hingga sistem menampilkan hasil klasifikasi. Perancangan ini bertujuan agar proses implementasi aplikasi dapat dilakukan secara sistematis sesuai dengan kebutuhan penelitian. Rancangan aplikasi ditunjukkan pada Gambar 3.47 berupa *Activity Diagram*



Gambar 3.47 *Activity Diagram* Alur Penggunaan Aplikasi

Diagram aktivitas pada Gambar 3.47 menggambarkan alur penggunaan aplikasi klasifikasi topik *podcast* yang dibagi menjadi dua swimlane, yaitu aktivitas

yang dilakukan oleh pengguna dan proses yang dijalankan oleh sistem. Alur dimulai ketika pengguna membuka aplikasi web dan mengakses halaman Upload Audio. Pengguna kemudian memilih berkas audio yang akan dianalisis dengan format yang didukung, yaitu MP3, WAV, atau M4A. Setelah berkas dipilih, pengguna menekan tombol Start Classification untuk memulai proses klasifikasi topik *podcast*.

Sistem selanjutnya melakukan proses validasi terhadap berkas audio yang diunggah. Validasi dilakukan untuk memastikan bahwa format berkas sesuai dengan ketentuan aplikasi dan ukuran berkas masih berada dalam batas yang diizinkan. Apabila proses validasi tidak berhasil, sistem akan menampilkan pesan kesalahan dan mengarahkan pengguna kembali ke halaman Upload Audio agar pengguna dapat mengunggah berkas yang sesuai. Apabila berkas dinyatakan valid, sistem melanjutkan proses analisis secara otomatis.

Tahap analisis diawali dengan proses *Speech-to-Text* menggunakan model Whisper Medium untuk mengubah data audio menjadi bentuk teks (*transcript*). Proses transkripsi hanya dilakukan terhadap 10 menit pertama dari audio. Pembatasan ini diterapkan untuk mengurangi waktu komputasi dan penggunaan sumber daya selama proses inferensi, karena model Whisper Medium memerlukan waktu pemrosesan yang relatif tinggi untuk audio berdurasi panjang. Teks hasil transkripsi selanjutnya diproses menggunakan tahapan *Natural Language Processing* (NLP) yang meliputi *case folding*, *stopword removal*, dan *Stemming* sehingga diperoleh teks yang lebih bersih dan siap digunakan pada proses klasifikasi.

Teks hasil *Preprocessing* kemudian diubah menjadi representasi numerik menggunakan metode *TF-IDF Vectorizer.* Representasi numerik tersebut menjadi masukan bagi model klasifikasi yang telah dibangun pada tahap pemodelan untuk melakukan prediksi terhadap kategori topik *podcast*.

Berdasarkan hasil prediksi tersebut, sistem menghasilkan informasi berupa topik *podcast*, *confidence score*, *analysis time*, dan *transcript* hasil proses *Speech-to-Text*. Informasi tersebut kemudian ditampilkan pada halaman hasil klasifikasi sehingga pengguna dapat mengetahui hasil analisis yang dilakukan oleh sistem.

Pada tahap akhir, pengguna dapat memilih untuk melakukan analisis terhadap berkas audio lain. Apabila pengguna memilih untuk melakukan analisis kembali, aplikasi akan mengarahkan pengguna ke halaman Upload Audio sehingga proses dapat diulang dari awal. Sebaliknya, apabila pengguna tidak melakukan analisis ulang, maka alur penggunaan aplikasi berakhir.

3.2.6.4 Implementasi Aplikasi

Tahap implementasi dilakukan dengan membangun aplikasi menggunakan *framework* Flask pada berkas app.py. Aplikasi mengintegrasikan model *Machine Learning* hasil pelatihan dengan antarmuka berbasis web sehingga proses klasifikasi topik *podcast* dapat dilakukan secara otomatis. Implementasi meliputi proses pemuatan model, pengaturan routing aplikasi, penerimaan masukan audio, proses transkripsi, *Preprocessing*, ekstraksi fitur menggunakan TF-IDF, serta proses klasifikasi.

Tahap pemuatan model dilakukan pada saat aplikasi dijalankan. Model Whisper Medium digunakan untuk proses *Speech-to-Text*, sedangkan *TF-IDF Vectorizer* dan model klasifikasi dimuat menggunakan pustaka Joblib. Konfigurasi label dimuat dari berkas label_config.json agar hasil prediksi dapat ditampilkan dalam bentuk nama kategori *podcast*.

```
vectorizer = joblib.load("./model/tfidf_vectorizer.pkl")
classifier = joblib.load("./model/best_model.pkl")

whisper_model = whisper.load_model("medium")
```

Gambar 3.48 Memuat Model

Routing aplikasi diimplementasikan menggunakan *framework* Flask. Endpoint / digunakan untuk menampilkan halaman utama aplikasi, sedangkan endpoint /predict digunakan untuk menerima permintaan klasifikasi melalui metode POST.

```python
@app.route("/")
def index():
    return render_template("index.html")

@app.route(
    "/predict",
    methods=["POST"]
)
def predict():
```

Gambar 3.49 Routing Aplikasi

Penerimaan masukan dilakukan melalui endpoint /predict. Sistem memeriksa keberadaan berkas audio yang diunggah oleh pengguna menggunakan metode POST. Berkas yang diterima divalidasi untuk memastikan bahwa data audio tersedia sebelum proses klasifikasi dijalankan. Berkas audio yang valid disimpan sementara ke dalam direktori uploads sebagai masukan pada proses transkripsi.

```python
@app.route("/predict", methods=["POST"])
def predict():

    if "audio" not in request.files:
        return jsonify({"error": "Tidak ada berkas audio"}), 400

    audio_file = request.files["audio"]

    filepath = os.path.join(
        UPLOAD_FOLDER,
        audio_file.filename
    )

    audio_file.save(filepath)
```

Gambar 3.50 Penerimaan Masukan Audio

Proses transkripsi dilakukan menggunakan model Whisper Medium. Berkas audio dibaca terlebih dahulu, kemudian durasi pemrosesan dibatasi hingga 10 menit pertama. Pembatasan tersebut membantu meningkatkan efisiensi waktu komputasi tanpa mengurangi informasi yang diperlukan dalam proses klasifikasi. Fungsi transcribe() menghasilkan teks yang selanjutnya digunakan pada tahap *Preprocessing* dan klasifikasi.

```
audio = whisper.load_audio(filepath)

max_samples = 600 * 16000

if len(audio) > max_samples:
    audio = audio[:max_samples]

result = whisper_model.transcribe(
    audio,
    language="id",
    fp16=False
)

transcript = result["text"].strip()
```

Gambar 3.51 Transkripsi Audio

Teks hasil transkripsi diproses menggunakan fungsi preprocess() yang meliputi tahapan *case folding*, *text cleaning*, tokenisasi, *stopword removal*, dan *Stemming*. Hasil *Preprocessing* kemudian diubah menjadi representasi numerik menggunakan metode *TF-IDF Vectorizer*. Representasi numerik tersebut selanjutnya digunakan sebagai masukan bagi model classifier untuk menghasilkan prediksi kategori topik *podcast*.

```
clean_text = preprocess(transcript)

vec = vectorizer.transform([clean_text])

pred_id = classifier.predict(vec)[0]
pred_label = id2label[pred_id]
```

Gambar 3.52 *Preprocessing* dan Klasifikasi

3.2.6.5 Pengujian Aplikasi

Tahap pengujian aplikasi dilakukan menggunakan metode *Black Box Testing* untuk mengevaluasi fungsionalitas seluruh fitur pada aplikasi web klasifikasi topik *podcast*. Pengujian dilakukan berdasarkan sejumlah skenario yang dirancang untuk memastikan bahwa setiap komponen sistem, mulai dari fitur unggah berkas audio (MP3, WAV, dan M4A), proses transkripsi menggunakan Whisper Medium, pra-pemrosesan teks (*Natural Language Processing*), ekstraksi fitur TF-IDF, proses klasifikasi menggunakan model *Support Vector Machine* (SVM) Linear, hingga penyajian hasil klasifikasi pada antarmuka pengguna, dapat berjalan sesuai dengan kebutuhan fungsional dan menghasilkan keluaran yang

diharapkan.

4. HASIL DAN PEMBAHASAN

4.1 *Business Understanding*

Penelitian ini bertujuan membangun sistem klasifikasi topik konten *podcast* berbahasa Indonesia secara otomatis dengan memanfaatkan teknologi *Speech-to-Text* (STT), *Natural Language Processing* (NLP), dan *Machine Learning*. Permasalahan yang menjadi dasar penelitian adalah keterbatasan identifikasi topik *podcast* berdasarkan metadata, seperti judul dan deskripsi video. Informasi tersebut belum selalu merepresentasikan keseluruhan pembahasan yang terdapat dalam audio *podcast*.

Tahapan pengolahan data dilakukan secara berurutan mulai dari konversi audio menjadi teks, *Preprocessing* teks, ekstraksi fitur menggunakan Term Frequency-Inverse Document Frequency (TF-IDF), hingga klasifikasi menggunakan algoritma *Machine Learning*. Model dengan kinerja terbaik selanjutnya diintegrasikan ke dalam aplikasi web berbasis Flask. Aplikasi tersebut memungkinkan pengguna mengunggah berkas audio *podcast* dan memperoleh hasil prediksi topik berdasarkan isi audio.

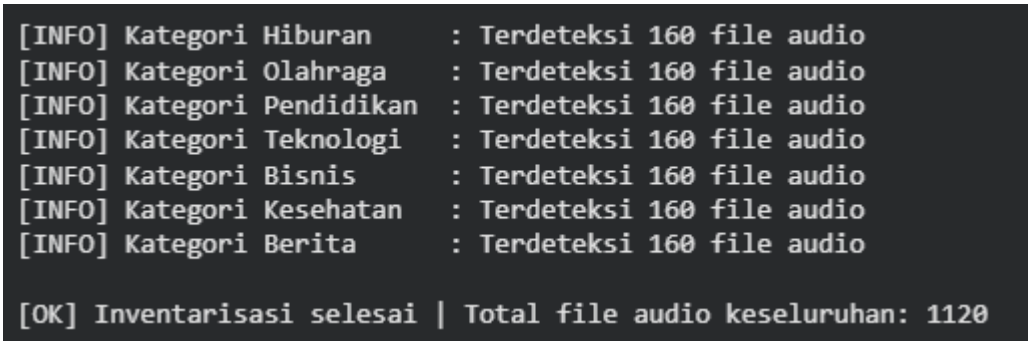
Pembahasan Evaluasi sistem dilakukan menggunakan metrik *Accuracy*, *Precision, Recall*, dan *F1-score* untuk mengetahui kemampuan model dalam mengklasifikasikan data *podcast*. Perbandingan hasil pengujian antaralgoritma disajikan pada Subbab 4.5. Karakteristik dan kondisi *dataset* yang digunakan dalam penelitian dibahas pada Subbab 4.2

4.2 *Data Understanding*

Tahap *Data Understanding* dilakukan untuk mengetahui karakteristik dan kondisi *dataset podcast* yang digunakan dalam penelitian. Pembahasan pada tahap ini meliputi jumlah data, distribusi kategori, serta hasil konversi audio menjadi teks. Informasi tersebut digunakan untuk mengetahui kondisi data sebelum memasuki tahapan pengolahan dan klasifikasi.

4.2.1 *Dataset Podcast*

Dataset penelitian terdiri atas 1.120 berkas audio *podcast* yang terbagi ke dalam tujuh kategori topik, yaitu Hiburan, Olahraga, Pendidikan, Teknologi, Bisnis, Kesehatan, dan Berita. Setiap kategori pada tahap awal terdiri atas 160 berkas audio. Seluruh berkas audio digunakan sebagai masukan pada proses *Speech-to-Text* menggunakan model Whisper. Hasil transkripsi kemudian digunakan sebagai data teks untuk tahap *Preprocessing* dan klasifikasi topik.



Gambar 4.1 Hasil Inventarisasi *Dataset Podcast*

Gambar 4.1 menunjukkan hasil inventarisasi berkas audio berdasarkan kategori topik. Setiap kategori teridentifikasi memiliki 160 berkas audio, sehingga jumlah keseluruhan *dataset* pada tahap awal mencapai 1.120 berkas audio. Distribusi jumlah berkas yang sama pada setiap kategori menunjukkan bahwa *dataset* awal memiliki jumlah sampel yang relatif seimbang antar kelas.

Dataset penelitian terdiri atas 1.120 berkas audio *podcast* yang terbagi ke dalam tujuh kategori topik, yaitu Hiburan, Olahraga, Pendidikan, Teknologi, Bisnis, Kesehatan, dan Berita. Setiap kategori terdiri atas 160 berkas audio pada tahap awal. Distribusi *dataset* awal tersebut digunakan untuk menggambarkan karakteristik dan keseimbangan jumlah data pada setiap kategori. Tabel 4.1 menyajikan distribusi berkas audio berdasarkan kategori topik *podcast***.**

Tabel 4.1 Inventarisasi dan Distribusi *Dataset Podcast*

Kategori Topik	**Berkas Audio Mentah**	**Persentase *Dataset***	**Format Berkas**	**Status Distribusi**
Teknologi	160	14,29%	MP3	Seimbang
Pendidikan	160	14,29%	MP3	Seimbang
Kesehatan	160	14,29%	MP3	Seimbang
Hiburan	160	14,29%	MP3	Seimbang

Kategori Topik	**Berkas Audio Mentah**	**Persentase *Dataset***	**Format Berkas**	**Status Distribusi**
Berita	160	14,29%	MP3	Seimbang
Bisnis	160	14,29%	MP3	Seimbang
Olahraga	160	14,29%	MP3	Seimbang
Total	1.120	100,00%	MP3	

Tabel 4.1 menunjukkan bahwa setiap kategori memiliki jumlah data yang sama, yaitu 160 berkas audio atau 14,29% dari total 1.120 berkas. Distribusi tersebut menunjukkan bahwa *dataset* awal memiliki komposisi kelas yang seimbang. *Dataset* selanjutnya digunakan sebagai masukan pada proses *Speech-to-Text* untuk memperoleh data teks yang akan diproses pada tahap berikutnya.

4.2.2 Hasil *Speech-to-Text*

Model OpenAI Whisper varian *medium* digunakan untuk mengonversi audio *podcast* menjadi teks transkrip. Proses transkripsi dilakukan terhadap seluruh 1.120 berkas audio yang telah dikumpulkan dari tujuh kategori topik. Konfigurasi transkripsi menggunakan parameter bahasa Indonesia (language='id') agar model mengenali ucapan dalam bahasa yang sesuai dengan karakteristik *dataset* penelitian.

Hasil proses *Speech-to-Text* menunjukkan bahwa 1.120 berkas audio berhasil ditranskripsikan dan tidak terdapat berkas yang gagal diproses. Setiap hasil transkripsi menghasilkan teks yang selanjutnya digunakan sebagai masukan pada tahap *Preprocessing* NLP. Proses transkripsi juga menghasilkan informasi jumlah kata pada setiap audio, seperti terlihat pada hasil keluaran yang menunjukkan jumlah kata berbeda sesuai dengan durasi dan isi masing-masing *podcast*.

```
============================================================
[INFO] TRANSKRIPSI SELESAI!
   - Berhasil diproses : 1120 file
   - Gagal diproses    : 0 file
   - Berkas CSV Output :
/content/drive/MyDrive/PodcastClassifier/data/dataset/dataset_transkripsi.csv
============================================================
```

Gambar 4.2 Hasil Proses *Speech-to-Text* Menggunakan *Whisper*

Hasil tersebut menunjukkan bahwa seluruh *dataset* audio berhasil dikonversi menjadi data teks dan dapat dilanjutkan ke tahap *Preprocessing* NLP. Tahap berikutnya dilakukan untuk membersihkan dan menormalisasi teks hasil transkripsi sebelum digunakan dalam proses ekstraksi fitur TF-IDF. Tabel 4.2 memperlihatkan contoh sampel hasil transkripsi audio *podcast* yang dihasilkan oleh model Whisper Medium dari berbagai kategori topik.

Tabel 4.2 Sampel Hasil Transkripsi Audio *Podcast*

Kategori Topik	**Berkas Audio Contoh**	**Sampel Teks Hasil Transkripsi Whisper Medium**
Teknologi	`tek_001.mp3`	"Selamat datang di *podcast* teknologi. Hari ini kita akan membahas perkembangan kecerdasan buatan dan integrasi sistem kecerdasan buatan pada aplikasi modern."
Bisnis	`bis_012.mp3`	"Strategi pemasaran digital dan pengelolaan modal usaha menjadi kunci utama bagi para pelaku UMKM untuk meningkatkan omzet bisnis pada tahun ini."
Kesehatan	`kes_005.mp3`	"Menjaga pola makan sehat serta rutin berolahraga setiap pagi dapat meningkatkan daya tahan tubuh dan mencegah risiko penyakit kronis."

Tabel 4.2 menunjukkan contoh hasil konversi audio *podcast* menjadi teks menggunakan model Whisper *Medium*. Tabel tersebut menampilkan tiga kategori topik, yaitu Teknologi, Bisnis, dan Kesehatan, beserta contoh berkas audio dan sampel teks hasil transkripsinya. Hasil transkripsi menunjukkan bahwa audio berhasil dikonversi menjadi teks yang memuat informasi sesuai dengan pembahasan pada masing-masing kategori. Transkrip pada kategori Teknologi memuat pembahasan mengenai perkembangan kecerdasan buatan dan integrasi sistem, kategori Bisnis membahas strategi pemasaran digital dan pengelolaan modal usaha, sedangkan kategori Kesehatan memuat pembahasan mengenai pola makan sehat dan aktivitas olahraga. Hasil transkripsi tersebut selanjutnya

digunakan sebagai data teks untuk tahap *Preprocessing* dan proses klasifikasi topik.

4.3 *Data Preparation*

Tahap *Data Preparation* dilakukan untuk mempersiapkan data teks hasil transkripsi agar dapat digunakan pada proses klasifikasi topik *podcast*. Tahapan ini mencakup *Preprocessing* teks, penyaringan data, pembagian data menjadi data latih dan data uji, serta ekstraksi fitur menggunakan *Term Frequency-Inverse Document Frequency* (TF-IDF). Hasil dari tahap ini berupa data teks yang telah diproses dan representasi numerik yang siap digunakan pada tahap *Modeling*.

4.3.1 Hasil *Preprocessing*

Tahap pra-pemrosesan teks (*Preprocessing*) dilakukan untuk membersihkan hasil transkripsi audio dari unsur yang tidak diperlukan serta menyeragamkan bentuk teks. Proses ini bertujuan menghasilkan data teks yang lebih terstruktur dan sesuai untuk tahap ekstraksi fitur menggunakan TF-IDF. Pra-pemrosesan diterapkan menggunakan fungsi preprocess_*text*() melalui enam tahapan utama sebagai berikut.

1) ***Case folding*:** Seluruh karakter dalam teks diubah menjadi huruf kecil (*lowercase*). Tahap ini bertujuan menyeragamkan bentuk penulisan kata sehingga kata yang sama dengan perbedaan huruf kapital tidak dianggap sebagai term yang berbeda.
2) ***Text Cleaning*:** Menghapus URL, e-mail, angka, simbol/tanda baca, huruf tunggal, dan spasi berlebihan menggunakan ekspresi reguler (regex).
3) ***Tokenizing*:** Teks yang telah dibersihkan dipecah menjadi sejumlah *token* berupa kata menggunakan fungsi *word_tokenize* dari pustaka NLTK. Setiap *token* menjadi unit kata yang dapat diproses pada tahap *stopword removal* dan *Stemming*.
4) ***Stopword removal*:** *Token* yang dianggap kurang memberikan informasi dalam proses klasifikasi dihapus. Daftar *stopword* mencakup kata umum dari Sastrawi dan NLTK serta *custom stopwords* yang disesuaikan dengan karakteristik percakapan *podcast*. *Custom stopwords* tersebut meliputi kata

sapaan seperti “halo”, “selamat”, dan “datang”, istilah umum seperti “*podcast*” dan “episode”, kata jeda seperti “ya”, “eh”, “gitu”, dan “kayak”, serta kata percakapan seperti “gue”, “lu”, dan “guys”. *Token* dengan panjang kurang dari atau sama dengan dua karakter juga dihapus.

5) ***Stemming*:** Setiap kata berimbuhan diubah menjadi bentuk dasar menggunakan algoritma *Stemming* yang tersedia pada pustaka PySastrawi. Tahap ini bertujuan menyamakan kata yang memiliki bentuk berbeda tetapi berasal dari kata dasar yang sama. Contohnya, kata “membahas” diubah menjadi “bahas” dan “perkembangan” menjadi “kembang”.
6) **Penghapusan *Token* Berulang Berturut-turut:** *Token* yang sama dan muncul secara berurutan dihapus untuk mengurangi pengulangan kata akibat karakteristik ucapan lisan. Tahap ini terutama ditujukan untuk menangani pengulangan kata atau gagap yang dapat muncul pada hasil transkripsi audio.

Tabel 4.3 memperlihatkan contoh transformasi teks pada setiap tahapan pra-pemrosesan dari teks transkrip mentah hingga menghasilkan sekumpulan kata dasar bersih.

Tabel 4.3 Contoh Hasil Transformasi Pra-pemrosesan Teks Per Tahap

Tahapan *Preprocessing*	**Teks Hasil Pemrosesan**
Teks Mentah (Whisper)	"Halo selamat datang di *podcast* teknologi, hari ini kita akan membahas perkembangan kecerdasan buatan yang semakin pesat berkembang di Indonesia!"
1. *Case folding*	"halo selamat datang di *podcast* teknologi, hari ini kita akan membahas perkembangan kecerdasan buatan yang semakin pesat berkembang di indonesia!"
2. *Text Cleaning*	"halo selamat datang di *podcast* teknologi hari ini kita akan membahas perkembangan kecerdasan buatan yang semakin pesat berkembang di indonesia"
3. *Tokenizing*	`['halo', 'selamat', 'datang', 'di', '*podcast*', 'teknologi', 'hari', 'ini', 'kita', 'akan', 'membahas', 'perkembangan',

Tahapan *Preprocessing*	**Teks Hasil Pemrosesan**
	'kecerdasan', 'buatan', 'yang', 'semakin', 'pesat', 'berkembang', 'di', 'indonesia']`
4. *Stopword removal*	`['teknologi', 'membahas', 'perkembangan', 'kecerdasan', 'buatan', 'semakin', 'pesat', 'berkembang', 'indonesia']`
5. *Stemming* (PySastrawi)	`['teknologi', 'bahas', 'kembang', 'cerdas', 'buat', 'semakin', 'pesat', 'kembang', 'indonesia']`
6. Hapus *Token* Berulang	"teknologi bahas kembang cerdas buat semakin pesat kembang indonesia"

Tabel 4.3 menunjukkan perubahan bertahap pada teks masukan. Kata umum seperti "di", "hari", "ini", "kita", "akan", "yang", serta kata percakapan lisan *podcast* seperti "halo", "selamat", "datang", dan "*podcast*" berhasil dieliminasi penuh pada tahap *Stopword removal* sesuai konfigurasi `custom_stopwords`. Tahap *Stemming* mengembalikan kata berimbuhan "membahas" menjadi kata dasar "bahas", "perkembangan" menjadi "kembang", "kecerdasan" menjadi "cerdas", dan "buatan" menjadi "buat".

Pada tahap keenam, yaitu penghapusan *token* berulang, sistem memeriksa *token* identik yang muncul secara berurutan. Sampel teks pada Tabel 4.3 tidak memiliki *token* yang berulang secara berdampingan, sehingga hasil pada tahap ini tidak mengalami perubahan dan tetap sama dengan hasil *Stemming*.

Proses *Preprocessing* diterapkan secara otomatis terhadap seluruh data hasil transkripsi menggunakan fungsi preprocess_*text*(). Setelah proses *Preprocessing* dan penyaringan akhir, diperoleh 1.115 sampel yang memenuhi kriteria untuk digunakan pada tahap berikutnya. Hasil pemrosesan disimpan pada kolom teks_bersih dalam berkas *dataset*_transkripsi_clean.csv.

Tabel 4.4 Distribusi *Dataset Podcast* Setelah *Preprocessing*

Kategori Topik	**Berkas Audio Mentah**	***Dataset* Bersih (Final)**	**Persentase *Dataset***	**Format Berkas**	**Status Distribusi**
Teknologi	160	160	14,35%	MP3	Seimbang
Pendidikan	160	160	14,35%	MP3	Seimbang
Kesehatan	160	160	14,35%	MP3	Seimbang
Hiburan	160	160	14,35%	MP3	Seimbang

Kategori Topik	**Berkas Audio Mentah**	***Dataset* Bersih (Final)**	**Persentase *Dataset***	**Format Berkas**	**Status Distribusi**
Berita	160	159	14,26%	MP3	Seimbang
Bisnis	160	158	14,17%	MP3	Seimbang
Olahraga	160	158	14,17%	MP3	Seimbang
Total	1.120	1.115	100,00%	MP3	

Tabel 4.1 menunjukkan distribusi *dataset* sebelum dan setelah proses *Preprocessing*. Kategori Teknologi, Pendidikan, Kesehatan, dan Hiburan masing-masing mempertahankan 160 sampel. Kategori Berita memiliki 159 sampel, sedangkan kategori Bisnis dan Olahraga masing-masing memiliki 158 sampel.

Tabel 4.5 Daftar Sampel Data yang Dieliminasi pada Tahap *Preprocessing*

Video ID	**Judul**	**Label**	**Teks Sampel**
2SBp8ZoOfbI	Live TalkShow Bersama Bapak Marketing Indonesia	bisnis	... ...
od7Ro9mcPdg	Investasi Rp 40 Triliun, Bisnis Online Nasional Semarak	bisnis	Terima kasih sudah menonton Terima kasih sudah menonton Terima kasih sudah menonton Terima kasih sudah menonton Terima k...
cx5fvIeKy1I	KETIKA KEVIN SANJAYA DITENGILIN PEMAIN INGGRIS. BALASAN KEVIN SUNGGUH SADIS, Lawan Minions Geleng	olahraga	2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 29 30 31 32 33 34 35 36 37 38 39 40 41 42 42...
pNMRgxStUb8	YUKE BONGKAR JOHN HERDMAN	olahraga	BUNYOKAY BUNYOKAY

Video ID	Judul	Label	Teks Sampel
	BAWA 11 PERSIB DI TIMNAS, COCOK JUARA AFF		BUNYOKAY BUNYOKAY BUNYOKAY BUNYOKAY BUNYOKAY BUNYOKAY BUNYOKAY BUNYOKAY BUNYOKAY BUNYOKAY BUNYOKAY BUN...
sp003VNBuDk	ProKontra Pencabutan Tiga Tap MPR: Dampaknya bagi Sistem Ketatanegaraan	olahraga	... ...

Jumlah *dataset* akhir sebanyak 1.115 sampel menunjukkan bahwa terdapat lima sampel yang tidak memenuhi kriteria setelah proses penyaringan. *Dataset* akhir tersebut selanjutnya digunakan pada proses pembagian data latih dan data uji serta ekstraksi fitur TF-IDF.

Berdasarkan hasil pemeriksaan terhadap data hasil transkripsi, ditemukan beberapa data yang menghasilkan teks tidak representatif sehingga tidak dapat digunakan dalam proses klasifikasi. Pemeriksaan dilakukan dengan melihat kesesuaian antara hasil transkripsi dengan kebutuhan data tekstual yang akan digunakan pada tahapan *preprocessing* dan ekstraksi fitur TF-IDF. Hasil pemeriksaan terhadap data yang dieliminasi ditunjukkan pada Tabel 4.5.

Berdasarkan Tabel 4.5, terdapat lima data yang memiliki permasalahan pada hasil transkripsinya. Data dengan Video ID 2SBp8ZoOfbI pada kategori bisnis menghasilkan transkripsi berupa deretan simbol titik (...) tanpa informasi linguistik yang dapat merepresentasikan isi audio. Kondisi serupa ditemukan pada Video ID

sp003VNBuDk, yang juga menghasilkan deretan simbol titik. Setelah dilakukan proses pembersihan karakter non-alfabet, kedua hasil transkripsi tersebut tidak menghasilkan kata yang dapat digunakan sebagai fitur sehingga data tidak memenuhi kebutuhan untuk tahapan klasifikasi.

Video ID od7Ro9mcPdg menghasilkan teks berupa pengulangan frasa *"*Terima kasih sudah menonton*"* secara berulang. Hasil tersebut tidak memberikan informasi yang cukup mengenai topik yang seharusnya direpresentasikan oleh data. Penggunaan teks tersebut dalam proses TF-IDF berpotensi menghasilkan fitur yang didominasi oleh kata atau frasa repetitif yang tidak berkaitan dengan karakteristik topik. Data tersebut kemudian dieliminasi agar tidak memberikan noise terhadap proses pembentukan fitur dan klasifikasi.

Pada Video ID cx5fvIeKy1I, hasil transkripsi didominasi oleh rangkaian angka yang berurutan, sedangkan Video ID pNMRgxStUb8 menghasilkan pengulangan frasa yang tidak memiliki makna linguistik yang jelas. Kedua hasil tersebut menunjukkan bahwa teks yang diperoleh tidak dapat merepresentasikan isi audio secara memadai. Penggunaan transkripsi tersebut dalam proses NLP dapat menghasilkan fitur yang tidak relevan terhadap kategori olahraga, sehingga kedua data tersebut dieliminasi.

Berdasarkan hasil pemeriksaan tersebut, kelima data tidak digunakan pada tahap pengolahan selanjutnya. Eliminasi dilakukan untuk memastikan bahwa *dataset* yang digunakan pada proses *preprocessing*, ekstraksi fitur TF-IDF, pelatihan model, dan evaluasi hanya terdiri atas data yang memiliki representasi tekstual yang memadai. Dengan demikian, dari 1.120 data awal, sebanyak 5 data dieliminasi sehingga diperoleh 1.115 data yang digunakan untuk tahapan penelitian berikutnya.

Data pada kolom teks_bersih selanjutnya digunakan sebagai masukan untuk proses pembobotan TF-IDF pada Subbab 4.3.2.

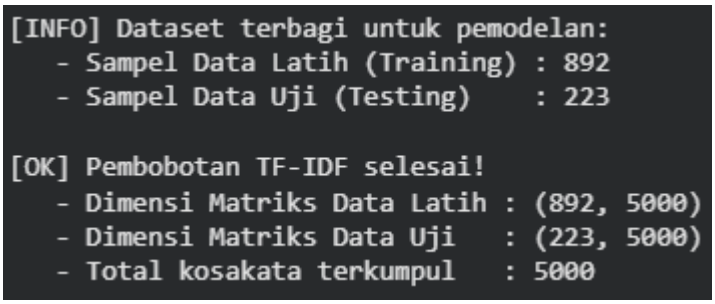
4.3.2 Hasil Ekstraksi Fitur TF-IDF

Setelah proses *Preprocessing* selesai, sebanyak 1.115 sampel pada kolom teks_bersih digunakan untuk proses ekstraksi fitur. Data tersebut telah melalui

pembagian menjadi data latih dan data uji sesuai prosedur yang ditetapkan pada tahap perancangan penelitian. Hasil pembagian menghasilkan 892 sampel data latih dan 223 sampel data uji.

Proses ekstraksi fitur menggunakan metode *Term Frequency-Inverse Document Frequency* (TF-IDF) menghasilkan representasi numerik dari teks *podcast*. Hasil transformasi menunjukkan bahwa matriks data latih memiliki dimensi 892 × 5.000, sedangkan matriks data uji memiliki dimensi 223 × 5.000. Jumlah fitur yang terbentuk adalah 5.000 term yang digunakan sebagai representasi numerik dari teks *podcast*.

Setiap baris pada matriks merepresentasikan satu dokumen *podcast*, sedangkan setiap kolom merepresentasikan fitur berupa *term* hasil ekstraksi TF-IDF. Nilai pada matriks menunjukkan bobot setiap *term* dalam dokumen. Representasi numerik tersebut selanjutnya digunakan sebagai fitur masukan pada tahap *modeling* untuk melatih dan menguji algoritma *Naive Bayes* dan *Support Vector Machine* (SVM) dalam melakukan klasifikasi topik *podcast*.



Gambar 4.3 Hasil Pembentukan Matriks *TF-IDF*

Gambar 4.2 menunjukkan hasil pembagian *dataset* dan proses ekstraksi fitur TF-IDF. Sebanyak 1.115 sampel terbagi menjadi 892 sampel data latih dan 223 sampel data uji. Proses ekstraksi fitur menghasilkan matriks TF-IDF dengan dimensi 892 × 5.000 pada data latih dan 223 × 5.000 pada data uji. Jumlah fitur yang dihasilkan sebanyak 5.000 term sesuai dengan batas maksimum fitur yang ditentukan dalam konfigurasi TF-IDF.

Hasil tersebut menunjukkan bahwa setiap sampel *podcast* telah berhasil direpresentasikan dalam bentuk numerik yang dapat digunakan oleh algoritma klasifikasi. Matriks TF-IDF selanjutnya digunakan sebagai fitur masukan untuk

proses pelatihan dan pengujian model *Naive Bayes* dan *Support Vector Machine* (SVM) pada tahap *modeling*.

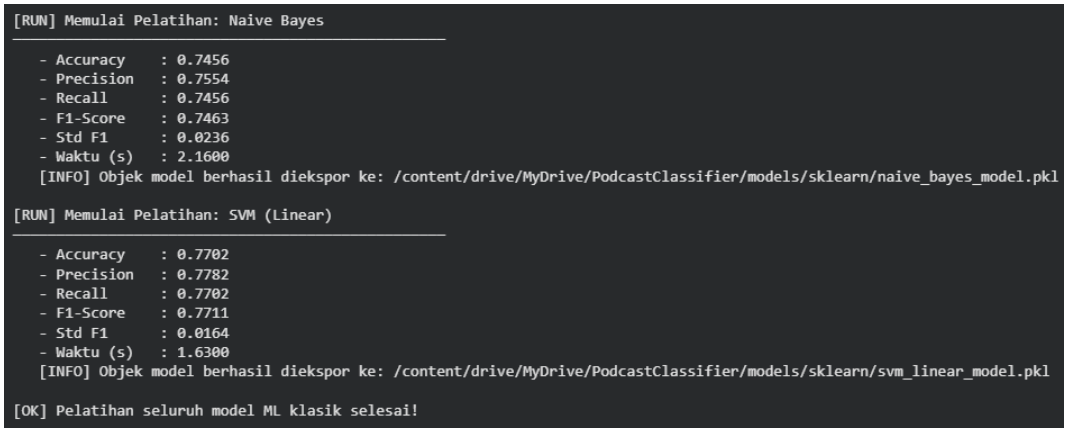
4.4 *Modeling*

Tahap *modeling* merupakan tahap penerapan algoritma klasifikasi terhadap fitur TF-IDF yang telah diperoleh pada tahap *Data Preparation*. Penelitian ini menggunakan dua algoritma, yaitu *Naive Bayes* dan *Support Vector Machine* (SVM). Kedua algoritma dilatih menggunakan data latih dan kemudian digunakan untuk melakukan prediksi terhadap data uji.

Setelah proses ekstraksi fitur TF-IDF selesai, *dataset* yang digunakan pada tahap *modeling* terdiri atas 892 sampel data latih dan 223 sampel data uji. Data latih digunakan untuk membentuk model klasifikasi, sedangkan data uji disiapkan untuk proses pengujian pada tahap *evaluation*.

Penelitian ini menggunakan dua algoritma klasifikasi, yaitu *Naive Bayes* dan *Support Vector Machine* (SVM) Linear. Kedua algoritma menerima matriks TF-IDF sebagai masukan dengan label kategori topik *podcast*. Proses pelatihan dilakukan secara terpisah pada masing-masing algoritma untuk menghasilkan model klasifikasi yang dapat digunakan dalam memprediksi tujuh kategori topik *podcast*.

Hasil pelatihan menunjukkan bahwa model *Naive Bayes* memperoleh *Accuracy* sebesar 0,7456, *Precision* sebesar 0,7554, *Recall* sebesar 0,7456, dan *F1-score* sebesar 0,7463, dengan *Std F1* sebesar 0,0236. Waktu yang diperlukan untuk proses pelatihan model adalah 2,1600 detik. Model SVM Linear memperoleh *Accuracy* sebesar 0,7702, *Precision* sebesar 0,7782, *Recall* sebesar 0,7702, dan *F1-score* sebesar 0,7711, dengan *Std F1* sebesar 0,0164 dan waktu pelatihan sebesar 1,6300 detik. Hasil proses pelatihan kedua model ditunjukkan pada Gambar 4.4.



Gambar 4.4 Hasil Pelatihan Model *Naive Bayes* dan SVM Linear

Setelah proses pelatihan selesai, kedua model berhasil diekspor dan disimpan sebagai berkas model. Model *Naive Bayes* disimpan dengan nama naive_bayes_model.pkl, sedangkan model SVM Linear disimpan dengan nama svm_linear_model.pkl. Berkas model tersebut digunakan pada tahapan berikutnya untuk melakukan pengujian terhadap 223 sampel data uji dan menentukan model yang memiliki kinerja terbaik.

4.5 *Evaluation*

Tahap *evaluation* dilakukan untuk mengetahui kinerja model *Naive Bayes* dan SVM Linear setelah proses pelatihan. Pengujian dilakukan menggunakan 223 sampel data uji yang telah dipisahkan dari data latih. Setiap model menghasilkan prediksi terhadap data uji yang kemudian dibandingkan dengan label aktual.

4.5.1 Perbandingan Kinerja Model

Pengujian Tahap evaluasi dilakukan untuk membandingkan kinerja model *Naive Bayes* dan SVM (Linear) berdasarkan hasil prediksi terhadap 223 sampel data uji. Data uji tidak digunakan dalam proses pelatihan model sehingga dapat digunakan untuk melihat kemampuan model dalam mengklasifikasikan data yang belum diproses selama pelatihan.

Pengujian menghasilkan empat metrik, yaitu *Accuracy, Precision, Recall,* dan *F1-score*. Nilai yang digunakan merupakan hasil perhitungan *weighted average* pada seluruh kategori topik *podcast*. Hasil evaluasi kedua model ditampilkan pada

Tabel 4.5.

Tabel 4.6 Perbandingan Metrik Performa Model Klasifikasi

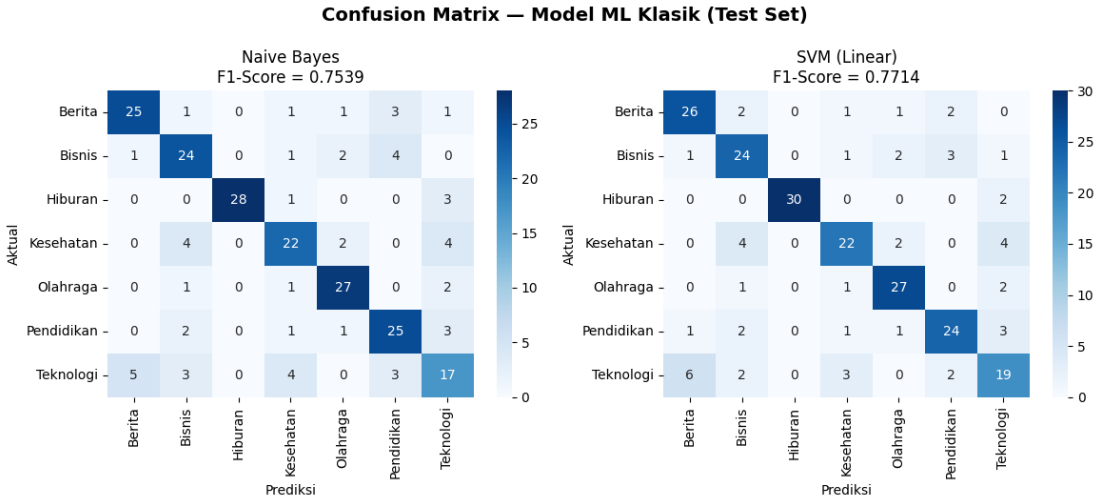
Algoritma Klasifikasi	***Accuracy***	***Precision***	***Recall***	**F1-Score**
Naive Bayes	0,7534	0,7570	0,7534	0,7539
SVM Linear	0,7713	0,7733	0,7713	0,7714

Berdasarkan Berdasarkan Tabel 4.4, model SVM Linear memperoleh nilai evaluasi yang lebih tinggi dibandingkan *Naive Bayes* pada seluruh metrik yang digunakan. SVM Linear memperoleh *Accuracy* sebesar 0,7713, *Precision* sebesar 0,7733, *Recall* sebesar 0,7713, dan *F1-Score* sebesar 0,7714. Sementara itu, *Naive Bayes* memperoleh *Accuracy* sebesar 0,7534, *Precision* sebesar 0,7570, *Recall* sebesar 0,7534, dan *F1-Score* sebesar 0,7539.

Hasil tersebut menunjukkan bahwa SVM Linear memiliki kinerja yang lebih baik dibandingkan *Naive Bayes* pada data uji. Selanjutnya, hasil klasifikasi dari kedua model dianalisis secara lebih rinci berdasarkan nilai *Precision, Recall,* dan *F1-Score* pada masing-masing kategori melalui *classification report,* serta pola kesalahan klasifikasi melalui *confusion matrix*.

4.5.2 *Confusion Matrix*

Selain menggunakan metrik *Accuracy, Precision, Recall,* dan *F1-score*, evaluasi model juga dilakukan menggunakan *Confusion Matrix*. Hasil *Confusion Matrix* digunakan untuk melihat pola prediksi pada masing-masing kategori topik *podcast*, termasuk jumlah data yang berhasil diklasifikasikan dengan benar dan data yang mengalami kesalahan klasifikasi. Nilai pada diagonal utama menunjukkan jumlah prediksi yang sesuai dengan kategori sebenarnya, sedangkan nilai di luar diagonal menunjukkan perpindahan prediksi dari satu kategori ke kategori lainnya.



Gambar 4.5 *Confusion Matrix* Model pada Data Uji

Gambar 4.5 memperlihatkan *Confusion Matrix* model *Naive Bayes* dan SVM Linear pada 223 data uji untuk tujuh kategori topik *podcast*. Pada model *Naive Bayes*, jumlah prediksi yang benar pada diagonal utama adalah 168 data, sedangkan 55 data lainnya mengalami kesalahan klasifikasi. Model SVM Linear menghasilkan 172 prediksi benar dan 51 prediksi salah. Hasil tersebut sesuai dengan nilai *Accuracy* masing-masing model, yaitu 0,7534 untuk *Naive Bayes* dan 0,7713 untuk SVM Linear.

Pada model *Naive Bayes*, kategori Hiburan memiliki jumlah prediksi benar tertinggi, yaitu 28 dari 32 data, sedangkan kategori Teknologi memiliki jumlah prediksi benar terendah, yaitu 17 dari 32 data. Kesalahan klasifikasi pada kategori Teknologi terutama diprediksi sebagai Berita sebanyak 5 data, Kesehatan sebanyak 4 data, Bisnis sebanyak 3 data, dan Pendidikan sebanyak 3 data. Pada model SVM Linear, kategori Hiburan juga menunjukkan jumlah prediksi benar tertinggi, yaitu 30 dari 32 data, sedangkan kategori Teknologi menghasilkan 19 prediksi benar dari 32 data. Kesalahan pada kategori Teknologi terutama terjadi pada prediksi sebagai Berita sebanyak 6 data, Kesehatan sebanyak 3 data, Bisnis sebanyak 2 data, dan Pendidikan sebanyak 2 data.

Secara keseluruhan, SVM Linear menghasilkan empat prediksi benar lebih banyak dibandingkan *Naive Bayes* pada data uji. Perbedaan tersebut menunjukkan bahwa SVM Linear mampu mengklasifikasikan lebih banyak data sesuai dengan

kategori sebenarnya, meskipun kedua model masih mengalami kesalahan klasifikasi terutama pada beberapa kategori yang memiliki pola teks yang saling berdekatan. Hasil *Confusion Matrix* ini selanjutnya digunakan sebagai dasar untuk melakukan analisis terhadap kelebihan dan kekurangan masing-masing model pada Subbab 4.5.4 Analisis Hasil Model

4.5.3 *Classification Report*

Hasil *classification report* pada data uji sebanyak 223 sampel menunjukkan kinerja model *Naive Bayes* dan SVM Linear pada masing-masing kategori topik *podcast*. Hasil tersebut mencakup nilai *Precision, Recall, F1-Score,* dan *Support* untuk tujuh kategori topik. Perbandingan hasil klasifikasi kedua model disajikan pada Tabel 4.6.

Tabel 4.7 *Classification Report* Per Kategori Topik

Model	**Kategori Topik**	***Precision***	***Recall***	**F1-Score**	**Support**
Naive Bayes	Berita	0.81	0.78	0.79	32
	Bisnis	0.69	0.75	0.72	32
	Hiburan	1.00	0.88	0.93	32
	Kesehatan	0.71	0.69	0.70	32
	Olahraga	0.82	0.87	0.84	31
	Pendidikan	0.71	0.78	0.75	32
	Teknologi	0.57	0.53	0.55	32
SVM Linear	Berita	0.76	0.81	0.79	32
	Bisnis	0.69	0.75	0.72	32
	Hiburan	1.00	0.94	0.97	32
	Kesehatan	0.76	0.69	0.72	32
	Olahraga	0.82	0.87	0.84	31
	Pendidikan	0.77	0.75	0.76	32
	Teknologi	0.61	0.59	0.60	32

Berdasarkan Tabel 4.5, hasil *classification report* menunjukkan kinerja model *Naive Bayes* dan SVM Linear pada masing-masing kategori topik *podcast*. Pada model *Naive Bayes*, nilai *F1-Score* tertinggi diperoleh kategori Hiburan sebesar 0,93, sedangkan nilai terendah terdapat pada kategori Teknologi sebesar 0,55. Pada model SVM Linear, kategori Hiburan juga memperoleh nilai *F1-Score* tertinggi sebesar 0,97, sedangkan nilai terendah terdapat pada kategori Teknologi

sebesar 0,60. Pada kategori Berita, Bisnis, dan Olahraga, kedua model menunjukkan hasil yang sama atau relatif sama, sedangkan pada kategori Kesehatan, Pendidikan, dan Teknologi, SVM Linear menunjukkan nilai *F1-Score* yang lebih tinggi dibandingkan *Naive Bayes*.

Nilai Support pada tabel menunjukkan jumlah data uji yang digunakan pada masing-masing kategori. Kategori Olahraga memiliki 31 sampel, sedangkan kategori lainnya memiliki 32 sampel. Perbedaan tersebut terjadi karena pembagian *dataset* menggunakan Stratified Train-Test Split dengan proporsi 80:20 menghasilkan total 223 data uji dari 1.115 sampel. Pembagian dilakukan secara proporsional dengan tetap menjaga jumlah keseluruhan data uji sebanyak 223 sampel. Apabila kategori Olahraga dibulatkan menjadi 32 sampel, jumlah keseluruhan data uji akan menjadi 224 sampel sehingga tidak sesuai dengan jumlah data uji yang dihasilkan dari pembagian 80:20. Oleh karena itu, kategori Olahraga memperoleh Support sebanyak 31 sampel.

```
Laporan Klasifikasi (Naive Bayes) – Test Set:
              precision    recall  f1-score   support

      Berita       0.81      0.78      0.79        32
      Bisnis       0.69      0.75      0.72        32
     Hiburan       1.00      0.88      0.93        32
   Kesehatan       0.71      0.69      0.70        32
    Olahraga       0.82      0.87      0.84        31
  Pendidikan       0.71      0.78      0.75        32
   Teknologi       0.57      0.53      0.55        32

    accuracy                           0.75       223
   macro avg       0.76      0.75      0.75       223
weighted avg       0.76      0.75      0.75       223


Laporan Klasifikasi (SVM (Linear)) – Test Set:
              precision    recall  f1-score   support

      Berita       0.76      0.81      0.79        32
      Bisnis       0.69      0.75      0.72        32
     Hiburan       1.00      0.94      0.97        32
   Kesehatan       0.76      0.69      0.72        32
    Olahraga       0.82      0.87      0.84        31
  Pendidikan       0.77      0.75      0.76        32
   Teknologi       0.61      0.59      0.60        32

    accuracy                           0.77       223
   macro avg       0.77      0.77      0.77       223
weighted avg       0.77      0.77      0.77       223


Ringkasan Performa Komparatif Model ML Klasik:
              Accuracy  Precision  Recall  F1-Score
Naive Bayes     0.7534     0.7570  0.7534    0.7539
SVM (Linear)    0.7713     0.7733  0.7713    0.7714
```

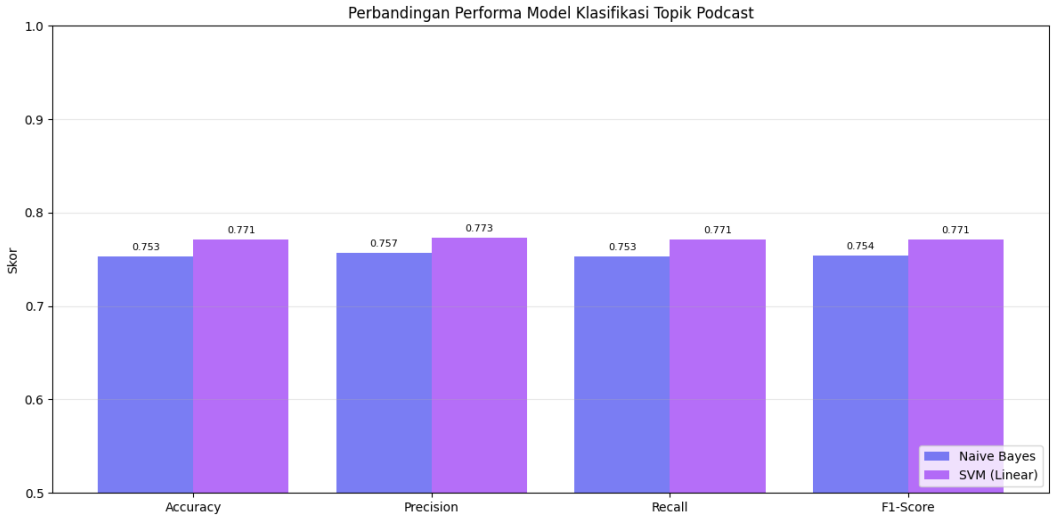
Gambar 4.6 Hasil *Classification Report* dan Ringkasan Performa Model

Gambar 4.6 memperlihatkan hasil evaluasi model *Naive Bayes* dan SVM Linear pada 223 data uji. Keluaran tersebut menampilkan nilai *Precision, Recall, F1-Score,* dan *Support* untuk setiap kategori topik *podcast*, serta ringkasan performa kedua model berdasarkan *Accuracy, Precision, Recall,* dan *F1-Score*. Berdasarkan ringkasan tersebut, model *Naive Bayes* memperoleh *Accuracy* sebesar 0,7534, *Precision* sebesar 0,7570, *Recall* sebesar 0,7534, dan *F1-Score* sebesar 0,7539. Model SVM Linear memperoleh *Accuracy* sebesar 0,7713, *Precision* sebesar 0,7733, *Recall* sebesar 0,7713, dan *F1-Score* sebesar 0,7714. Hasil tersebut menunjukkan bahwa SVM Linear memiliki nilai evaluasi yang lebih tinggi

dibandingkan *Naive Bayes* pada seluruh metrik yang ditampilkan dalam ringkasan performa.

4.5.4 Analisis Hasil Model

Berdasarkan hasil pengujian pada 223 data uji, kedua algoritma mampu melakukan klasifikasi terhadap tujuh kategori topik *podcast*. Perbandingan performa menunjukkan bahwa model SVM Linear memperoleh nilai evaluasi yang lebih tinggi dibandingkan *Naive Bayes* pada seluruh metrik yang digunakan. *Naive Bayes* memperoleh *Accuracy* sebesar 0,7534, *Precision* sebesar 0,7570, *Recall* sebesar 0,7534, dan *F1-Score* sebesar 0,7539. SVM Linear memperoleh *Accuracy* sebesar 0,7713, *Precision* sebesar 0,7733, *Recall* sebesar 0,7713, dan *F1-Score* sebesar 0,7714.



Gambar 4.7 Perbandingan Kinerja Model Klasifikasi Topik *Podcast*

Gambar 4.7 memperlihatkan perbandingan nilai *Accuracy*, *Precision*, *Recall*, dan *F1-Score* antara model *Naive Bayes* dan SVM Linear. Pada seluruh metrik, nilai SVM Linear berada di atas *Naive Bayes*. Selisih *Accuracy* kedua model sebesar 0,0179, sedangkan selisih *Precision* sebesar 0,0163. Pada *Recall*, SVM Linear lebih tinggi sebesar 0,0179, dan pada *F1-Score* memiliki selisih sebesar 0,0175. Hasil tersebut menunjukkan bahwa SVM Linear memberikan performa klasifikasi yang lebih baik secara keseluruhan pada *dataset podcast* yang digunakan.

Hasil *Confusion Matrix* juga mendukung perbandingan tersebut. SVM Linear menghasilkan 172 prediksi benar, sedangkan *Naive Bayes* menghasilkan 168 prediksi benar dari 223 data uji. SVM Linear juga menunjukkan peningkatan prediksi benar pada kategori Hiburan dan Teknologi dibandingkan *Naive Bayes.* Berdasarkan keseluruhan hasil evaluasi, SVM Linear dipilih sebagai model dengan kinerja terbaik dalam klasifikasi topik *podcast* pada penelitian ini.

4.6 *Deployment*

4.6.1 Hasil Ekspor Model

Best model yaitu SVM model classifier dalam format .pkl disimpan pada direktori models pada folder *podcastclassifier*. Folder *Podcastclassifier* adalah folder untuk menyimpan aplikasi app.py ("Klasifikasi Topik *Podcast*"). Aplikasi tersebut selanjutnya dapat diakses secara publik melalui tautan http://195.88.211.42/

4.6.2 Hasil Rancangan dan Implementasi Model pada Aplikasi Web



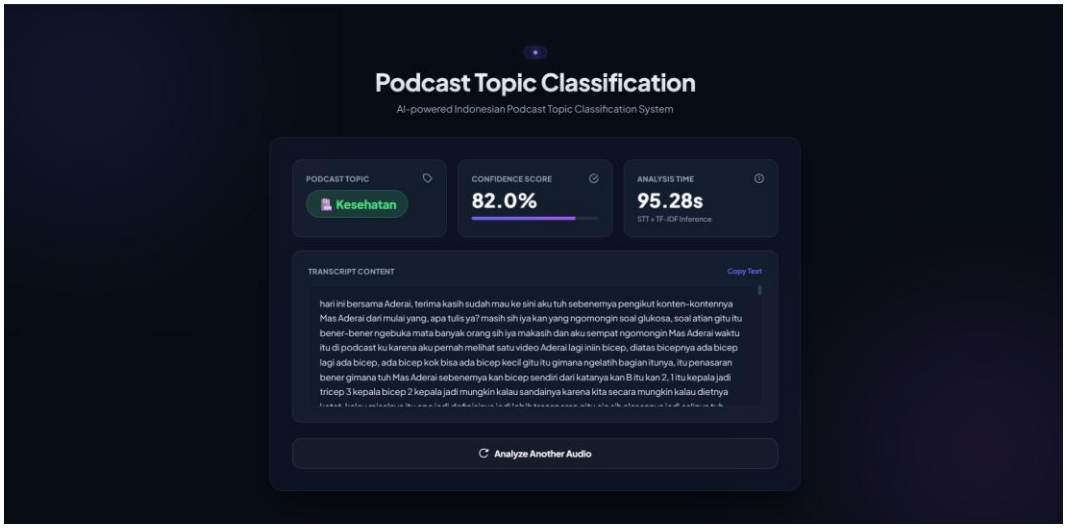
Gambar 4.8 Halaman *Upload* Audio

Halaman *Upload* Audio digunakan sebagai antarmuka untuk memasukkan berkas audio *podcast* ke dalam sistem. Pengguna dapat memilih atau memasukkan berkas audio dengan format yang didukung oleh aplikasi sebelum memulai proses klasifikasi.



Gambar 4.9 Proses Analisis Audio

Setelah berkas audio diproses, sistem menampilkan tahapan analisis yang sedang berlangsung. Proses tersebut mencakup pengolahan audio dan transkripsi menggunakan Whisper Medium sebelum data dilanjutkan ke tahap analisis dan klasifikasi.



Gambar 4.10 Hasil Klasifikasi dan Transkripsi

Setelah seluruh proses selesai, aplikasi menampilkan hasil klasifikasi berupa kategori topik *podcast*, *confidence score*, waktu analisis, serta teks hasil transkripsi. Informasi tersebut menjadi keluaran akhir dari proses klasifikasi yang dapat dilihat oleh pengguna.

4.6.3 *Deployment* Menggunakan Ngrok

Setelah aplikasi Flask berhasil diintegrasikan dengan model klasifikasi dan seluruh komponen pendukungnya, dilakukan *deployment* sementara menggunakan Ngrok pada lingkungan Google Colab. Aplikasi Flask dijalankan pada port 5000, kemudian Ngrok digunakan untuk membuat *tunnel* yang menghubungkan aplikasi Flask dengan jaringan internet.

Hasil proses *deployment* ditunjukkan pada Gambar 4.X. Berdasarkan *output* yang ditampilkan, *server* Flask berhasil dijalankan dan dapat menerima koneksi melalui alamat http://127.0.0.1:5000 serta alamat jaringan internal http://172.28.0.12:5000. Ngrok kemudian berhasil membuat *tunnel* dari alamat publik menuju localhost:5000, sehingga aplikasi dapat diakses melalui URL publik yang dihasilkan.

Deployment berhasil dilakukan dengan menjalankan aplikasi Flask pada port 5000 dan menghubungkannya melalui Ngrok. Port 5000 digunakan karena merupakan port yang umum digunakan sebagai port pengembangan pada aplikasi Flask dan termasuk *non-privileged* port, sehingga dapat digunakan untuk menjalankan *server* aplikasi tanpa memerlukan hak akses khusus pada sistem. Penggunaan port tersebut juga memudahkan konfigurasi koneksi antara *server* Flask dan layanan tunneling Ngrok yang diarahkan ke localhost:5000. *Output* menunjukkan bahwa *server* Flask berhasil dijalankan dan tunnel Ngrok berhasil dibuat dengan mengarahkan URL publik menuju localhost:5000. URL publik tersebut selanjutnya dapat digunakan untuk mengakses aplikasi web melalui browser.

Deployment yang dilakukan bersifat sementara karena aplikasi bergantung pada *server* Flask dan sesi Google Colab yang sedang aktif. Aplikasi yang telah berhasil diakses melalui URL publik kemudian digunakan pada tahap uji coba aplikasi untuk memastikan setiap fungsi berjalan sesuai dengan rancangan.

4.6.3 Hasil Pengujian Black Box

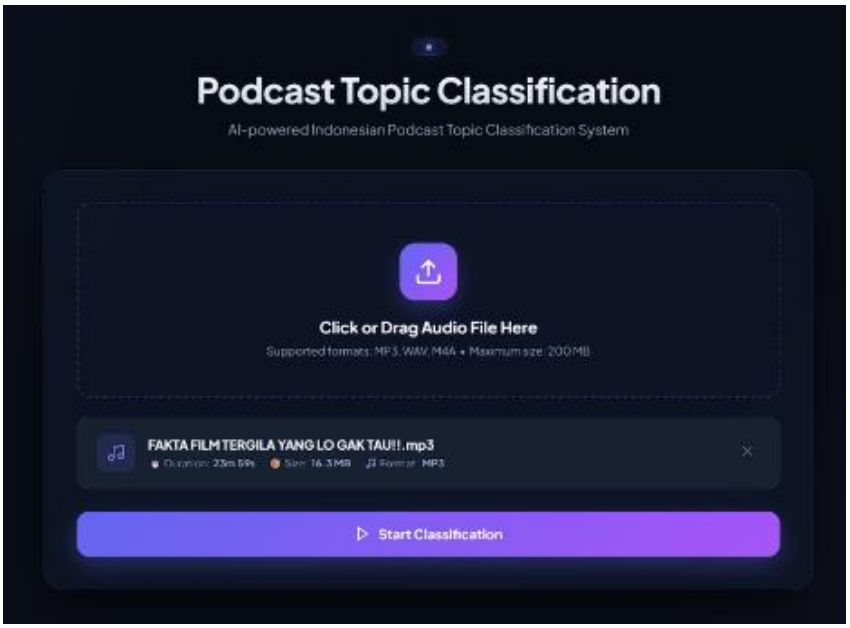
Pengujian aplikasi dilakukan menggunakan metode *Black Box Testing* untuk memastikan setiap fungsi utama pada aplikasi klasifikasi topik *podcast* dapat

berjalan sesuai dengan kebutuhan yang telah ditentukan. Pengujian difokuskan pada fungsi unggah berkas audio, validasi format, pemotongan audio, proses transkripsi dan *Preprocessing*, klasifikasi menggunakan SVM Linear, serta tampilan hasil pada antarmuka aplikasi.

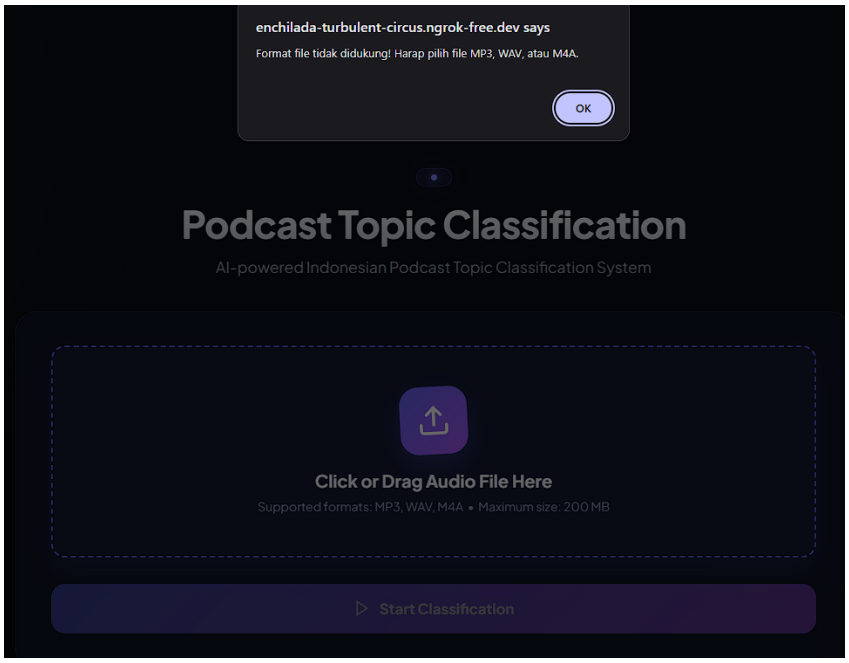
Tabel 4.8 Hasil Pengujian Black Box Aplikasi Web Klasifikasi Topik *Podcast*

Kasus Pengujian	**Skenario Uji**	**Hasil yang Diharapkan**	**Hasil Pengujian**	**Status**
Unggah Berkas Audio	Memilih berkas audio format `.mp3`, `.wav`, atau `.m4a`	Berkas berhasil diunggah ke folder sistem dan tombol proses aktif	Berkas tersimpan di direktori *server* dan proses klasifikasi berjalan	Berhasil
Validasi Format	Mengunggah berkas selain format audio (misal `.pdf` atau `.txt`)	Sistem menolak berkas dan menampilkan pesan kesalahan	Sistem menolak unggahan dan menampilkan notifikasi peringatan	Berhasil
Transkripsi & *Preprocessing*	Memproses audio yang diunggah	Whisper menghasilkan teks transkrip dan pustaka Sastrawi membersihkan teks	Teks transkrip bersih berhasil terbentuk di backend	Berhasil
Klasifikasi SVM	Mengklasifikasikan fitur TF-IDF teks transkrip	Model SVM memprediksi label kategori dan nilai probabilitas	Topik prediksi dan *confidence score* berhasil dihitung	Berhasil
Tampilan Hasil UI	Menampilkan hasil pada antarmuka pengguna	Antarmuka web menampilkan nama topik, *confidence score*, durasi, dan transkrip	Seluruh informasi berhasil dirender pada halaman hasil UI	Berhasil

Hasil Berdasarkan Tabel 4.7, seluruh fungsi yang diuji memperoleh status Berhasil. Fungsi unggah audio dapat menerima format yang telah ditentukan, sedangkan validasi format dapat menolak berkas yang tidak sesuai.



Gambar 4.11 Hasil Uji Unggah Berkas Audio



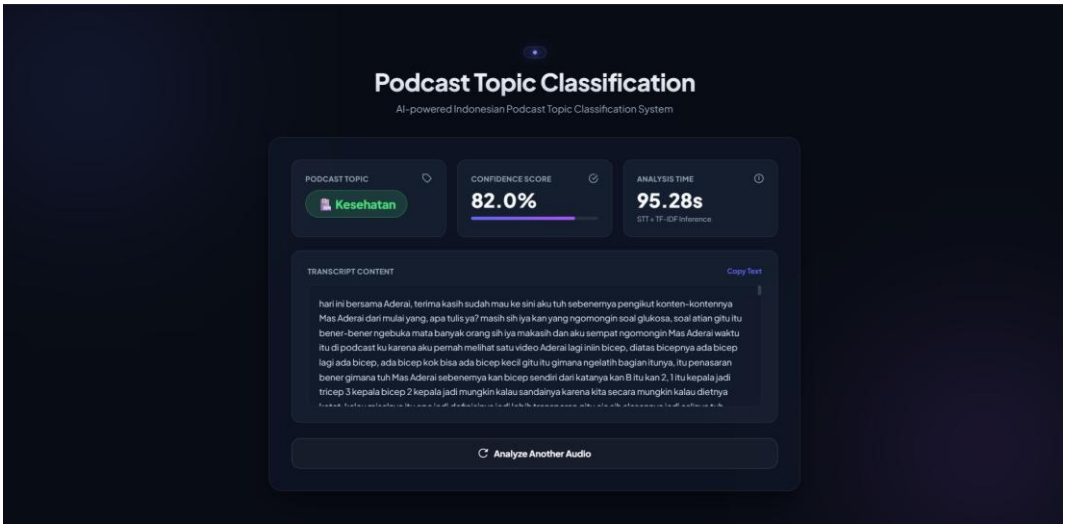
Gambar 4.12 Hasil Uji Validasi Format

Proses transkripsi menggunakan Whisper Medium dan *Preprocessing* teks menggunakan Sastrawi dapat menghasilkan teks yang siap digunakan pada tahap klasifikasi.



Gambar 4.13 Hasil Uji Transkripsi dan *Preprocessing*

Fitur TF-IDF berhasil diproses menggunakan model SVM Linear sehingga sistem dapat menghasilkan kategori topik dan *confidence score*. Hasil akhir juga berhasil ditampilkan pada antarmuka aplikasi beserta informasi pendukung berupa durasi dan teks transkripsi.



Gambar 4.14 Hasil Uji Klasifikasi SVM

Berdasarkan hasil analisis uji coba sistem, aplikasi web yang dibangun berhasil menjalankan fungsi validasi masukan berkas audio secara presisi. Sistem mampu memverifikasi berkas audio yang sesuai dengan format yang didukung (seperti .mp3, .wav, .m4a, dan .webm) serta menolak unggahan berkas yang tidak sesuai atau melebihi batas ukuran maksimal, dilanjutkan dengan menampilkan notifikasi peringatan yang responsif dan informatif kepada pengguna.

Aplikasi berhasil melakukan proses transkripsi ucapan audio secara *real-time* memanfaatkan model *Speech-to-Text* (STT) OpenAI Whisper dan mengklasifikasikan topik podcast berdasarkan representasi fitur teks ucapan tersebut. Pemrosesan transkripsi dibatasi secara spesifik pada 10 menit pertama audio (600 detik). Pembatasan durasi ini dilakukan untuk mengoptimalkan efisiensi komputasi pada lingkungan *server* tanpa mengurangi akurasi model, sebab segmen 10 menit pertama terbukti telah memuat kata kunci inti (*domain-specific keywords*) yang sangat representatif untuk menggambarkan keseluruhan konteks dan topik utama podcast tersebut.

Akhir pemrosesan pada antarmuka aplikasi web, secara otomatis berhasil menyajikan empat komponen informasi utama secara visual dan terstruktur kepada pengguna. Sistem menampilkan topik podcast hasil prediksi, nilai tingkat kepercayaan yang mengukur kepastian prediktor model dalam bentuk persentase, waktu analisis yang mencatat durasi total alur pemrosesan STT dan inferensi *Machine Learning* dalam satuan detik, serta konten transkrip yang menyajikan teks ucapan asli hasil ekstraksi sinyal audio. Penyajian keempat komponen informasi ini secara simultan memberikan transparansi data serta kemudahan interpretasi hasil klasifikasi secara komprehensif bagi pengguna.

5. KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil penelitian yang telah dilakukan, dapat disimpulkan bahwa sistem klasifikasi topik *podcast* berbahasa Indonesia berhasil dibangun dengan mengintegrasikan Whisper Medium, *Preprocessing* teks, TF-IDF, dan algoritma SVM Linear ke dalam aplikasi berbasis Flask. Hasil evaluasi pada data uji menunjukkan bahwa SVM Linear memberikan kinerja lebih baik dibandingkan *Naive Bayes* dengan nilai *Accuracy* sebesar 0,7713, *Precision* sebesar 0,7733, *Recall* sebesar 0,7713, dan *F1-Score* sebesar 0,7714. Hasil pengujian *Black Box Testing* juga menunjukkan bahwa seluruh fungsi utama aplikasi, mulai dari unggah audio, validasi format, pemotongan audio, transkripsi, *Preprocessing*, klasifikasi, hingga tampilan hasil dapat berjalan sesuai dengan hasil yang diharapkan.

5.2 Saran

Penelitian selanjutnya disarankan untuk menggunakan jumlah dan variasi *dataset* yang lebih besar serta distribusi data yang lebih seimbang pada setiap kategori topik agar model dapat memperoleh representasi data yang lebih beragam. Pengembangan juga dapat dilakukan dengan membandingkan algoritma klasifikasi dan metode ekstraksi fitur lainnya untuk memperoleh model dengan kinerja yang lebih optimal. Dari sisi aplikasi, pengembangan dapat dilakukan dengan menghadirkan antarmuka yang responsif atau aplikasi tersendiri berbasis *mobile*, mengingat aplikasi pada penelitian ini hanya dapat diakses secara optimal melalui perangkat *desktop*/web dan belum mendukung penggunaan pada perangkat *mobile*.

Daftar Pustaka

Albab, M. U., Karuniawati, Y., & Fawaiq, M. N. (2023). Optimization of the Stemming Technique on Text Preprocessing President 3 Periods Topic. *Jurnal Transformatika*, *20*(2). https://doi.org/10.26623/transformatika.v20i2.5374

Aliman, W. (2021). Perancangan Perangkat Lunak untuk Menggambar Diagram Berbasis Android. *Syntax Literate: Jurnal Ilmiah Indonesia*, *6*(6), 3091. https://doi.org/10.36418/syntax-literate.v6i6.1404

Aquino, É. R., de Saqui-Sannes, P., & Vingerhoeds, R. (2020). A Methodological Assistant for Use Case Diagrams. *Proceedings of the 8th International Conference on Model-Driven Engineering and Software Development*, 227–236. https://doi.org/10.5220/0008938002270236

Ardi, A., & Kurniawan. (2024). Optimization of Naïve Bayes Classifier Method Using Term Frequency-Inverse Document Frequency Approach (TF-IDF) Approach for Sentiment Analysis. *JSAI (Journal Scientific and Applied Informatics)*, *7*(3), 458–463. https://doi.org/10.36085/jsai.v7i3.7153

Azmi, B. N., Hermawan, A., & Avianto, D. (2023). Analisis Pengaruh Komposisi Data Training dan Data Testing pada Penggunaan PCA dan Algoritma Decision Tree untuk Klasifikasi Penderita Penyakit Liver. *JTIM: Jurnal Teknologi Informasi Dan Multimedia*. https://doi.org/10.35746/jtim.v4i4.298

Ciksadan, C., Soim, S., & Jami, N. (2024). Desain dan Pengembangan Website untuk Mendeteksi Malware Menggunakan Framework Flask yang Diintegrasikan dengan Machine Learning. *Jurnal Teknologi Sistem Informasi Dan Aplikasi*, *7*(3), 1213–1218. https://doi.org/10.32493/jtsi.v7i3.42003

Clifton, A., Reddy, S., Yu, Y., Pappu, A., Rezapour, R., Bonab, H., Eskevich, M., Jones, G., Karlgren, J., Carterette, B., & Jones, R. (2020). 100,000 *Podcast*s: A Spoken English Document Corpus. *Proceedings of the 28th International Conference on Computational Linguistics*, 5903–5917. https://doi.org/10.18653/v1/2020.coling-main.519

Das, M., Kamalanathan, S., & Alphonse, P. J. A. (2023). *A Comparative Study on TF-IDF feature Weighting Method and its Analysis using Unstructured Dataset*. https://doi.org/10.48550/arXiv.2308.04037

DataReportal. (2025). *Digital 2025: Indonesia*.

https://datareportal.com/reports/digital-2025-indonesia

Fathony, I. A. N., Mareta, A., Adiana, B. E., Wardhani, O., & Halim, D. A. (2025). Optimisasi Whisper Speech-to-Text Bahasa Indonesia dengan Hybrid Cloud dan Multi-Engine. *Elkom: Jurnal Elektronika Dan Komputer*, *18*(1), 60–72. https://doi.org/10.51903/5n2d3s08

Gahman, N., & Elangovan, V. (2023). A Comparison of Document Similarity Algorithms. *International Journal of Artificial Intelligence & Applications*, *14*(2), 41–50. https://doi.org/10.5121/ijaia.2023.14204

Gerliandeva, A., Chrisnanto, Y. H., & Ashaury, H. (2024). Optimasi Klasifikasi Sentimen pada Komentar Online menggunakan Multinomial Naïve Bayes dan Ekstraksi Fitur TF-IDF serta N-grams. *Jurnal Pekommas*, *9*(2), 259–272. https://doi.org/10.56873/jpkm.v9i2.5585

Grandini, M., Bagli, E., & Visani, G. (2020). *Metrics for Multi-Class Classification: an Overview*. https://doi.org/10.48550/arXiv.2008.05756

Hidayat, M., & Wibowo, A. (2024). SVM Optimization with Information Gain Feature Selection to Increase the Accuracy of Sentiment Analysis of Increasing the Cost of the Hajj. *Jurnal Teknik Informatika (JUTIF)*, *5*(4). https://doi.org/10.52436/1.jutif.2024.5.4.2217

Huda, M. N., Burhan, M., Satibi, A., Pradita, H. A., Saifudin, A., & Kusyadi, I. (2022). Implementasi Black Box Testing pada Aplikasi Sistem Kasir dengan Menggunakan Teknik Equivalence Partitions. *Jurnal Teknologi Sistem Informasi Dan Aplikasi*, *5*(2), 120–127. https://doi.org/10.32493/jtsi.v5i2.17645

Illowsky, B., & Dean, S. (2023). *Introductory Statistics 2e*. OpenStax. https://openstax.org/books/introductory-statistics-2e/pages/1-introduction

Iskandar, J. W., & Nataliani, Y. (2021). Perbandingan Naïve Bayes, SVM, dan k-NN untuk Analisis Sentimen Gadget Berbasis Aspek. *Jurnal RESTI (Rekayasa Sistem Dan Teknologi Informasi)*, *5*(6), 1120–1126. https://doi.org/10.29207/resti.v5i6.3588

Januzaj, Y., & Luma, A. (2022). Cosine Similarity – A Computing Approach to Match Similarity Between Higher Education Programs and Job Market Demands Based on Maximum Number of Common Words. *International Journal of Emerging Technologies in Learning*, *17*(12), 258–268. https://doi.org/10.3991/ijet.v17i12.30375

Jones, R., Zamani, H., Schedl, M., Chen, C.-W., Reddy, S., Clifton, A., Karlgren,

J., Hashemi, H., Pappu, A., Nazari, Z., Yang, L., Semerci, O., Bouchard, H., & Carterette, B. (2021). Current Challenges and Future Directions in *Podcast* Information Access. *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1554–1565. https://doi.org/10.1145/3404835.3462805

Jurafsky, D., & Martin, J. H. (2025). *Speech and Language Processing* (3rd ed. draft). https://web.stanford.edu/~jurafsky/slp3/

Khoiroh, R. F., Julianto, E., Ardiyansa, S. A., Fajri, H. A., Abi, A., Yasa, R., & Sangapta, B. (2024). Implementasi Speech Recognition Whisper pada Debat Calon Wakil Presiden Republik Indonesia. *EXPLORE*, *14*(2), 67–74. https://doi.org/10.35200/ex.v14i2.115

Koç, H., Erdoğan, A. M., Barjakly, Y., & Peker, S. (2021). UML Diagrams in Software Engineering Research: A Systematic Literature Review. *Proceedings*, *74*(1), 13. https://doi.org/10.3390/proceedings2021074013

Koentjoro, E. Y., & Budiardjo, H. (2025). Implementasi Model Client Server Menggunakan Ngrok. *Jurnal Ilmiah SCROLL: Jendela Teknologi Informasi*, *13*(1), 32–35. https://journal.univ45sby.ac.id/index.php/SCROLL/article/view/5

Maatuk, A. M., & Abdelnabi, E. A. (2021). *Generating UML Use Case and Activity Diagrams Using NLP Techniques and Heuristics Rules*. https://doi.org/10.1007/978-3-030-70513-1_4

Malik, M., Malik, M. K., Mehmood, K., & Makhdoom, I. (2021). Automatic Speech Recognition: A Survey. *Multimedia Tools and Applications*. https://doi.org/10.1007/s11042-020-10073-7

Maulana, B. A., Fahmi, M. J., Imran, A. M., & Hidayati, N. (2024). Analisis Sentimen Terhadap Aplikasi Pluang Menggunakan Algoritma Naive Bayes dan Support Vector Machine (SVM). *MALCOM: Indonesian Journal of Machine Learning and Computer Science*, *4*(2), 375–384. https://doi.org/10.57152/malcom.v4i2.1206

Opitz, J. (2024). A Closer Look at Classification Evaluation Metrics and a Critical Reflection of Common Evaluation Practice. *Transactions of the Association for Computational Linguistics*, *12*, 820–836. https://doi.org/10.1162/tacl_a_00675

Opoku Asare, K., Terhorst, Y., Vega, J., Peltonen, E., Lagerspetz, E., & Ferreira, D. (2021). Predicting Depression From Smartphone Behavioral Markers Using Machine Learning Methods, Hyperparameter Optimization, and

Feature Importance Analysis: Exploratory Study. *JMIR mHealth and uHealth*, *9*(7), e26540. https://doi.org/10.2196/26540

Pranata, R. A., Rudiman, & Verdikha, N. A. (2024). Metode Pembobotan TF-IDF untuk Klasifikasi Teks Quick Count Pemilihan Wakil Presiden Indonesia 2024 pada X Twitter dengan Metode SVM. *Jurnal Teknologi Informasi*, *18*(2). https://doi.org/10.47111/jti.v18i2.14934

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2022). *Robust Speech Recognition via Large-Scale Weak Supervision*. https://doi.org/10.48550/arXiv.2212.04356

Rakajati, B., & Hidayat, E. Y. (2024). Perbandingan Metode Naïve Bayes dan Support Vector Machine Pada Klasifikasi 22 Bahasa Daerah. *JURNAL MEDIA INFORMATIKA BUDIDARMA*, *8*(1), 221–230. https://doi.org/10.30865/mib.v8i1.7236

Research, E. (2024). *The Podcast Consumer 2024*. https://www.edisonresearch.com/the-podcast-consumer-2024-by-edison-research/

Rime, J., Pike, C., & Collins, T. (2022). What is a podcast? Considering innovations in podcasting through the six-tensions framework. *Convergence: The International Journal of Research into New Media Technologies*, *28*(5), 1260–1282. https://doi.org/10.1177/13548565221104444

Rosid, M. A., Fitrani, A. S., Astutik, I. R. I., Mulloh, N. I., & Gozali, H. A. (2020). Improving Text Preprocessing for Student Complaint Document Classification Using Sastrawi. *IOP Conference Series: Materials Science and Engineering*, *874*, 012017. https://doi.org/10.1088/1757-899X/874/1/012017

Schröer, C., Kruse, F., & Marx Gómez, J. C. (2021). A Systematic Literature Review on Applying CRISP-DM Process Model. *Procedia Computer Science*, *181*, 526–534. https://doi.org/10.1016/j.procs.2021.01.199

Suhaeni, C., Kamila, S. A., Fahira, F., Yusran, M., & Alfa Dito, G. (2025). Exploring a Large Language Model on the ChatGPT Platform for Indonesian Text Preprocessing Tasks. *Indonesian Journal of Statistics and Its Applications*, *9*(1), 100–116. https://doi.org/10.29244/ijsa.v9i1p100-116

Vrigazova, B. (2021). *The Proportion for Splitting Data into Training and Test Set for the Bootstrap in Classification Problems*. https://doi.org/10.2478/bsrj-2021-0015

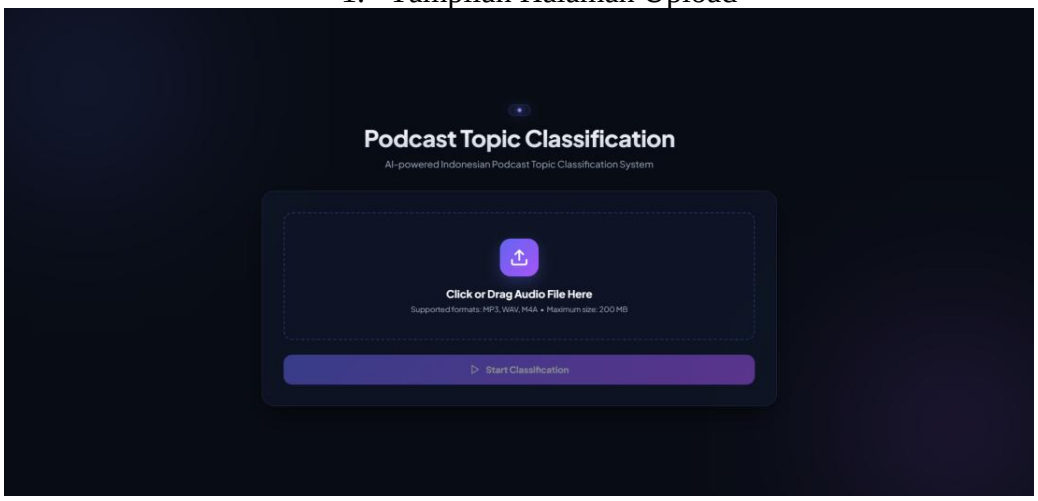
Walingkas, H. L., & Saian, P. O. N. (2023). Penerapan Framework Flask pada Pembangunan Sistem Informasi Pemasok Barang. *Jurnal JTIK (Jurnal Teknologi Informasi Dan Komunikasi)*, *7*(2), 227–234. https://doi.org/10.35870/jtik.v7i2.729

Widodo, S., Brawijaya, H., & Samudi. (2022). Stratified K-fold Cross Validation Optimization on Machine Learning for Prediction. *SinkrOn: Jurnal Dan Penelitian Teknik Informatika*, *6*(4), 2407–2414. https://doi.org/10.33395/sinkron.v7i4.11792

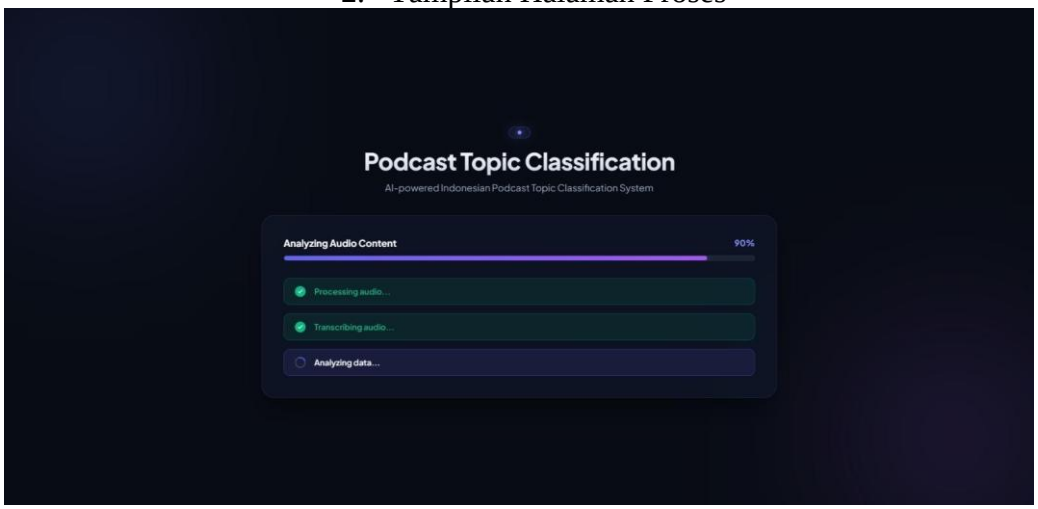
Zidan, M., Nur'aini, S., Wibowo, N. C. H., & Ulinuha, M. A. (2022). Black Box Testing pada Aplikasi Single Sign On (SSO) di Diskominfostandi Menggunakan Teknik Equivalence Partitions. *Walisongo Journal of Information Technology*, *4*(2), 127–137. https://doi.org/10.21580/wjit.2022.4.2.12135

LAMPIRAN TAMPILAN APLIKASI

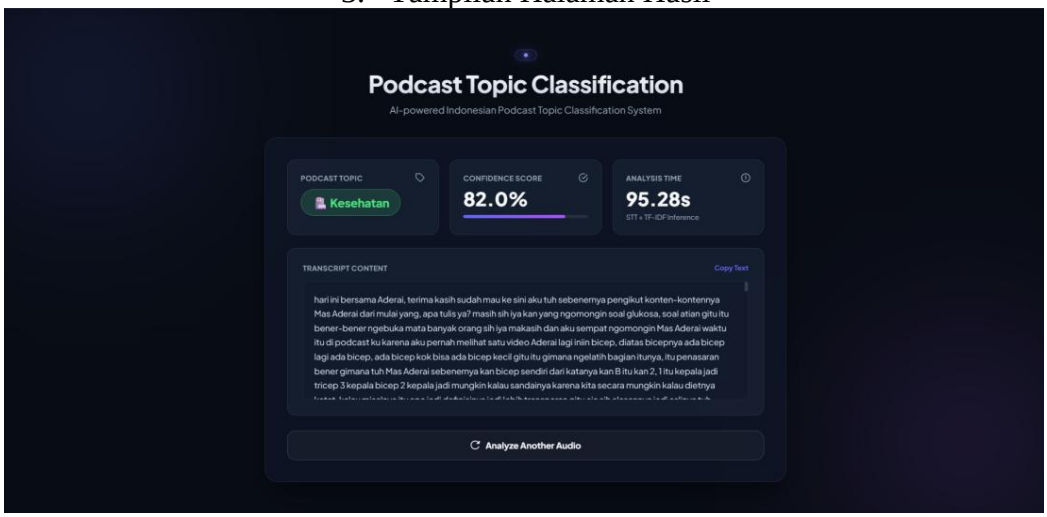
1. Tampilan Halaman Upload



2. Tampilan Halaman Proses



3. Tampilan Halaman Hasil



LISTING PROGRAM

```
# ========================================
# CELL 1: Mount Google Drive & Inisialisasi Path Utama
# ========================================
from google.colab import drive

# 1. Hubungkan Google Drive
drive.mount('/content/drive')

# 2. Definisikan Path Utama (Wajib dijalankan untuk sel-sel berikutnya)
BASE_DIR = '/content/drive/MyDrive/PodcastClassifier'

print(f"Google Drive terhubung.")
print(f"Direktori Utama: {BASE_DIR}")

# ========================================
# CELL 2: Install Pustaka Tambahan yang Dibutuhkan
# ========================================
%%capture
# Menginstal paket tambahan yang belum ada di Google Colab bawaan secara
      senyap (-q)
# Menyertakan pyngrok untuk webapp dan jiwer untuk evaluasi WER
!pip install -q openai-whisper PySastrawi pyngrok yt-dlp jiwer
!apt-get install -q -y ffmpeg  # Diperlukan oleh Whisper untuk pemrosesan
      file audio

print("[OK] Semua dependensi tambahan berhasil diinstal!")

# ========================================
# CELL 3: Import Semua Library & Inisialisasi Lingkungan
# ========================================
import os
import re
import json
import time
import warnings
from pathlib import Path
from datetime import datetime

# Matikan pesan peringatan yang mengganggu keluaran konsol
warnings.filterwarnings('ignore')

# 1. Pustaka Pengolah Data, Visualisasi & Komputasi Dasar
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
import nltk
import torch

# 2. Pengunduhan Sumber Daya NLTK (punkt_tab ditambahkan untuk
      kompatibilitas Python 3.12+)
print("[INFO] Mengunduh NLTK resources...")
nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)
nltk.download('stopwords', quiet=True)
print("[OK] NLTK resources siap digunakan!")

# 3. Pustaka Scikit-learn (Machine Learning Klasik)
from sklearn.model_selection import StratifiedKFold, cross_validate,
```

```
      train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC
from sklearn.metrics import (accuracy_score, precision_score,
      recall_score,
                           f1_score, classification_report,
      confusion_matrix)
from sklearn.pipeline import Pipeline

# 4. Pustaka PySastrawi (Stopwords & Stemmer Bahasa Indonesia)
from Sastrawi.Stemmer.StemmerFactory import StemmerFactory
from Sastrawi.StopWordRemover.StopWordRemoverFactory import
      StopWordRemoverFactory

# 6. Konfigurasi Device Komputasi (T4 GPU disarankan di Colab)
device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f"\n[OK] Impor berhasil | Device komputasi aktif:
      {device.upper()}")

# ========================================
# CELL 4: Inventarisasi Berkas Audio Podcast (Mendukung .webm)
# ========================================
AUDIO_DIR = f'{BASE_DIR}/data/audio'

LABELS = ['hiburan','olahraga','pendidikan','teknologi', 'bisnis',
      'kesehatan', 'berita']

audio_inventory = {}
for label in LABELS:
    folder = os.path.join(AUDIO_DIR, label)
    if os.path.exists(folder):
        # Memindai berkas dengan format audio standar termasuk .webm dari
      YouTube
        files = [f for f in os.listdir(folder) if
      f.lower().endswith(('.mp3', '.wav', '.m4a', '.webm'))]
        audio_inventory[label] = files
        print(f"[INFO] Kategori {label.capitalize():12s}: Terdeteksi
      {len(files):3d} file audio")
    else:
        print(f"[PERINGATAN] Folder kategori tidak ditemukan: {folder}")

total_files = sum(len(v) for v in audio_inventory.values())
print(f"\n[OK] Inventarisasi selesai | Total file audio keseluruhan:
      {total_files}")

# ========================================
# CELL 5: Memuat Model Whisper (Akurasi Terbaik)
# ========================================
import whisper

# Pilihan model: tiny | base | small | medium | large-v3
# Rekomendasi untuk akurasi terbaik & aman di Google Colab T4 GPU:
      'medium'
# Model 'medium' menghasilkan transkripsi Bahasa Indonesia yang lebih
      terstruktur
# sehingga akurasi klasifikasi topik akhir akan meningkat tajam.
WHISPER_MODEL_SIZE = 'medium'

print(f"[INFO] Memuat model Whisper '{WHISPER_MODEL_SIZE}'...")
whisper_model = whisper.load_model(WHISPER_MODEL_SIZE, device=device)
print(f"[OK] Model Whisper '{WHISPER_MODEL_SIZE}' berhasil dimuat!")
```

```
# ========================================
# CELL 6: Proses Transkripsi Otomatis dengan Whisper (Resume-friendly)
# ========================================
# Lokasi penyimpanan dataset hasil transkripsi di Google Drive
SAVE_PATH = f'{BASE_DIR}/data/dataset/dataset_transkripsi.csv'
JSON_PATH = f'{BASE_DIR}/data/dataset/transkripsi_detail.json'

# Pastikan folder penyimpanan dataset sudah ada
os.makedirs(os.path.dirname(SAVE_PATH), exist_ok=True)

# ── 1. Load Progress Sebelumnya (Jika Ada) ──────────────────────────────
if os.path.exists(SAVE_PATH):
    df_done = pd.read_csv(SAVE_PATH)
    # Pastikan kolom 'audio_file' ada untuk mendeteksi file yang sudah selesai
    if 'audio_file' not in df_done.columns:
        df_done['audio_file'] = ''
    done_list = df_done.to_dict('records')
    print(f"[INFO] Progress ditemukan! Melanjutkan dari {len(done_list)} audio yang sudah ditranskripsi.")
else:
    done_list = []
    print("[INFO] Memulai transkripsi baru dari nol.")

# Set nama file yang sudah selesai agar tidak ditranskripsi ulang
done_files = {str(d.get('audio_file', '')).strip() for d in done_list}

hasil_semua = done_list.copy()
gagal = []

# ── 2. Proses Transkripsi Menggunakan Whisper ──────────────────────────
print("\n" + "="*60)
print(f"[INFO] MEMULAI TRANSKRIPSI (Model: {WHISPER_MODEL_SIZE.upper()})")
print("="*60)

for label in LABELS:
    # Ambil list file hasil inventarisasi dari Cell 5
    files = audio_inventory.get(label, [])
    if not files:
        print(f"\n[INFO] Kategori {label.upper()}: Tidak ada file audio. Melewati!")
        continue

    print(f"\n[INFO] Kategori {label.upper()}: Memproses {len(files)} file audio...")
    folder = os.path.join(AUDIO_DIR, label)

    for i, audio_name in enumerate(files, 1):
        # 1. Cek apakah file ini sudah pernah ditranskripsi sebelumnya
        if audio_name in done_files:
            print(f"   [{i:03d}/{len(files)}] [SKIP] Sudah ditranskripsi: {audio_name[:50]}...")
            continue

        path = os.path.join(folder, audio_name)

        # 2. Skip jika file kosong/corrupt (di bawah 10KB)
        if os.path.getsize(path) < 10_000:
            print(f"   [{i:03d}/{len(files)}] [WARNING] File rusak/terlalu kecil: {audio_name[:50]}...")
```

```
            continue

        print(f"\n   [{i:03d}/{len(files)}] [RUN] Transcribing:
       {audio_name[:55]}...")
        try:
            t0 = time.time()

            # Jalankan model Whisper
            hasil = whisper_model.transcribe(
                path,
                language='id',
                task='transcribe',
                verbose=False,
                no_speech_threshold=0.6,
                condition_on_previous_text=True,
            )

            teks = hasil['text'].strip()
            durasi = time.time() - t0
            n_kata = len(teks.split())

            print(f"      [OK] Berhasil dalam {durasi:.1f}s | {n_kata}
       kata")
            print(f"      [PREVIEW] \"{teks[:100]}...\"")

            # Simpan hasil per file ke dalam list progress
            record = {
                'teks': teks,
                'label': label,
                'n_kata': n_kata,
                'audio_file': audio_name,
            }
            hasil_semua.append(record)
            done_files.add(audio_name)

            # 3. Langsung simpan ke CSV Drive secara realtime (aman jika
       Colab mendadak terputus)
            df_progress = pd.DataFrame(hasil_semua)
            df_progress[['teks', 'label', 'audio_file']].to_csv(
                SAVE_PATH, index=False, encoding='utf-8-sig'
            )

        except Exception as e:
            print(f"      [ERROR] Gagal memproses {audio_name}: {e}")
            gagal.append(audio_name)

# ── 3. Rekap Akhir Transkripsi
    ──────────────────────────────────────────────────
print("\n" + "="*60)
print("[INFO] TRANSKRIPSI SELESAI!")
print(f"   - Berhasil diproses : {len(hasil_semua)} file")
print(f"   - Gagal diproses    : {len(gagal)} file")
print(f"   - Berkas CSV Output : {SAVE_PATH}")
print("="*60)

# Simpan cadangan detail lengkap dalam bentuk JSON
with open(JSON_PATH, 'w', encoding='utf-8') as f:
    json.dump(hasil_semua, f, ensure_ascii=False, indent=2)

# ========================================
# CELL 7: Evaluasi Kualitas Transkripsi (Word Error Rate - WER)
# ========================================
from jiwer import wer, cer
```

```
def calculate_wer_sample(reference_text, hypothesis_text):
    """
    Menghitung WER & CER antara teks acuan (manual) dan teks hipotesis
       (Whisper).
    Akurasi transkripsi dinilai baik jika nilai WER berada di bawah 20%.
    """
    word_error_rate = wer(reference_text.lower(),
       hypothesis_text.lower())
    char_error_rate = cer(reference_text.lower(),
       hypothesis_text.lower())
    return {
        'WER': round(word_error_rate * 100, 2),
        'CER': round(char_error_rate * 100, 2)
    }

# Contoh pengujian (Ganti dengan teks referensi transkripsi manual Anda)
reference  = "teknologi kecerdasan buantan semakin berkembang pesat di
      indonesia"
hypothesis = "teknologi kecerdasan buatan semakin berkembang pesat di
      indonesia"

metrics = calculate_wer_sample(reference, hypothesis)

print("[INFO] Hasil Evaluasi Transkripsi Sampel:")
print(f"   - Word Error Rate (WER)      : {metrics['WER']}%")
print(f"   - Character Error Rate (CER)  : {metrics['CER']}%")
print("\n[INFO] Acuan Standar Kualitas:")
print("   - WER < 20%  : Baik (Akurasi transkripsi tinggi)")
print("   - WER 20-30% : Cukup (Teks masih dapat dipahami)")
print("   - WER > 30%  : Kurang (Disarankan meningkatkan varian
      model/kualitas audio)")

# ========================================
# CELL 8: Inisialisasi Pustaka Preprocessing NLP (Sastrawi & Stopwords)
# ========================================
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from Sastrawi.Stemmer.StemmerFactory import StemmerFactory
from Sastrawi.StopWordRemover.StopWordRemoverFactory import
      StopWordRemoverFactory

# Pastikan resource NLTK diunduh secara aman
nltk.download('punkt', quiet=True)
nltk.download('stopwords', quiet=True)

# 1. Inisialisasi Stemmer Bahasa Indonesia (PySastrawi Nazief-Adriani)
factory_stemmer = StemmerFactory()
stemmer = factory_stemmer.create_stemmer()

# 2. Stopwords Bahasa Indonesia bawaan Sastrawi
factory_sw = StopWordRemoverFactory()
sastrawi_stopwords = set(factory_sw.get_stop_words())

# 3. Stopwords Bahasa Indonesia bawaan NLTK
try:
    nltk_stopwords_id = set(stopwords.words('indonesian'))
except Exception:
    nltk_stopwords_id = set()

# 4. Stopwords Kustom khusus percakapan lisan / podcast (Filler Words &
      Kata Ganti Lisan)
```

```
custom_stopwords = {
    # Kata jeda/filler lisan
    'ya', 'iya', 'eh', 'ah', 'oh', 'um', 'mm', 'hmm',
    'gitu', 'gini', 'kayak', 'kaya', 'tuh', 'nih', 'sih',
    'dong', 'deh', 'kok', 'lho', 'lha', 'kan', 'udah',
    'nah', 'yuk', 'wah', 'wow', 'oke', 'ok',
    'baik', 'terima', 'kasih', 'sama', 'selamat', 'kali',

    # Kata ganti informal khas podcast
    'gue', 'gua', 'lu', 'lo', 'aku', 'kamu', 'guys', 'bro', 'sis',

    # Kata percakapan umum podcast
    'podcast', 'ngobrol', 'santai', 'cerita', 'bincang', 'halo',
    'episode', 'kali', 'sebenarnya', 'banget', 'bisa', 'jadi'
}

# Gabungkan seluruh stopwords menjadi satu set unik
ALL_STOPWORDS = sastrawi_stopwords | nltk_stopwords_id | custom_stopwords

print("[OK] Stemmer PySastrawi berhasil diinisialisasi.")
print(f"[OK] Total daftar kata buang (Stopwords): {len(ALL_STOPWORDS)}
      kata")

# ========================================
# CELL 9: Pipeline Preprocessing NLP Bahasa Indonesia
# ========================================
def preprocess_text(text, use_Stemming=True):
    """
    Pipeline Preprocessing NLP Bahasa Indonesia:
    1. Case folding (Konversi ke huruf kecil)
    2. Text Cleaning (Pembersihan URL, email, simbol, spasi ganda)
    3. Tokenisasi (Pemberhentian kata menjadi token tunggal)
    4. Stopword removal (Pembuangan kata umum & custom stopwords)
    5. Stemming (Menggunakan Sastrawi untuk mengembalikan kata dasar)
    6. Pembersihan duplikasi kata berturut-turut (akibat gagap suara)
    """
    if not isinstance(text, str) or len(text.strip()) == 0:
        return ''

    # ── Langkah 1: Case folding ──────────────────────────
    text = text.lower()

    # ── Langkah 2: Text Cleaning ─────────────────────────
    text = re.sub(r'http\S+|www\S+', ' ', text)          # Hapus URL
    text = re.sub(r'\S+@\S+', ' ', text)                  # Hapus email
    text = re.sub(r'[^a-zA-Z\s]', ' ', text)              # Hapus angka &
      simbol
    text = re.sub(r'\b[a-z]\b', ' ', text)                # Hapus huruf
      tunggal
    text = re.sub(r'\s+', ' ', text).strip()              # Normalisasi
      spasi

    # ── Langkah 3: Tokenisasi ────────────────────────────
    tokens = word_tokenize(text)

    # ── Langkah 4: Stopword removal ──────────────────────
    tokens = [t for t in tokens if t not in ALL_STOPWORDS and len(t) > 2]

    # ── Langkah 5: Stemming ──────────────────────────────
    if use_Stemming and tokens:
        tokens = [stemmer.stem(t) for t in tokens]

    # Langkah 6: Hapus duplikasi token berturut-turut (duplikasi Whisper)
```

```
    cleaned_tokens = []
    for i, t in enumerate(tokens):
        if i == 0 or t != tokens[i-1]:
            cleaned_tokens.append(t)

    return ' '.join(cleaned_tokens)


# Test Preprocessing
sample = "Halo selamat datang di podcast teknologi, hari ini kita akan
       membahas perkembangan kecerdasan buatan dan machine learning yang
       semakin pesat berkembang di Indonesia!"
print("[INFO] Uji Coba Preprocessing Teks:")
print(f"   - Teks Asli   : \"{sample}\"")
print(f"   - Teks Bersih : \"{preprocess_text(sample)}\"")

# ========================================
# CELL 11: Terapkan Preprocessing ke Seluruh Dataset (Fixed)
# ========================================
import os
import json
import pandas as pd
import matplotlib.pyplot as plt
from tqdm.auto import tqdm

# 1. Definisikan path file (Pastikan nama file SAMA dan KONSISTEN)
DATASET_RAW_PATH   = f'{BASE_DIR}/data/dataset/dataset_transkripsi.csv'
DATASET_CLEAN_PATH =
       f'{BASE_DIR}/data/dataset/dataset_transkripsi_clean.csv'  #
       Disesuaikan (tanpa angka 1)

# 2. Load dataset mentah
if not os.path.exists(DATASET_RAW_PATH):
    raise FileNotFoundError(f"File mentah tidak ditemukan di:
       {DATASET_RAW_PATH}. Pastikan file sudah diunggah ke Google
       Drive.")

df = pd.read_csv(DATASET_RAW_PATH, encoding='utf-8')
print(f"[INFO] Dataset mentah dibaca dari Drive: {len(df)} baris")

# 3. Normalisasi Nama Kolom
df.columns = df.columns.str.strip().str.lower()

# Mengecek dan mengubah kolom teks (teks_transkrip / teks_transkripsi /
       text) menjadi 'teks'
for col_candidate in ['teks_transkrip', 'teks_transkripsi', 'text']:
    if col_candidate in df.columns and 'teks' not in df.columns:
        df.rename(columns={col_candidate: 'teks'}, inplace=True)
        print(f"[INFO] Kolom '{col_candidate}' dinormalisasi menjadi
       'teks'")

if 'teks' not in df.columns and 'title' in df.columns:
    df['teks'] = df['title']
    print("[INFO] Menggunakan kolom 'title' sebagai fallback kolom
       'teks'")

# 4. Filtering Awal (Hapus teks kosong & teks pendek)
df = df.dropna(subset=['teks'])
df = df[df['teks'].astype(str).str.strip().str.len() > 50]
print(f"[INFO] Setelah filtering awal (teks > 50 karakter): {len(df)}
       baris")

# 5. Terapkan Preprocessing NLP
```

```
tqdm.pandas()
print("\n[INFO] Menjalankan Preprocessing NLP (Stemming & Stopwords)...")
df['teks_bersih'] = df['teks'].progress_apply(lambda x:
      preprocess_text(x, use_Stemming=True))

# 6. Filtering Akhir
df = df[df['teks_bersih'].str.strip().str.len() > 10]
print(f"[INFO] Setelah Preprocessing & filtering akhir: {len(df)} baris")

# 7. Pemetaan Label ke ID secara Dinamis
unique_labels = sorted(df['label'].unique())
label2id = {label: idx for idx, label in enumerate(unique_labels)}
id2label = {idx: label for label, idx in label2id.items()}

# Simpan konfigurasi label ke berkas JSON
os.makedirs(f'{BASE_DIR}/models', exist_ok=True)
with open(f'{BASE_DIR}/models/label_config.json', 'w') as f_cfg:
    json.dump({"label2id": label2id, "id2label": id2label}, f_cfg,
      indent=2)

df['label_id'] = df['label'].map(label2id)

# 8. Simpan dataset bersih ke path utama (dataset_transkripsi_clean.csv)
os.makedirs(os.path.dirname(DATASET_CLEAN_PATH), exist_ok=True)
df.to_csv(DATASET_CLEAN_PATH, index=False, encoding='utf-8')
print(f"\n[OK] Dataset bersih berhasil disimpan ke:
      {DATASET_CLEAN_PATH}")
print(f"[INFO] Distribusi kelas dataset final ({len(unique_labels)}
      kelas):")
print(df['label'].value_counts())

# ── 9. Visualisasi Distribusi Kelas
      ──────────────────────────────────────
os.makedirs(f'{BASE_DIR}/outputs/figures', exist_ok=True)

plt.figure(figsize=(10, 4.5))
colors = ['#6366f1', '#a855f7', '#ec4899', '#3b82f6', '#10b981',
      '#f59e0b', '#ef4444'][:len(unique_labels)]
df['label'].value_counts().plot(kind='bar', color=colors)
plt.title(f'Distribusi Kelas Dataset Podcast ({len(unique_labels)}
      Kelas)', fontsize=12, fontweight='bold')
plt.xlabel('Kategori Topik', fontsize=10, fontweight='bold')
plt.ylabel('Jumlah Sampel', fontsize=10, fontweight='bold')
plt.xticks(rotation=15)
plt.tight_layout()

fig_path = f'{BASE_DIR}/outputs/figures/distribusi_kelas.png'
plt.savefig(fig_path, dpi=150)
plt.show()
print(f"[INFO] Visualisasi distribusi kelas disimpan ke: {fig_path}")

# ========================================
# CELL 12: Analisis Distribusi Kelas Dataset (EDA)
# ========================================
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os

# 1. Load dataset bersih hasil Preprocessing
dataset_path = f'{BASE_DIR}/data/dataset/dataset_clean.csv'

if not os.path.exists(dataset_path):
```

```
    print(f"[PERINGATAN] Berkas dataset bersih tidak ditemukan di:
       {dataset_path}")
    print("Harap selesaikan proses Preprocessing di sel atas terlebih
       dahulu.")
else:
    df = pd.read_csv(dataset_path)

    # Hitung jumlah sampel per kelas/topik
    class_counts = df['label'].value_counts()
    print("="*50)
    print("DISTRIBUSI KELAS DATASET PODCAST:")
    print("="*50)
    for cls, cnt in class_counts.items():
        print(f" - Kategori {cls.capitalize():12s} : {cnt:3d} sampel")
    print(f"Total Sampel Keseluruhan : {len(df)} sampel\n")

    # 2. Visualisasi Grafik Batang Distribusi Kelas (Premium Styling)
    plt.figure(figsize=(8, 5))

    # Gunakan palet warna yang premium terkoordinasi
    colors = ['#6366f1', '#a855f7', '#ec4899', '#3b82f6']

    sns.set_style("whitegrid")
    ax = sns.barplot(
        x=class_counts.index.map(str.capitalize),
        y=class_counts.values,
        palette=colors,
        hue=class_counts.index.map(str.capitalize),
        legend=False
    )

    # Tambahkan label angka di atas setiap batang grafik
    for p in ax.patches:
        ax.annotate(
            f"{int(p.get_height())}",
            (p.get_x() + p.get_width() / 2., p.get_height()),
            ha='center', va='center',
            xy*text*=(0, 8),
            *text*coords='offset points',
            fontsize=10,
            fontweight='bold',
            color='#374151'
        )

    plt.title('Distribusi Kategori Topik Podcast (Dataset Bersih)',
       fontsize=14, fontweight='bold', pad=15)
    plt.xlabel('Kategori Topik', fontsize=11, fontweight='bold',
       labelpad=10)
    plt.ylabel('Jumlah Sampel', fontsize=11, fontweight='bold',
       labelpad=10)
    plt.ylim(0, max(class_counts.values) * 1.15)
    plt.grid(axis='y', linestyle='--', alpha=0.5)

    # Simpan visualisasi grafik distribusi
    os.makedirs(f'{BASE_DIR}/outputs/figures', exist_ok=True)
    fig_dist_path =
       f'{BASE_DIR}/outputs/figures/dataset_distribution.png'
    plt.savefig(fig_dist_path, dpi=150, bbox_inches='tight')
    plt.show()
    print(f"[INFO] Grafik distribusi dataset berhasil disimpan ke:
       {fig_dist_path}")

# ========================================
```

```
# CELL 13: Persiapan Data & Pembobotan Fitur TF-IDF
# ==========================================
import os
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer

# 1. Load dataset bersih dari Cell 11
clean_path = f'{BASE_DIR}/data/dataset/dataset_transkripsi_clean.csv'
if not os.path.exists(clean_path):
    clean_path = 'dataset_transkripsi_clean.csv'

df = pd.read_csv(clean_path, encoding='utf-8')
df = df.dropna(subset=['teks_bersih', 'label_id']).reset_index(drop=True)

X = df['teks_bersih'].values
y = df['label_id'].values.astype(int)

# 2. Bagi dataset (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"[INFO] Dataset terbagi:")
print(f"   - Sampel Data Latih (Training) : {len(X_train)}")
print(f"   - Sampel Data Uji (Testing)    : {len(X_test)}")

# 3. Inisialisasi TF-IDF Vectorizer
tfidf_vectorizer = TfidfVectorizer(
    ngram_range=(1, 2),
    max_features=5000,
    sublinear_tf=True,
    min_df=2,
    analyzer='word',
    strip_accents='unicode'
)

# 4. Transformasi fitur -> Menghasilkan X_train_tfidf dan X_test_tfidf
X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)
X_test_tfidf  = tfidf_vectorizer.transform(X_test)

print(f"\n[OK] Pembobotan TF-IDF selesai!")
print(f"   - Dimensi Matriks Data Latih : {X_train_tfidf.shape}")
print(f"   - Dimensi Matriks Data Uji   : {X_test_tfidf.shape}")

# 5. Simpan objek vectorizer
VECTORIZER_PATH = f'{BASE_DIR}/models/sklearn/tfidf_vectorizer.pkl'
os.makedirs(os.path.dirname(VECTORIZER_PATH), exist_ok=True)
joblib.dump(tfidf_vectorizer, VECTORIZER_PATH)
print(f"[INFO] Objek TF-IDF Vectorizer disimpan ke: {VECTORIZER_PATH}")

# ==========================================
# CELL 15: Definisi Model Machine Learning Klasik
# ==========================================
from sklearn.calibration import CalibratedClassifierCV
from sklearn.svm import LinearSVC
from sklearn.naive_bayes import MultinomialNB

# Mendefinisikan kamus model untuk dibandingkan (Hanya Naive Bayes dan
      SVM Linear)
MODELS = {
    'Naive Bayes': MultinomialNB(alpha=1.0),
```

```
    'SVM (Linear)': CalibratedClassifierCV(LinearSVC(C=1.0,
      max_iter=10000, random_state=42)),
}

print(f"[OK] Model ML Klasik berhasil dikonfigurasi:
      {list(MODELS.keys())}")

# ========================================
# CELL 16: Pelatihan Model dengan 5-Fold Stratified Cross-Validation
# ========================================
# Inisialisasi Stratified K-Fold
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

cv_results_summary = {}
trained_models     = {}

# Matriks skor evaluasi yang dicatat selama cross-validation
SCORING = ['accuracy', 'precision_weighted', 'recall_weighted',
      'f1_weighted']

for model_name, model in MODELS.items():
    print(f"\n[RUN] Memulai Pelatihan: {model_name}")
    print("—" * 50)

    start_time = time.time()

    # 1. Menjalankan 5-Fold Cross-Validation secara paralel (n_jobs=-1)
    cv_results = cross_validate(
        model,
        X_train_tfidf, y_train,
        cv=skf,
        scoring=SCORING,
        return_train_score=False,
        n_jobs=-1
    )

    elapsed_time = time.time() - start_time

    # 2. Merekap rata-rata metrik evaluasi
    summary = {
        'Accuracy'  : cv_results['test_accuracy'].mean(),
        'Precision' : cv_results['test_precision_weighted'].mean(),
        'Recall'    : cv_results['test_recall_weighted'].mean(),
        'F1-Score'  : cv_results['test_f1_weighted'].mean(),
        'Std F1'    : cv_results['test_f1_weighted'].std(),
        'Waktu (s)' : round(elapsed_time, 2)
    }
    cv_results_summary[model_name] = summary

    # Cetak hasil metrik ke konsol
    for metric_name, value in summary.items():
        if isinstance(value, float):
            print(f"   - {metric_name:12s}: {value:.4f}")
        else:
            print(f"   - {metric_name:12s}: {value}")

    # 3. Fit final model menggunakan seluruh data latih (X_train)
    model.fit(X_train_tfidf, y_train)
    trained_models[model_name] = model

    # 4. Ekspor objek model hasil latih ke Google Drive
    safe_name = model_name.lower().replace(' ', '_').replace('(',
      '').replace(')', '')
```

```
    model_path = f'{BASE_DIR}/models/sklearn/{safe_name}_model.pkl'
    joblib.dump(model, model_path)
    print(f"   [INFO] Objek model berhasil diekspor ke: {model_path}")

print("\n[OK] Pelatihan seluruh model ML klasik selesai!")

# ========================================
# CELL 17: Evaluasi Hasil Prediksi Seluruh Model ML Klasik (Data Uji
      Hold-out)
# ========================================
# 1. Dapatkan nama label secara dinamis dari label unik pada dataset
      (Cell 12)
label_names_cap = [lbl.capitalize() for lbl in unique_labels]
test_results = {}

# 2. Lakukan pengujian hold-out pada masing-masing model
for model_name, model in trained_models.items():
    y_pred = model.predict(X_test_tfidf)

    test_results[model_name] = {
        'Accuracy'  : accuracy_score(y_test, y_pred),
        'Precision' : precision_score(y_test, y_pred,
      average='weighted'),
        'Recall'    : recall_score(y_test, y_pred, average='weighted'),
        'F1-Score'  : f1_score(y_test, y_pred, average='weighted'),
    }

    print(f"\nLaporan Klasifikasi ({model_name}) — Test Set:")
    print(classification_report(y_test, y_pred,
      target_names=label_names_cap))

# 3. Buat dataframe komparasi performa model
df_test_results = pd.DataFrame(test_results).T.round(4)
print("\nRingkasan Performa Komparatif Model ML Klasik:")
print(df_test_results.to_string())

# 4. Ekspor hasil perbandingan performa ke Google Drive
report_dir = f'{BASE_DIR}/outputs/reports'
os.makedirs(report_dir, exist_ok=True)
report_csv_path = f'{report_dir}/ml_test_results.csv'
df_test_results.to_csv(report_csv_path)

# ========================================
# CELL 18: Visualisasi Confusion Matrix untuk Seluruh Model ML Klasik
# ========================================
num_models = len(trained_models)
cols = 3
rows = (num_models + cols - 1) // cols

# Buat grid subplot dinamis berdasarkan jumlah model yang dilatih
fig, axes = plt.subplots(rows, cols, figsize=(6 * cols, 5 * rows))
axes = axes.flatten() if num_models > 1 else [axes]

# Gambarkan confusion matrix pada masing-masing subplot
for idx, (model_name, model) in enumerate(trained_models.items()):
    ax = axes[idx]
    y_pred = model.predict(X_test_tfidf)
    cm = confusion_matrix(y_test, y_pred)

    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=label_names_cap, yticklabels=label_names_cap,
      ax=ax)
```

```
    ax.set_title(f'{model_name}\nF1-Score =
      {test_results[model_name]["F1-Score"]:.4f}', fontsize=12)
    ax.set_ylabel('Aktual')
    ax.set_xlabel('Prediksi')

# Hapus kotak visualisasi kosong jika jumlah model tidak memenuhi
      kelipatan kolom
for idx in range(num_models, len(axes)):
    fig.delaxes(axes[idx])

plt.suptitle('Confusion Matrix — Model ML Klasik (Test Set)',
      fontsize=14, y=1.02)
plt.tight_layout()

# Simpan gambar visualisasi
fig_out_path = f'{BASE_DIR}/outputs/figures/confusion_matrix_ml.png'
plt.savefig(fig_out_path, dpi=150, bbox_inches='tight')
plt.show()
print(f"[INFO] Visualisasi Confusion Matrix disimpan ke: {fig_out_path}")

# ========================================
# CELL 19: Komparasi Kinerja Seluruh Model Klasifikasi ML Klasik
# ========================================
# 1. Kumpulkan metrik performa seluruh model ML klasik
comparison_data = {}
for model_name, model in trained_models.items():
    y_pred = model.predict(X_test_tfidf)
    comparison_data[model_name] = {
        'Accuracy'  : round(accuracy_score(y_test, y_pred), 4),
        'Precision' : round(precision_score(y_test, y_pred,
      average='weighted'), 4),
        'Recall'    : round(recall_score(y_test, y_pred,
      average='weighted'), 4),
        'F1-Score'  : round(f1_score(y_test, y_pred, average='weighted'),
      4),
    }

# 2. Buat Dataframe perbandingan terurut berdasarkan skor F1 tertinggi
df_comparison = pd.DataFrame(comparison_data).T
df_comparison = df_comparison.sort_values('F1-Score', ascending=False)

print("=" * 60)
print("[INFO] TABEL PERBANDINGAN PERFORMA MODEL KLASIFIKASI")
print("=" * 60)
print(df_comparison.to_string())
print("=" * 60)

best_model_name = df_comparison['F1-Score'].idxmax()
best_f1 = df_comparison.loc[best_model_name, 'F1-Score']
print(f"\n[OK] Model Terbaik Dinilai dari F1-Score: {best_model_name}
      (F1-Score = {best_f1:.4f})")

# 3. Ekspor tabel komparasi ke Google Drive
os.makedirs(f'{BASE_DIR}/outputs/reports', exist_ok=True)
comparison_csv_path = f'{BASE_DIR}/outputs/reports/model_comparison.csv'
df_comparison.to_csv(comparison_csv_path)
print(f"[INFO] Tabel komparasi berhasil diekspor ke:
      {comparison_csv_path}")

# ── 4. Visualisasi Grafik Batang Perbandingan Model
      ──────────────────────────
metrics = ['Accuracy', 'Precision', 'Recall', 'F1-Score']
models_list = list(comparison_data.keys())
```

```
# Menggunakan 5 warna premium terkoordinasi untuk 5 model klasik
colors = ['#6366f1', '#a855f7', '#ec4899', '#3b82f6', '#10b981']

fig, ax = plt.subplots(figsize=(12, 6))
x = np.arange(len(metrics))
width = 0.8 / len(models_list)

for i, model_name in enumerate(models_list):
    color = colors[i % len(colors)]
    values = [comparison_data[model_name][m] for m in metrics]

    offset = (i - (len(models_list) - 1) / 2) * width
    bars = ax.bar(x + offset, values, width, label=model_name,
       color=color, alpha=0.85)

    for bar, val in zip(bars, values):
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() +
       0.005,
                f'{val:.3f}', ha='center', va='bottom', fontsize=8)

ax.set_ylim(0.5, 1.0)
ax.set_xticks(x)
ax.set_xticklabels(metrics)
ax.set_ylabel('Skor')
ax.set_title('Perbandingan Performa Model Klasifikasi Topik Podcast')
ax.legend(loc='lower right')
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()

# Simpan gambar perbandingan model
fig_comp_path = f'{BASE_DIR}/outputs/figures/model_comparison_bar.png'
plt.savefig(fig_comp_path, dpi=150)
plt.show()
print(f"[INFO] Visualisasi perbandingan model disimpan ke:
       {fig_comp_path}")

# ========================================
# CELL 20: Ekspor Model Terbaik & Sinkronisasi Konfigurasi Label
# ========================================
# 1. Ambil nama model terbaik secara otomatis dari hasil komparasi
BEST_MODEL_NAME = best_model_name
print(f"[INFO] Mengekspor objek model terbaik: {BEST_MODEL_NAME}")

# 2. Ekspor objek model klasik terbaik & vectorizer TF-IDF ke Google
       Drive
os.makedirs(f'{BASE_DIR}/models/sklearn', exist_ok=True)
best_model_obj = trained_models[BEST_MODEL_NAME]

joblib.dump(best_model_obj, f'{BASE_DIR}/models/sklearn/best_model.pkl')
joblib.dump(tfidf_vectorizer,
       f'{BASE_DIR}/models/sklearn/tfidf_vectorizer.pkl')
print(f"   [OK] Model {BEST_MODEL_NAME} disimpan sebagai:
       best_model.pkl")

# 3. Simpan konfigurasi label dinamis & nama model terbaik untuk dibaca
       oleh Flask Web App
label_config = {
    'label2id': label2id,    # Dinamis dari Cell 12
    'id2label': id2label,    # Dinamis dari Cell 12
    'best_model': BEST_MODEL_NAME
}
```

```
config_json_path = f'{BASE_DIR}/models/label_config.json'
with open(config_json_path, 'w', encoding='utf-8') as f_cfg:
    json.dump(label_config, f_cfg, indent=2, ensure_ascii=False)

print(f"[OK] Berkas label_config.json berhasil diselaraskan ke:
      {config_json_path}")
print("\n[INFO] Daftar artefak model siap pakai untuk webapp:")
print(f"   - Vectorizer TF-IDF :
      {BASE_DIR}/models/sklearn/tfidf_vectorizer.pkl")
print(f"   - Model ML Klasik   :
      {BASE_DIR}/models/sklearn/best_model.pkl")
print(f"   - Konfigurasi Label : {BASE_DIR}/models/label_config.json")
# ── 4. Ekspor Laporan Klasifikasi Model Terbaik ke File TXT & CSV
      ──────────────
from sklearn.metrics import classification_report
y_pred_best = best_model_obj.predict(X_test_tfidf)

# Simpan format teks laporan klasifikasi
report_text = classification_report(y_test, y_pred_best,
      target_names=label_names_cap)
report_txt_path =
      f'{BASE_DIR}/outputs/reports/best_model_classification_report.txt'
with open(report_txt_path, 'w', encoding='utf-8') as f_rep:
    f_rep.write(f"MODEL TERBAIK: {BEST_MODEL_NAME}\n")
    f_rep.write("="*60 + "\n")
    f_rep.write(report_text)
print(f"[INFO] Laporan Klasifikasi (TXT) disimpan ke: {report_txt_path}")

# Simpan format CSV untuk kemudahan penyalinan tabel skripsi
report_dict = classification_report(y_test, y_pred_best,
      target_names=label_names_cap, output_dict=True)
df_report = pd.DataFrame(report_dict).transpose()
report_csv_path =
      f'{BASE_DIR}/outputs/reports/best_model_classification_report.csv'
df_report.to_csv(report_csv_path)
print(f"[INFO] Laporan Klasifikasi (CSV) disimpan ke: {report_csv_path}")

# ==========================================
# CELL 21: Inisialisasi Direktori Web Application
# ==========================================
# Inisialisasi Direktori Web Application
WEBAPP_DIR = f'{BASE_DIR}/webapp'

# Membuat folder webapp, templates, static, dan uploads secara aman jika
      belum ada
os.makedirs(f'{WEBAPP_DIR}/templates', exist_ok=True)
os.makedirs(f'{WEBAPP_DIR}/static', exist_ok=True)
os.makedirs(f'{WEBAPP_DIR}/uploads', exist_ok=True)

print(f"[OK] Direktori Web Application berhasil dikonfigurasi:")
print(f"   - Web App Root : {WEBAPP_DIR}")
print(f"   - Uploads Temp : {WEBAPP_DIR}/uploads")

# ==========================================
# CELL 22: Tulis app.py (100% Murni Whisper STT & Machine Learning)
# ==========================================
app_py_code = '''
import os, re, json, time
import joblib
import numpy as np
import whisper
import torch
from flask import Flask, request, jsonify, render_template
```

```
import nltk
from nltk.tokenize import word_tokenize
from Sastrawi.Stemmer.StemmerFactory import StemmerFactory
from Sastrawi.StopWordRemover.StopWordRemoverFactory import
       StopWordRemoverFactory

nltk.download("punkt", quiet=True)

app = Flask(__name__)
app.config["MAX_CONTENT_LENGTH"] = 200 * 1024 * 1024  # Maksimal 200 MB
UPLOAD_FOLDER = "uploads"
os.makedirs(UPLOAD_FOLDER, exist_ok=True)

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
PARENT_DIR = os.path.dirname(BASE_DIR)

# ── 1. Load Model Machine Learning & Whisper STT saat Startup
      ──────────────
print("[INFO] Memuat Model Klasifikasi Topik Podcast & Whisper STT...")

# Standar urutan alfabetis scikit-learn
id2label = {
    0: "berita", 1: "bisnis", 2: "hiburan",
    3: "kesehatan", 4: "olahraga", 5: "pendidikan", 6: "teknologi"
}

possible_dirs = [
    os.path.join(PARENT_DIR, "models", "sklearn"),
    os.path.join(PARENT_DIR, "models"),
    os.path.join(PARENT_DIR, "model"),
    os.path.join(BASE_DIR, "models", "sklearn"),
    os.path.join(BASE_DIR, "models"),
    os.path.join(BASE_DIR, "model"),
    "/content/drive/MyDrive/PodcastClassifier/models/sklearn",
    "/content/drive/MyDrive/PodcastClassifier/models",
    "/content/drive/MyDrive/PodcastClassifier/model",
    "./models/sklearn", "./models", "./model"
]

for d in possible_dirs:
    cfg_p = os.path.join(d, "label_config.json")
    if os.path.exists(cfg_p):
        try:
            with open(cfg_p, "r", encoding="utf-8") as f:
                cfg = json.load(f)
                if "id2label" in cfg:
                    id2label = {int(k): v for k, v in
      cfg["id2label"].items()}
            print(f"[OK] Konfigurasi Label dimuat dari {cfg_p}")
            break
        except Exception:
            pass

vectorizer = None
classifier = None

for d in possible_dirs:
    v_path = os.path.join(d, "tfidf_vectorizer.pkl")
    m_path = os.path.join(d, "best_model.pkl")

    if os.path.exists(v_path) and vectorizer is None:
        try:
            vectorizer = joblib.load(v_path)
```

```
            print(f"[OK] TF-IDF Vectorizer dimuat dari {v_path}")
        except Exception as e:
            print(f"[WARNING] Gagal memuat {v_path}: {e}")

    if os.path.exists(m_path) and classifier is None:
        try:
            classifier = joblib.load(m_path)
            print(f"[OK] Classifier ML Klasik dimuat dari {m_path}")
        except Exception as e:
            print(f"[WARNING] Gagal memuat {m_path}: {e}")

# Load Whisper STT Model ('medium' di GPU Colab)
device = "cuda" if torch.cuda.is_available() else "cpu"
whisper_model = whisper.load_model("medium", device=device)
print(f"[OK] Model Whisper STT berhasil dimuat pada perangkat
      [{device.upper()}]!")

# Inisialisasi Tool Preprocessing NLP Sastrawi & Stopwords Lisan Podcast
factory_sw = StopWordRemoverFactory()
stopwords_base = factory_sw.get_stop_words()
custom_stopwords = {
    'ya', 'iya', 'eh', 'ah', 'oh', 'um', 'mm', 'hmm', 'gitu', 'gini',
      'kayak', 'kaya', 'tuh', 'nih', 'sih',
    'dong', 'deh', 'kok', 'lho', 'lha', 'kan', 'udah', 'nah', 'yuk',
      'wah', 'wow', 'oke', 'ok',
    'baik', 'terima', 'kasih', 'sama', 'selamat', 'gue', 'gua', 'lu',
      'lo', 'aku', 'kamu', 'guys', 'bro', 'sis',
    'podcast', 'ngobrol', 'santai', 'cerita', 'bincang', 'halo',
      'episode', 'kali', 'sebenarnya', 'banget'
}
stopwords_set = set(stopwords_base + list(custom_stopwords))

factory_st = StemmerFactory()
stemmer = factory_st.create_stemmer()

def preprocess(text):
    text = text.lower()
    text = re.sub(r"[^a-zA-Z\\s]", " ", text)
    text = re.sub(r"\\s+", " ", text).strip()
    tokens = word_tokenize(text)
    tokens = [t for t in tokens if t not in stopwords_set and len(t) > 2]
    tokens = [stemmer.stem(t) for t in tokens]
    return " ".join(tokens)


# ── 2. Route Utama & API Endpoint (100% Murni STT Whisper + ML)
      ──────────────

@app.route("/")
def index():
    return render_template("index.html")


@app.route("/predict", methods=["POST"])
def predict():
    if vectorizer is None or classifier is None:
        return jsonify({"error": "Berkas model tfidf_vectorizer.pkl /
      best_model.pkl belum dimuat. Harap jalankan CELL 20 (Ekspor Model)
      di Colab terlebih dahulu."}), 500

    if "audio" not in request.files:
        return jsonify({"error": "Tidak ada berkas audio yang
      diunggah"}), 400
```

```
    audio_file = request.files["audio"]
    if audio_file.filename == "":
        return jsonify({"error": "Nama berkas audio kosong"}), 400

    filepath = os.path.join(UPLOAD_FOLDER, audio_file.filename)
    audio_file.save(filepath)

    try:
        t0 = time.time()

        # Step 1: Transkripsi Murni Suara Audio dengan Whisper STT
        audio = whisper.load_audio(filepath)
        max_samples = 600 * 16000  # Maksimal 10 menit pertama
        if len(audio) > max_samples:
            audio = audio[:max_samples]

        result = whisper_model.transcribe(audio, language="id",
       fp16=False)
        transcript = result["text"].strip()
        stt_time = round(time.time() - t0, 2)

        # Step 2: Preprocessing NLP Sastrawi
        clean_text = preprocess(transcript)

        # Step 3: Prediksi Murni Model Machine Learning (SVM)
        vec = vectorizer.transform([clean_text])
        pred_id = int(classifier.predict(vec)[0])
        pred_label = id2label.get(pred_id, "berita")

        confidence_score = 95.0
        if hasattr(classifier, "predict_proba"):
            pred_proba = classifier.predict_proba(vec)[0]
            confidence_score = round(float(np.max(pred_proba)) * 100, 1)

        response = {
            "transcript": transcript,
            "clean_text": clean_text,
            "prediction": pred_label,
            "confidence": confidence_score,
            "stt_time_sec": stt_time
        }
    except Exception as e:
        response = {"error": f"Gagal memproses audio: {str(e)}"}
    finally:
        if filepath and os.path.exists(filepath):
            try:
                os.remove(filepath)
            except Exception:
                pass

    return jsonify(response)


if __name__ == "__main__":
    print("=" * 70)
    print(" 🚀 KLASIFIKASI TOPIK PODCAST SERVER RUNNING ON
      http://127.0.0.1:5000")
    print("=" * 70)
    app.run(debug=True, host="0.0.0.0", port=5000)
'''

with open(f'{WEBAPP_DIR}/app.py', 'w', encoding='utf-8') as f:
```

```
    f.write(app_py_code.strip())

print("☑ app.py (Klasifikasi Topik Podcast) berhasil dibuat di Google
      Colab!")

# ========================================
# CELL 23: Tulis index.html (UI Klasifikasi Topik Podcast Modern Dark
      Mode - 200 MB)
# ========================================

html_code = '''<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
      scale=1.0">
    <title>Klasifikasi Topik Podcast — AI Powered</title>
    <!-- Modern Google Typography -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link
      href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:w
      ght@300;400;500;600;700;800&display=swap" rel="stylesheet">
    <style>
        :root {
            --bg-dark: #080c14;
            --card-bg: rgba(15, 23, 42, 0.65);
            --card-border: rgba(255, 255, 255, 0.08);
            --card-border-hover: rgba(99, 102, 241, 0.4);

            --primary: #6366f1;
            --primary-gradient: linear-gradient(135deg, #6366f1 0%,
      #a855f7 100%);
            --accent-cyan: #06b6d4;
            --accent-purple: #a855f7;

            --text-main: #f8fafc;
            --text-muted: #94a3b8;
            --text-dim: #64748b;

            --radius-lg: 24px;
            --radius-md: 16px;
            --radius-sm: 12px;
            --transition: all 0.3s cubic-bezier(0.4, 0, 0.2, 1);
        }

        * {
            box-sizing: border-box;
            margin: 0;
            padding: 0;
            font-family: 'Plus Jakarta Sans', sans-serif;
            -webkit-font-smoothing: antialiased;
        }

        body {
            background-color: var(--bg-dark);
            color: var(--text-main);
            min-height: 100vh;
            display: flex;
            flex-direction: column;
            align-items: center;
            justify-content: center;
```

```
    padding: 40px 20px;
    overflow-x: hidden;
    position: relative;
}

.ambient-glow {
    position: fixed;
    width: 600px;
    height: 600px;
    border-radius: 50%;
    filter: blur(140px);
    pointer-events: none;
    z-index: 0;
    opacity: 0.25;
}

.glow-1 {
    top: -150px;
    left: -150px;
    background: radial-gradient(circle, #6366f1 0%, rgba(99, 102,
241, 0) 70%);
}

.glow-2 {
    bottom: -150px;
    right: -150px;
    background: radial-gradient(circle, #a855f7 0%, rgba(168, 85,
247, 0) 70%);
}

.app-container {
    width: 100%;
    max-width: 960px;
    z-index: 1;
    position: relative;
}

header {
    text-align: center;
    margin-bottom: 40px;
}

.badge-pill {
    display: inline-flex;
    align-items: center;
    gap: 8px;
    padding: 6px 16px;
    border-radius: 30px;
    background: rgba(99, 102, 241, 0.12);
    border: 1px solid rgba(99, 102, 241, 0.3);
    color: #818cf8;
    font-size: 0.82rem;
    font-weight: 600;
    letter-spacing: 0.5px;
    text-transform: uppercase;
    margin-bottom: 16px;
    box-shadow: 0 0 20px rgba(99, 102, 241, 0.15);
}

.badge-dot {
    width: 7px;
    height: 7px;
    border-radius: 50%;
```

```
        background-color: #818cf8;
        box-shadow: 0 0 10px #818cf8;
    }

    h1 {
        font-size: 2.75rem;
        font-weight: 800;
        line-height: 1.2;
        letter-spacing: -1px;
        background: linear-gradient(180deg, #ffffff 0%, #cbd5e1
100%);
        -webkit-background-clip: text;
        -webkit-text-fill-color: transparent;
        margin-bottom: 12px;
    }

    .subtitle {
        font-size: 1.1rem;
        color: var(--text-muted);
        font-weight: 400;
        max-width: 580px;
        margin: 0 auto;
    }

    .glass-card {
        background: var(--card-bg);
        backdrop-filter: blur(20px);
        -webkit-backdrop-filter: blur(20px);
        border: 1px solid var(--card-border);
        border-radius: var(--radius-lg);
        padding: 40px;
        box-shadow: 0 25px 50px -12px rgba(0, 0, 0, 0.5);
        transition: var(--transition);
    }

    .drop-zone {
        border: 2px dashed rgba(99, 102, 241, 0.3);
        border-radius: var(--radius-md);
        padding: 48px 24px;
        text-align: center;
        cursor: pointer;
        transition: var(--transition);
        background: rgba(15, 23, 42, 0.4);
        position: relative;
        overflow: hidden;
    }

    .drop-zone:hover, .drop-zone.dragover {
        border-color: var(--primary);
        background: rgba(99, 102, 241, 0.08);
        transform: translateY(-2px);
        box-shadow: 0 15px 30px rgba(99, 102, 241, 0.15);
    }

    .icon-wrapper {
        width: 72px;
        height: 72px;
        border-radius: 20px;
        background: var(--primary-gradient);
        display: flex;
        align-items: center;
        justify-content: center;
        margin: 0 auto 20px auto;
```

```
    box-shadow: 0 10px 25px rgba(99, 102, 241, 0.3);
    transition: var(--transition);
}

.drop-zone:hover .icon-wrapper {
    transform: scale(1.08) rotate(-3deg);
}

.icon-wrapper svg {
    width: 36px;
    height: 36px;
    fill: none;
    stroke: #ffffff;
    stroke-width: 2.2;
    stroke-linecap: round;
    stroke-linejoin: round;
}

.drop-title {
    font-size: 1.25rem;
    font-weight: 700;
    color: var(--text-main);
    margin-bottom: 8px;
}

.drop-info {
    font-size: 0.88rem;
    color: var(--text-muted);
}

.file-preview {
    display: none;
    margin-top: 24px;
    background: rgba(30, 41, 59, 0.6);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: var(--radius-md);
    padding: 20px 24px;
    align-items: center;
    gap: 16px;
    animation: fadeIn 0.4s ease-out forwards;
}

.file-icon {
    width: 48px;
    height: 48px;
    border-radius: 12px;
    background: rgba(99, 102, 241, 0.15);
    display: flex;
    align-items: center;
    justify-content: center;
    color: #818cf8;
    flex-shrink: 0;
}

.file-details {
    flex-grow: 1;
    overflow: hidden;
}

.file-name {
    font-weight: 700;
    font-size: 1rem;
    color: var(--text-main);
```

```
    white-space: nowrap;
    overflow: hidden;
    text-overflow: ellipsis;
    margin-bottom: 4px;
}

.file-meta-row {
    display: flex;
    gap: 16px;
    font-size: 0.82rem;
    color: var(--text-muted);
}

.meta-tag {
    display: inline-flex;
    align-items: center;
    gap: 4px;
}

.btn-primary {
    width: 100%;
    margin-top: 28px;
    padding: 18px 32px;
    border-radius: var(--radius-md);
    border: none;
    background: var(--primary-gradient);
    color: #ffffff;
    font-size: 1.05rem;
    font-weight: 700;
    cursor: pointer;
    transition: var(--transition);
    box-shadow: 0 10px 30px rgba(99, 102, 241, 0.35);
    display: flex;
    align-items: center;
    justify-content: center;
    gap: 10px;
}

.btn-primary:hover:not(:disabled) {
    transform: translateY(-2px);
    box-shadow: 0 15px 35px rgba(99, 102, 241, 0.5);
    filter: brightness(1.08);
}

.btn-primary:disabled {
    opacity: 0.5;
    cursor: not-allowed;
    box-shadow: none;
}

.progress-section {
    display: none;
    animation: fadeIn 0.4s ease-out forwards;
}

.progress-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 12px;
}

.progress-title {
```

```
    font-size: 1.1rem;
    font-weight: 700;
    color: var(--text-main);
}

.progress-val {
    font-size: 1rem;
    font-weight: 800;
    color: #818cf8;
}

.progress-bar-bg {
    width: 100%;
    height: 10px;
    background: rgba(30, 41, 59, 0.8);
    border-radius: 20px;
    overflow: hidden;
    margin-bottom: 32px;
    border: 1px solid rgba(255, 255, 255, 0.05);
}

.progress-bar-fill {
    height: 100%;
    width: 0%;
    background: var(--primary-gradient);
    border-radius: 20px;
    transition: width 0.4s cubic-bezier(0.4, 0, 0.2, 1);
    box-shadow: 0 0 15px rgba(99, 102, 241, 0.6);
}

.steps-container {
    display: flex;
    flex-direction: column;
    gap: 16px;
}

.step-item {
    display: flex;
    align-items: center;
    gap: 16px;
    padding: 14px 20px;
    border-radius: var(--radius-sm);
    background: rgba(30, 41, 59, 0.4);
    border: 1px solid rgba(255, 255, 255, 0.05);
    transition: var(--transition);
}

.step-item.active {
    background: rgba(99, 102, 241, 0.12);
    border-color: rgba(99, 102, 241, 0.3);
}

.step-item.completed {
    background: rgba(16, 185, 129, 0.1);
    border-color: rgba(16, 185, 129, 0.25);
}

.step-spinner {
    width: 20px;
    height: 20px;
    border: 2.5px solid rgba(255, 255, 255, 0.2);
    border-top-color: var(--primary);
    border-radius: 50%;
```

```
    animation: spin 0.8s linear infinite;
}

.step-check {
    width: 20px;
    height: 20px;
    border-radius: 50%;
    background: #10b981;
    display: flex;
    align-items: center;
    justify-content: center;
    color: #ffffff;
}

.step-text {
    font-size: 0.95rem;
    font-weight: 600;
    color: var(--text-muted);
}

.step-item.active .step-text {
    color: var(--text-main);
}

.step-item.completed .step-text {
    color: #10b981;
}

.result-section {
    display: none;
    animation: fadeIn 0.5s ease-out forwards;
}

.results-grid {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 20px;
    margin-bottom: 24px;
}

.result-card {
    background: rgba(30, 41, 59, 0.5);
    border: 1px solid rgba(255, 255, 255, 0.08);
    border-radius: var(--radius-md);
    padding: 24px;
    position: relative;
    overflow: hidden;
}

.card-header-sm {
    font-size: 0.82rem;
    font-weight: 700;
    text-transform: uppercase;
    letter-spacing: 0.5px;
    color: var(--text-muted);
    margin-bottom: 12px;
    display: flex;
    align-items: center;
    justify-content: space-between;
}

.topic-badge {
    display: inline-flex;
```

```
    align-items: center;
    gap: 8px;
    padding: 10px 20px;
    border-radius: 30px;
    font-size: 1.35rem;
    font-weight: 800;
    letter-spacing: -0.3px;
}

.topic-teknologi { background: rgba(56, 189, 248, 0.15); color:
#38bdf8; border: 1px solid rgba(56, 189, 248, 0.3); }
.topic-bisnis { background: rgba(251, 191, 36, 0.15); color:
#fbbf24; border: 1px solid rgba(251, 191, 36, 0.3); }
.topic-pendidikan { background: rgba(45, 212, 191, 0.15); color:
#2dd4bf; border: 1px solid rgba(45, 212, 191, 0.3); }
.topic-kesehatan { background: rgba(74, 222, 128, 0.15); color:
#4ade80; border: 1px solid rgba(74, 222, 128, 0.3); }
.topic-olahraga { background: rgba(248, 113, 113, 0.15); color:
#f87171; border: 1px solid rgba(248, 113, 113, 0.3); }
.topic-berita { background: rgba(192, 132, 252, 0.15); color:
#c084fc; border: 1px solid rgba(192, 132, 252, 0.3); }
.topic-hiburan { background: rgba(244, 114, 182, 0.15); color:
#f472b6; border: 1px solid rgba(244, 114, 182, 0.3); }

.stat-value {
    font-size: 2.2rem;
    font-weight: 800;
    color: var(--text-main);
    line-height: 1;
    margin-bottom: 8px;
}

.mini-progress {
    width: 100%;
    height: 6px;
    background: rgba(255, 255, 255, 0.08);
    border-radius: 10px;
    overflow: hidden;
    margin-top: 12px;
}

.mini-progress-fill {
    height: 100%;
    background: var(--primary-gradient);
    border-radius: 10px;
}

.transcript-card {
    background: rgba(30, 41, 59, 0.5);
    border: 1px solid rgba(255, 255, 255, 0.08);
    border-radius: var(--radius-md);
    padding: 28px;
    margin-bottom: 28px;
}

.transcript-box {
    background: rgba(15, 23, 42, 0.6);
    border: 1px solid rgba(255, 255, 255, 0.05);
    border-radius: var(--radius-sm);
    padding: 20px;
    max-height: 220px;
    overflow-y: auto;
    color: #cbd5e1;
```

```
            font-size: 0.98rem;
            line-height: 1.7;
            white-space: pre-wrap;
        }

        .transcript-box::-webkit-scrollbar {
            width: 6px;
        }

        .transcript-box::-webkit-scrollbar-thumb {
            background: rgba(255, 255, 255, 0.15);
            border-radius: 10px;
        }

        .btn-outline {
            width: 100%;
            padding: 16px 32px;
            border-radius: var(--radius-md);
            border: 1px solid rgba(255, 255, 255, 0.15);
            background: rgba(255, 255, 255, 0.04);
            color: var(--text-main);
            font-size: 1rem;
            font-weight: 700;
            cursor: pointer;
            transition: var(--transition);
            display: flex;
            align-items: center;
            justify-content: center;
            gap: 10px;
        }

        .btn-outline:hover {
            background: rgba(255, 255, 255, 0.08);
            border-color: rgba(255, 255, 255, 0.3);
            transform: translateY(-2px);
        }

        @keyframes fadeIn {
            from { opacity: 0; transform: translateY(10px); }
            to { opacity: 1; transform: translateY(0); }
        }

        @keyframes spin {
            to { transform: rotate(360deg); }
        }

        @media (max-width: 868px) {
            .results-grid { grid-template-columns: 1fr; }
            h1 { font-size: 2.1rem; }
            .glass-card { padding: 24px; }
        }
    </style>
</head>
<body>

    <div class="ambient-glow glow-1"></div>
    <div class="ambient-glow glow-2"></div>

    <div class="app-container">

        <header>
            <div class="badge-pill">
                <span class="badge-dot"></span>
```

```
    </div>
    <h1>Podcast Topic Classification</h1>
    <p class="subtitle">AI-powered Indonesian Podcast Topic
Classification System</p>
 </header>

 <main class="glass-card">

    <!-- STATE 1: UPLOAD AREA -->
    <div id="uploadSection">
        <div class="drop-zone" id="dropZone">
            <input type="file" id="audioInput"
accept=".mp3,.wav,.m4a" style="display: none;">
            <div class="icon-wrapper">
                <svg viewBox="0 0 24 24">
                    <path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-
2-2v-4"></path>
                    <polyline points="17 8 12 3 7 8"></polyline>
                    <line x1="12" y1="3" x2="12" y2="15"></line>
                </svg>
            </div>
            <div class="drop-title">Click or Drag Audio File
Here</div>
            <div class="drop-info">Supported formats: MP3, WAV,
M4A  •  Maximum size: 200 MB</div>
        </div>

        <div class="file-preview" id="filePreview">
            <div class="file-icon">
                <svg width="24" height="24" viewBox="0 0 24 24"
fill="none" stroke="currentColor" stroke-width="2">
                    <path d="M9 18V5l12-2v13"></path>
                    <circle cx="6" cy="18" r="3"></circle>
                    <circle cx="18" cy="16" r="3"></circle>
                </svg>
            </div>
            <div class="file-details">
                <div class="file-name"
id="fileName">podcast_audio_sample.mp3</div>
                <div class="file-meta-row">
                    <span class="meta-tag">⏱ Duration: <strong
id="fileDuration">--m --s</strong></span>
                    <span class="meta-tag">📦 Size: <strong
id="fileSize">-- MB</strong></span>
                    <span class="meta-tag">🎵 Format: <strong
id="fileFormat">MP3</strong></span>
                </div>
            </div>
            <button type="button" id="removeFileBtn"
style="background:none; border:none; color:var(--text-dim);
cursor:pointer; padding:6px;">
                <svg width="20" height="20" viewBox="0 0 24 24"
fill="none" stroke="currentColor" stroke-width="2"><line x1="18"
y1="6" x2="6" y2="18"></line><line x1="6" y1="6" x2="18"
y2="18"></line></svg>
            </button>
        </div>

        <button class="btn-primary" id="startBtn" disabled>
            <svg width="20" height="20" viewBox="0 0 24 24"
fill="none" stroke="currentColor" stroke-width="2.2"><polygon
points="5 3 19 12 5 21 5 3"></polygon></svg>
```

```
            Start Classification
        </button>
    </div>

    <!-- STATE 2: PROGRESS AREA -->
    <div class="progress-section" id="progressSection">
        <div class="progress-header">
            <div class="progress-title">Analyzing Audio
Content</div>
            <div class="progress-val"
id="progressPercent">0%</div>
        </div>

        <div class="progress-bar-bg">
            <div class="progress-bar-fill"
id="progressBarFill"></div>
        </div>

        <div class="steps-container">
            <div class="step-item" id="step1">
                <div class="step-icon-box"><div class="step-
spinner"></div></div>
                <div class="step-text">Processing audio...</div>
            </div>
            <div class="step-item" id="step2">
                <div class="step-icon-box"
style="opacity:0.3;"><svg width="20" height="20" viewBox="0 0 24
24" fill="none" stroke="currentColor" stroke-width="2"><circle
cx="12" cy="12" r="10"></circle></svg></div>
                <div class="step-text">Transcribing
audio...</div>
            </div>
            <div class="step-item" id="step3">
                <div class="step-icon-box"
style="opacity:0.3;"><svg width="20" height="20" viewBox="0 0 24
24" fill="none" stroke="currentColor" stroke-width="2"><circle
cx="12" cy="12" r="10"></circle></svg></div>
                <div class="step-text">Analyzing data...</div>
            </div>
        </div>
    </div>

    <!-- STATE 3: RESULT CARDS -->
    <div class="result-section" id="resultSection">
        <div class="results-grid">

            <div class="result-card">
                <div class="card-header-sm">
                    <span>Podcast Topic</span>
                    <svg width="18" height="18" viewBox="0 0 24
24" fill="none" stroke="currentColor" stroke-width="2"><path
d="M20.59 13.41l-7.17 7.17a2 2 0 0 1-2.83 0L2 12V2h10l8.59 8.59a2
2 0 0 1 0 2.82z"></path><line x1="7" y1="7" x2="7.01"
y2="7"></line></svg>
                </div>
                <div id="topicBadgeContainer">
                    <span class="topic-badge topic-teknologi"
id="resTopic">
                        💻 Teknologi
                    </span>
                </div>
            </div>
```

```
                <div class="result-card">
                    <div class="card-header-sm">
                        <span>Confidence Score</span>
                        <svg width="18" height="18" viewBox="0 0 24
   24" fill="none" stroke="currentColor" stroke-width="2"><path
   d="M22 11.08V12a10 10 0 1 1-5.93-9.14"></path><polyline points="22
   4 12 14.01 9 11.01"></polyline></svg>
                    </div>
                    <div class="stat-value"
   id="resConfidence">0%</div>
                    <div class="mini-progress">
                        <div class="mini-progress-fill"
   id="resConfidenceBar" style="width: 0%;"></div>
                    </div>
                </div>

                <div class="result-card">
                    <div class="card-header-sm">
                        <span>Analysis Time</span>
                        <svg width="18" height="18" viewBox="0 0 24
   24" fill="none" stroke="currentColor" stroke-width="2"><circle
   cx="12" cy="12" r="10"></circle><polyline points="12 6 12 16
   14"></polyline></svg>
                    </div>
                    <div class="stat-value"
   id="resAnalysisTime">0s</div>
                    <div style="font-size:0.8rem; color:var(--text-
   muted); margin-top:4px;">STT + TF-IDF Inference</div>
                </div>

            </div>

            <div class="transcript-card">
                <div class="card-header-sm" style="margin-
   bottom:16px;">
                    <span>Transcript Content</span>
                    <button type="button" id="copyBtn"
   style="background:none; border:none; color:var(--primary); font-
   size:0.8rem; font-weight:700; cursor:pointer;">Copy Text</button>
                </div>
                <div class="transcript-box"
   id="resTranscript">...</div>
            </div>

            <button class="btn-outline" id="resetBtn">
                <svg width="18" height="18" viewBox="0 0 24 24"
   fill="none" stroke="currentColor" stroke-width="2.2"><path
   d="M21.5 2v6h-6M21.34 15.57a10 10 0 1 1-.57-8.38l5.67-
   5.67"></path></svg>
                Analyze Another Audio
            </button>
        </div>

    </main>
</div>

<script>
    const dropZone = document.getElementById('dropZone');
    const audioInput = document.getElementById('audioInput');
    const filePreview = document.getElementById('filePreview');
    const fileName = document.getElementById('fileName');
    const fileDuration = document.getElementById('fileDuration');
```

```
 const fileSize = document.getElementById('fileSize');
 const fileFormat = document.getElementById('fileFormat');
 const removeFileBtn = document.getElementById('removeFileBtn');
 const startBtn = document.getElementById('startBtn');

 const uploadSection = document.getElementById('uploadSection');
 const progressSection =
document.getElementById('progressSection');
 const resultSection = document.getElementById('resultSection');

 const progressBarFill =
document.getElementById('progressBarFill');
 const progressPercent =
document.getElementById('progressPercent');

 const step1 = document.getElementById('step1');
 const step2 = document.getElementById('step2');
 const step3 = document.getElementById('step3');

 const resTopic = document.getElementById('resTopic');
 const resConfidence = document.getElementById('resConfidence');
 const resConfidenceBar =
document.getElementById('resConfidenceBar');
 const resAnalysisTime =
document.getElementById('resAnalysisTime');
 const resTranscript = document.getElementById('resTranscript');
 const resetBtn = document.getElementById('resetBtn');
 const copyBtn = document.getElementById('copyBtn');

 let selectedFile = null;

 const TOPIC_CONFIG = {
     'teknologi': { icon: '💻', label: 'Teknologi', class: 'topic-
teknologi' },
     'bisnis': { icon: '💼', label: 'Bisnis', class: 'topic-
bisnis' },
     'pendidikan': { icon: '🎓', label: 'Pendidikan', class:
'topic-pendidikan' },
     'kesehatan': { icon: '🏥', label: 'Kesehatan', class: 'topic-
kesehatan' },
     'olahraga': { icon: '⚽', label: 'Olahraga', class: 'topic-
olahraga' },
     'berita': { icon: '📰', label: 'Berita', class: 'topic-
berita' },
     'hiburan': { icon: '🎬', label: 'Hiburan', class: 'topic-
hiburan' }
 };

 dropZone.addEventListener('click', () => audioInput.click());

 dropZone.addEventListener('dragover', (e) => {
     e.preventDefault();
     dropZone.classList.add('dragover');
 });

 dropZone.addEventListener('dragleave', () => {
     dropZone.classList.remove('dragover');
 });

 dropZone.addEventListener('drop', (e) => {
     e.preventDefault();
```

```
        dropZone.classList.remove('dragover');
        if (e.dataTransfer.files.length > 0) {
            handleFileSelect(e.dataTransfer.files[0]);
        }
    });

    audioInput.addEventListener('change', (e) => {
        if (e.target.files.length > 0) {
            handleFileSelect(e.target.files[0]);
        }
    });

    function handleFileSelect(file) {
        const ext = file.name.split('.').pop().toLowerCase();
        if (!['mp3', 'wav', 'm4a'].includes(ext)) {
            alert('Format file tidak didukung! Harap pilih file MP3,
WAV, atau M4A.');
            return;
        }
        if (file.size > 200 * 1024 * 1024) {
            alert('Ukuran file melebihi batas maksimal 200 MB!');
            return;
        }

        selectedFile = file;
        fileName.textContent = file.name;
        fileSize.textContent = (file.size / (1024 * 1024)).toFixed(1)
+ ' MB';
        fileFormat.textContent = ext.toUpperCase();

        const audioObj = new Audio();
        audioObj.src = URL.createObjectURL(file);
        audioObj.onloadedmetadata = () => {
            const mins = Math.floor(audioObj.duration / 60);
            const secs = Math.floor(audioObj.duration % 60);
            fileDuration.textContent = `${mins}m ${secs}s`;
        };
        audioObj.onerror = () => {
            fileDuration.textContent = '--m --s';
        };

        filePreview.style.display = 'flex';
        startBtn.disabled = false;
    }

    removeFileBtn.addEventListener('click', (e) => {
        e.stopPropagation();
        selectedFile = null;
        audioInput.value = '';
        filePreview.style.display = 'none';
        startBtn.disabled = true;
    });

    startBtn.addEventListener('click', () => {
        if (!selectedFile) return;

        uploadSection.style.display = 'none';
        progressSection.style.display = 'block';

        runPipeline();
    });

    function runPipeline() {
```

```
      let progress = 0;
      progressBarFill.style.width = '0%';
      progressPercent.textContent = '0%';

      setStepState(step1, 'active');
      setStepState(step2, 'pending');
      setStepState(step3, 'pending');

      const formData = new FormData();
      formData.append('audio', selectedFile);

      const interval = setInterval(() => {
          if (progress < 90) progress += 1;
          progressBarFill.style.width = progress + '%';
          progressPercent.textContent = progress + '%';

          if (progress >= 30 && progress < 70) {
              setStepState(step1, 'completed');
              setStepState(step2, 'active');
          } else if (progress >= 70) {
              setStepState(step2, 'completed');
              setStepState(step3, 'active');
          }
      }, 300);

      fetch('/predict', {
          method: 'POST',
          body: formData
      })
      .then(res => res.json())
      .then(data => {
          clearInterval(interval);
          progressBarFill.style.width = '100%';
          progressPercent.textContent = '100%';
          setStepState(step3, 'completed');
          setTimeout(() => showRealResults(data), 400);
      })
      .catch(err => {
          clearInterval(interval);
          alert('Eror Jaringan / Koneksi Terputus: ' +
err.message);
          resetBtn.click();
      });
  }

  function showRealResults(data) {
      if (data.error) {
          alert('Eror: ' + data.error);
          resetBtn.click();
          return;
      }

      const predTopic = (data.prediction ||
'berita').toLowerCase();
      const cfg = TOPIC_CONFIG[predTopic] ||
TOPIC_CONFIG['berita'];

      resTopic.className = 'topic-badge ' + cfg.class;
      resTopic.innerHTML = `${cfg.icon} ${cfg.label}`;

      const confVal = data.confidence ?
parseFloat(data.confidence).toFixed(1) : '0';
      resConfidence.textContent = confVal + '%';
```

```
            resConfidenceBar.style.width = confVal + '%';

            const sttSecs = data.stt_time_sec ?
        parseFloat(data.stt_time_sec).toFixed(2) : '0';
            resAnalysisTime.textContent = sttSecs + 's';

            resTranscript.textContent = data.transcript || 'Transkripsi
        audio tidak tersedia.';

            progressSection.style.display = 'none';
            resultSection.style.display = 'block';
          }

          function setStepState(stepEl, state) {
            const iconBox = stepEl.querySelector('.step-icon-box');
            stepEl.className = 'step-item ' + state;

            if (state === 'active') {
                iconBox.innerHTML = '<div class="step-spinner"></div>';
                iconBox.style.opacity = '1';
            } else if (state === 'completed') {
                iconBox.innerHTML = '<div class="step-check"><svg
        width="12" height="12" viewBox="0 0 24 24" fill="none"
        stroke="currentColor" stroke-width="3"><polyline points="20 6 9 17
        4 12"></polyline></svg></div>';
                iconBox.style.opacity = '1';
            } else {
                iconBox.innerHTML = '<svg width="20" height="20"
        viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
        width="2"><circle cx="12" cy="12" r="10"></circle></svg>';
                iconBox.style.opacity = '0.3';
            }
          }

          resetBtn.addEventListener('click', () => {
            resultSection.style.display = 'none';
            uploadSection.style.display = 'block';

            selectedFile = null;
            audioInput.value = '';
            filePreview.style.display = 'none';
            startBtn.disabled = true;
          });

          copyBtn.addEventListener('click', () => {

        navigator.clipboard.writeText(resTranscript.textContent.trim()).th
        en(() => {
                copyBtn.textContent = 'Copied!';
                setTimeout(() => copyBtn.textContent = 'Copy Text',
        2000);
            });
          });
        </script>
</body>
</html>'''

with open(f'{WEBAPP_DIR}/templates/index.html', 'w', encoding='utf-8') as
        f:
    f.write(html_code.strip())

print("☑ index.html (Klasifikasi Topik Podcast 200 MB) berhasil dibuat
```

```
       di Google Colab!")

# ========================================
# CELL 24: Jalankan Flask Server & Hubungkan Tunnel Ngrok
# ========================================
#@markdown ### 🔑 Konfigurasi Ngrok Authtoken
NGROK_TOKEN = "3HicvlptRwwrdhTYXEJCTaMF3Hi_BZF2LrZJDHgdw7GzjpZF" #@param
       {type:"string"}

!pip install flask-ngrok pyngrok -q
from pyngrok import ngrok, conf
import os, sys, time, threading, importlib

# ── 1. BERSIHKAN KONEKSI NGROK LAMA & UNLOAD MODULE CACHE
       ──────────────────────

try:
    for t in ngrok.get_tunnels():
        ngrok.disconnect(t.public_url)
    ngrok.kill()
    time.sleep(1)
    print("[OK] Sesi Ngrok lama berhasil dibersihkan.")
except Exception:
    pass

# Hapus cache module 'app' dari memori Python Colab agar selalu membaca
       model terbaru
if 'app' in sys.modules:
    del sys.modules['app']

# ── 2. DETEKSI PATH OTOMATIS
       ──────────────────────────────────────────────────

if 'BASE_DIR' not in globals():
    BASE_DIR = '/content/drive/MyDrive/PodcastClassifier'

if 'WEBAPP_DIR' not in globals():
    WEBAPP_DIR = f'{BASE_DIR}/webapp'

conf.get_default().auth_token = NGROK_TOKEN

# 3. Pindah ke direktori webapp & reload module app secara paksa
os.chdir(WEBAPP_DIR)
if WEBAPP_DIR not in sys.path:
    sys.path.append(WEBAPP_DIR)

print("⏳ Sedang memuat model Whisper & NLP...")
try:
    import app as app_module
    importlib.reload(app_module)
    flask_app = app_module.app
except Exception as e:
    print(f"\n❌ Gagal memuat aplikasi Flask / Model: {e}")
    flask_app = None

if flask_app is not None:
    print("\n Memulai server Flask di latar belakang...")
    def run_server():
        flask_app.run(host="0.0.0.0", port=5000, debug=False,
       use_reloader=False)

    server_thread = threading.Thread(target=run_server, daemon=True)
    server_thread.start()
    time.sleep(2)
```

```
# 4. Hubungkan Tunnel Ngrok Baru secara Aman
try:
    public_url = ngrok.connect(5000)
    print(f"\n APLIKASI WEB BERHASIL DIDEPLOY SECARA ONLINE!")
    print(f" Tautan Akses Web Publik Anda:")
    print(f"   {public_url}")
    print(f"\nTekan tombol Stop pada cell ini untuk menghentikan
   server.")
except Exception as e:
    print(f"\nGagal membuka tunnel Ngrok: {e}")
```